

Computer Vision

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

Student Scientific Communications Session

Master și Doctorat ▾ 			
Partener	Temă	Domeniu Eligibil	Detalii
Adobe	Zero-Shot and Few-Shot Generalization for Novel Image Editing Tasks using Aligned Foundation Models	Informatică sau CTI	 Zero-Shot and Few-Shot Generalization for Novel Image Editing Tasks using Aligned Foundation Models.pdf
Adobe	Fine-Grained Image Interactive Editing using Structured Multimodal Prompts	Informatică sau CTI	 Fine-Grained Image Interactive Editing using Structured Multimodal Prompts.pdf
Adobe	Top-Down Decomposition of AI-Generated Visual Designs into Editable, Layered Representations Using Vision-Language Models	Informatică sau CTI	 Top-Down Decomposition of AI-Generated Visual Designs into Editable, Layered Representations Using Vision-Language Models.pdf
Adobe	Personalized Image Editing via Memory-Augmented Diffusion Models	Informatică sau CTI	 Personalized Image Editing via Memory-Augmented Diffusion Models.pdf
ppc	Explainable AI in Market Prediction: Causal and Regime-Aware Explanations	Matematică, Informatică sau CTI	 Explainable AI in Market Prediction: Causal and Regime-Aware Explanations.pdf

Details about proposed themes: <https://tinyurl.com/TemeFirmeSCSS>

Competition Registration: <https://tinyurl.com/InscriereFirmeSCSS>

Student Scientific Communications Session

<https://fmi.unibuc.ro/sesiune-de-comunicari-stiintifice-studentesti/>

Premii:

Banii vor fi distribuiți în mod egal între membrii echipei câștigătoare.

- **Pentru secțiunile cu teme de la firme:**

- Premiul I: 2000 RON
- Premiul II: 1500 RON
- Premiul III: 1000 RON

- **Pentru secțiunile cu temă la liberă alegere:**

- Premiul I: 1000 RON
- Premiul II: 750 RON
- Premiul III: 500 RON
- Mențiuni: 300 RON

Course structure

1. Low-level vision: Features and filters

Linear filters, color, texture, edge detection, template matching

2. Mid-level vision: Grouping and fitting

Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. Mid-level vision: Multiple views

Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. High-level vision: Object recognition

Object classification, object detection, part based models, bovw models

5. High-level vision: Video understanding

Object tracking, background subtraction, motion descriptors, optical flow

Texture

Images can naturally be decomposed in:

pixel → edge → texton → superpixel → part → object

1.Pixel

- the smallest unit of an image, a single point with a specific color (or intensity in grayscale).
- each pixel doesn't carry much information, it's only when grouping pixels together meaningful structures emerge.

2.Edge

- an edge is a boundary where there is a sharp change in color or intensity.
- by detecting edges, we start to see outlines and boundaries of shapes, the first step in for higher-level features.

3.Texton

- a “texton” is a small, fundamental visual pattern (like a local texture element) which often repeats in a region. For example, the basic brick shape in a brick wall, or a small patch of repeated grain in wood texture.
- textons capture local texture statistic, recognizing them help moving beyond just raw edges to textures/patterns.

4.Superpixel

- a cluster of pixels (often hundreds or thousands) that share similar color or texture. Instead of looking at each individual pixel, you group them into a larger, more meaningful “patch.”
- superpixels simplify the image representation. Each superpixel represents a region that's uniform in some way (color, texture, brightness), which makes further processing more efficient and structured.

5.Part

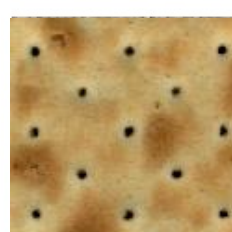
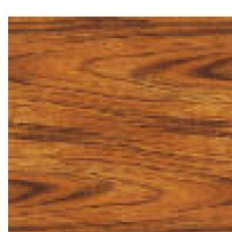
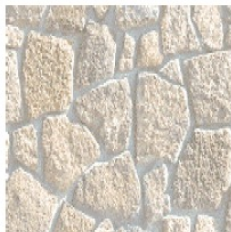
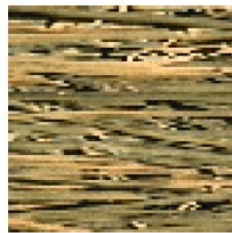
- a significant portion of an object. For instance, if you have a face, “parts” would be the eyes, nose, or mouth; if you have a car, “parts” might be the wheels, doors, or windshield.
- once you group superpixels into larger, functionally relevant segments, these start to represent meaningful “parts” of an object. Parts are closer to what we humans intuitively recognize.

6.Object

- the complete, recognizable entity in the scene (e.g., a person, a car, a tree).
- combining all the parts (which are themselves made up of superpixels, which in turn come from pixels) yields the full, identifiable object.

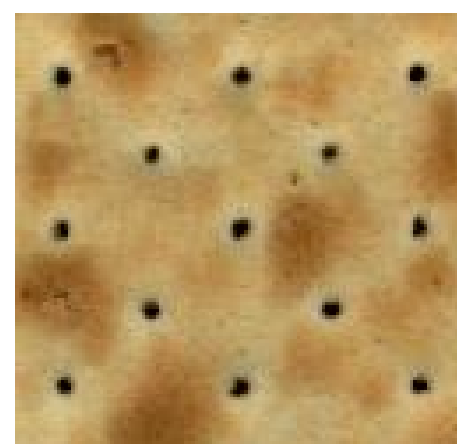
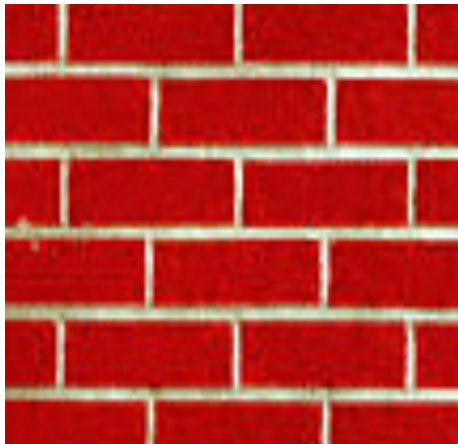
Why do we care about texture?

- Texture is a powerful visual cue for recognizing materials and surfaces.
- It often helps distinguish objects even when their shape is similar.
- Texture is useful for:
 - material recognition (wood, metal, fabric)
 - segmentation (grouping similar regions)
 - scene understanding (grass, road, sky)
 - land cover classification (satellite images)
 - tissue classification (medical imaging)



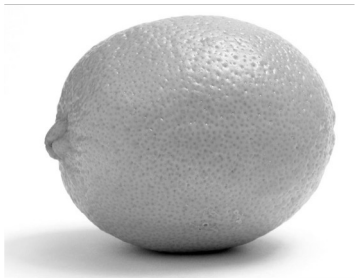
What is texture?

- Texture = spatial arrangement of local intensity/color variations
- It arises from repeated or statistically similar patterns. It is characterized by:
 - scale (fine $\leftrightarrow$ coarse)
 - orientation
 - regularity vs randomness (regular $\leftrightarrow$ irregular)



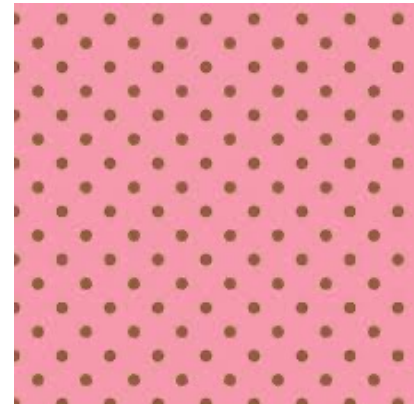
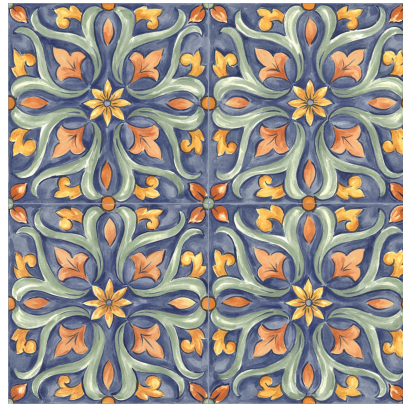
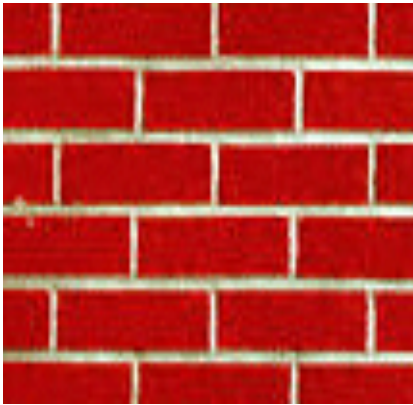
Texture vs shape

- Texture $\neq$ shape $\rightarrow$ it describes surface appearance
- Shape describes the global geometry of an object
- Texture describes repeated local variations in intensity or color.
- Two objects may have similar shape but different texture.
- Conversely, very different objects may share similar texture statistics.



Texels / textons

- A texel (or texton) is a basic local element that seems to repeat in a texture.
- In very regular textures, texels are easy to imagine:
brick wall → one brick, tiled floor → one tile, polka dots → one dot.



- In natural textures, texels are less obvious: grass, leaves, sand



For many real images, texels are often not explicitly identifiable. Therefore, we describe texture statistically instead of explicitly detecting texels.

A major challenge: scale

- Texture depends on scale
 - The same surface looks different at different scales
 - Fine scale → detailed patterns (leaf)
 - Medium scale → structure (tree)
 - Coarse scale → global texture (forest)
- There is no single correct scale for texture
- Good descriptors must capture multiple scales/orientations



Leaf → fine-scale texture



Tree → structural level

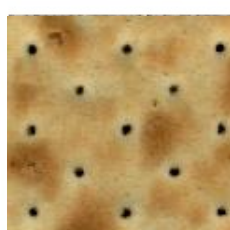
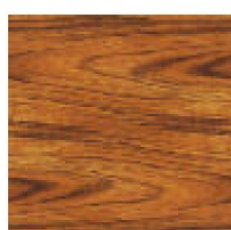
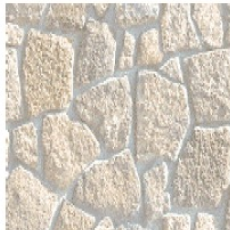
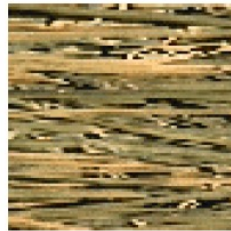


Forest → statistical texture level

Texture is not a property of the object alone, but of the observation scale.

Goal: represent texture numerically

- We need features that capture:
 - local patterns
 - distribution of structures
 - multi-scale information
 - orientation



Two main ways to describe texture

1. Statistical approach: describe texture using numerical measurements computed from the image:

- edge density
- local binary patterns
- co-occurrence statistics
- histograms of responses

2. Filter-based approach: analyze the image with a bank of filters sensitive to:

- orientation
- scale
- frequency
- spots / bars / edges

Simple statistical measures

Segmenting out texels is difficult in real images.

A highly textured region usually produces:

- more edges
- stronger gradients
- richer orientation distributions



A first idea is to use edge-based statistics.

- edge density: how many edge pixels exist in a fixed-sized region with N pixels

$$\mathbf{F}_{\text{edgeness}} = |\{ \mathbf{p} \mid \text{gradient_magnitude}(\mathbf{p}) \geq \text{threshold} \}| / \mathbf{N}$$

- gradient magnitude histogram or gradient direction histogram (both normalized)

$$\mathbf{F}_{\text{magdir}} = (\mathbf{H}_{\text{magnitude}}, \mathbf{H}_{\text{direction}})$$

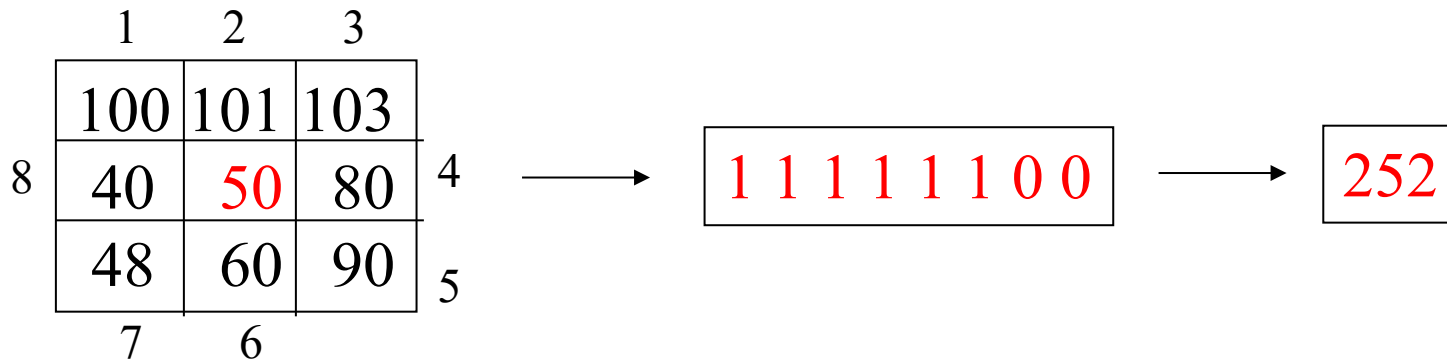
These features are simple, intuitive, and computationally efficient.

Local Binary Patterns

- LBP is a simple but powerful local texture descriptor.

- For each pixel:

- compare the center pixel with its neighbors,
- write 1 if a neighbor is larger, 0 otherwise,
- obtain a binary code.



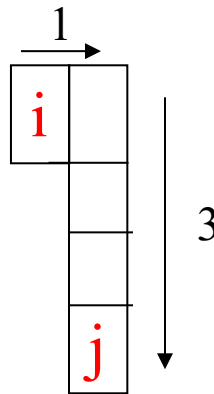
- Represent the texture by the histogram of LBP codes over a region.
- LBP captures local micro-patterns such as: flat regions, edges, corners, small spots.

Gray-Level Co-occurrence Matrix (GLCM)

- The co-occurrence matrix C_d counts how often two intensity values appear in a given spatial relationship.
- It depends on a displacement vector $d = (\Delta r, \Delta c)$.
- Entry $C_d(i, j)$ counts how often a pixel of value i occurs together with a neighboring pixel of value j at offset d .

1	1	0	0
1	1	0	0
0	0	2	2
0	0	2	2
0	0	2	2
0	0	2	2

grayscale image



$d = (3, 1)$

	0	1	2
0	1	0	3
1	2	0	2
2	0	0	1

C_d

co-occurrence matrix

- Instead of looking only at individual pixel values, GLCM models pairwise spatial structure.

Features extracted from the GLCM

From C_d we can compute N_d , the normalized co-occurrence matrix, where each value is divided by the sum of all the values. From the normalized co-occurrence matrix, we can derive features such as:

- Energy – measures uniformity,
- Entropy – measures randomness,
- Contrast – measures intensity variation,
- Homogeneity – measures how concentrated values are near the diagonal,
- Correlation – measures dependency between neighboring intensities.

$$Energy = \sum_i \sum_j N_d^2(i, j)$$

$$Entropy = - \sum_i \sum_j N_d(i, j) \log_2 N_d(i, j)$$

$$Contrast = \sum_i \sum_j (i - j)^2 N_d(i, j)$$

$$Homogeneity = \sum_i \sum_j \frac{N_d(i, j)}{1 + |i - j|}$$

$$Correlation = \frac{\sum_i \sum_j (i - \mu_i)(j - \mu_j) N_d(i, j)}{\sigma_i \sigma_j}$$

where μ_i, μ_j are the means and σ_i, σ_j are the standard deviations of the row and column

The result depends on the choice of offset d , therefore scale and direction matter.

But how do you choose d ?

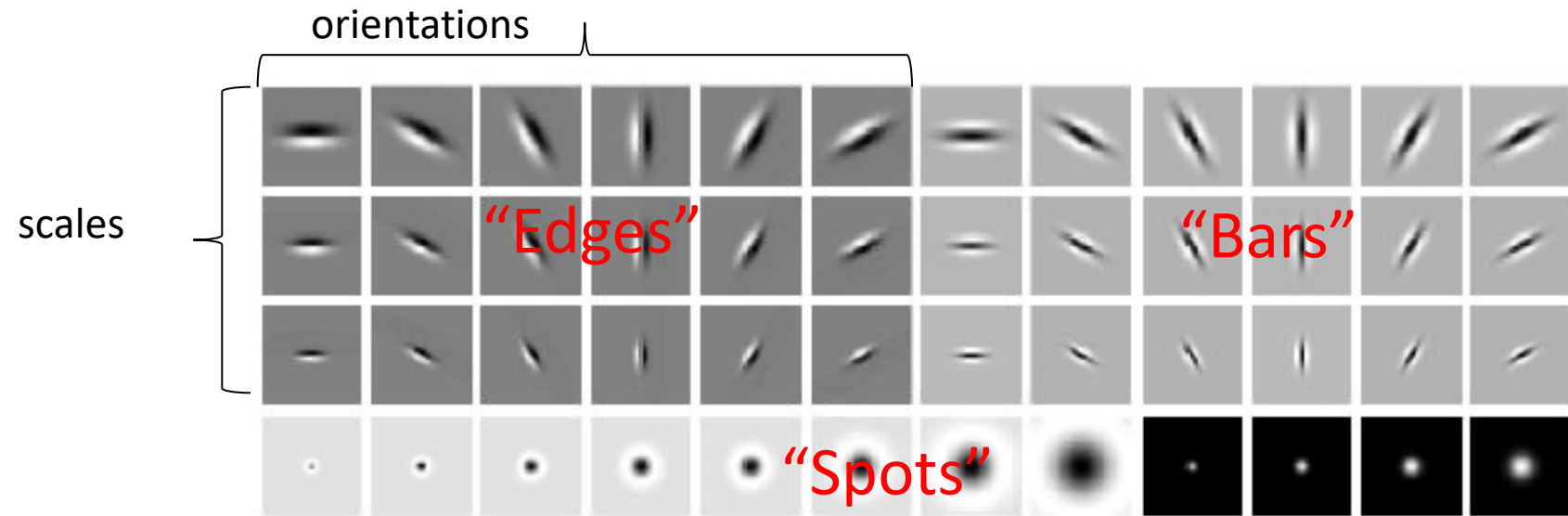
From C_d we can compute N_d , the normalized co-occurrence matrix, where each value is divided by the sum of all the values. From the normalized co-occurrence matrix, we can derive several features. The result depends on the choice of offset d , therefore scale and direction matter.

- How to choose d ?
- This is actually a critical question with **all** the statistical texture methods.
- Are the “texels” tiny, medium, large, all three ...?
- Not really a solved problem.

Filter banks for texture

- Another powerful idea: analyze a texture through the responses of a set of filters.
- A filter bank usually contains filters at multiple:
 - scales,
 - orientations,
 - pattern types (edges, bars, spots)
- Each filter responds strongly to a specific kind of local structure.
- For each image region, we can summarize filter responses using: mean, standard deviation, histograms, pooled absolute responses.

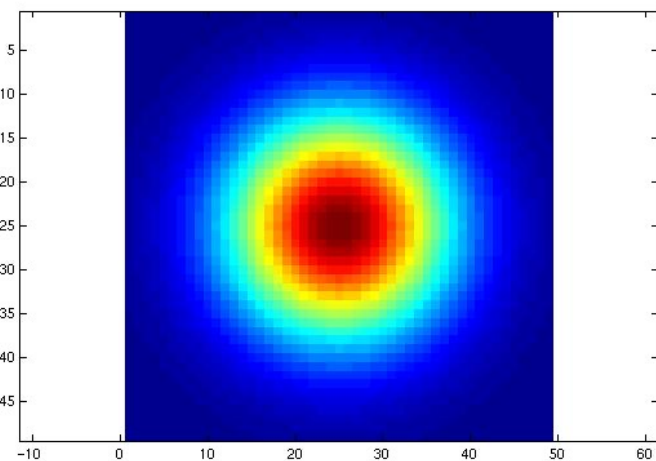
Filter banks



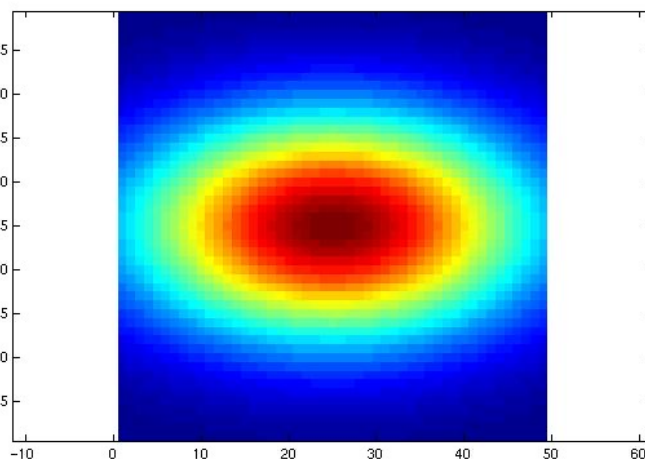
- A filter bank usually contains filters at multiple:
 - scales,
 - orientations,
 - pattern types (edges, bars, spots)

Multivariate Gaussian

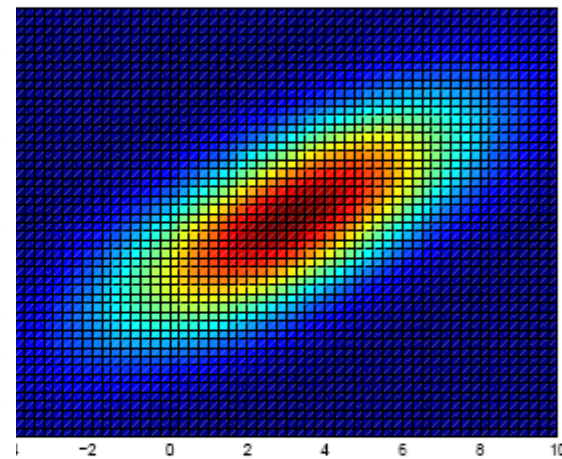
$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$



$$\Sigma = \begin{bmatrix} 9 & 0 \\ 0 & 9 \end{bmatrix}$$

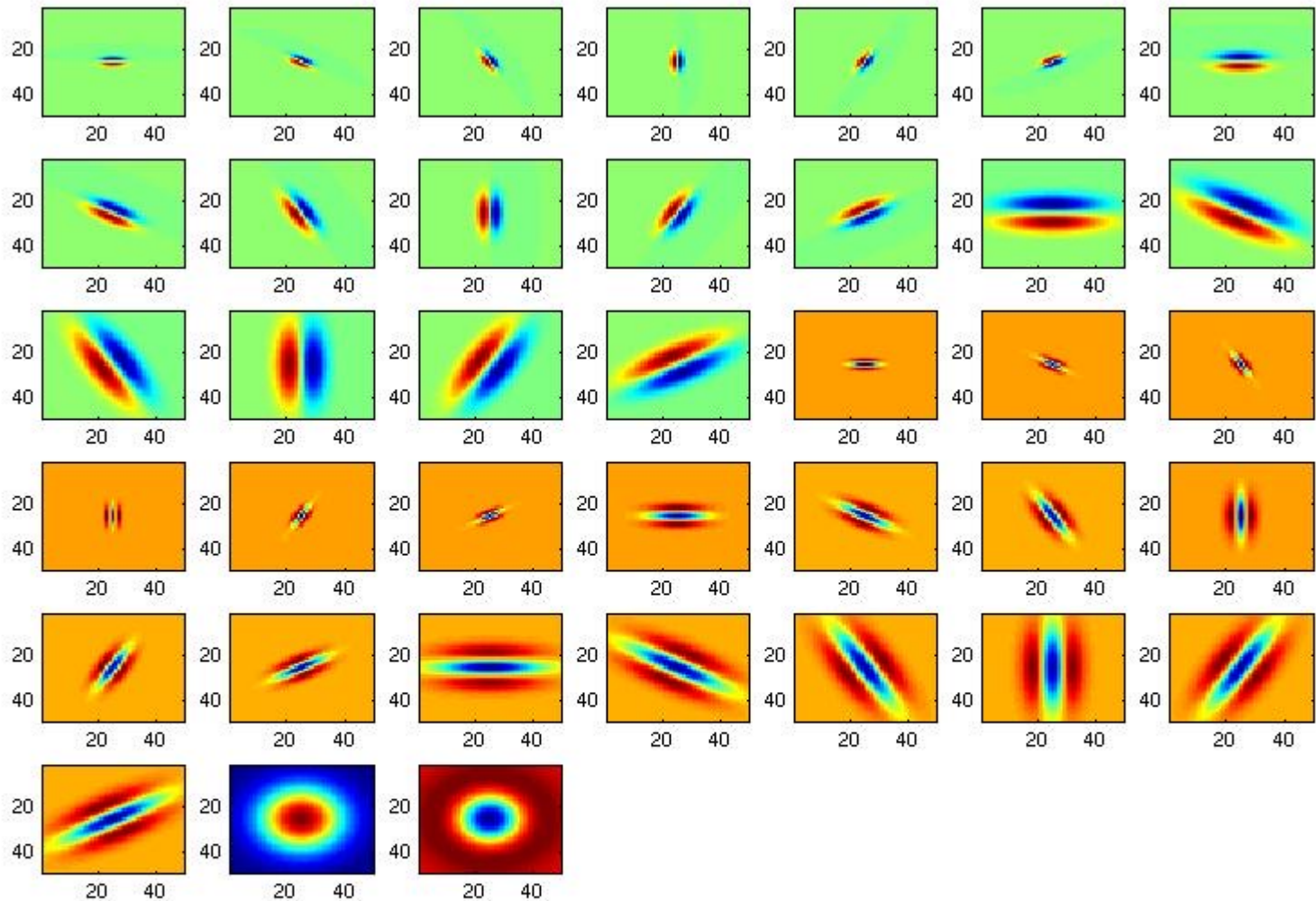


$$\Sigma = \begin{bmatrix} 16 & 0 \\ 0 & 9 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 10 & 5 \\ 5 & 5 \end{bmatrix}$$

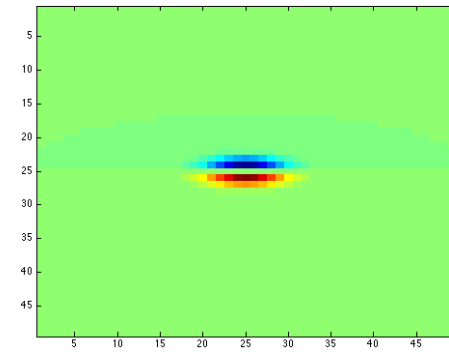
Filter bank



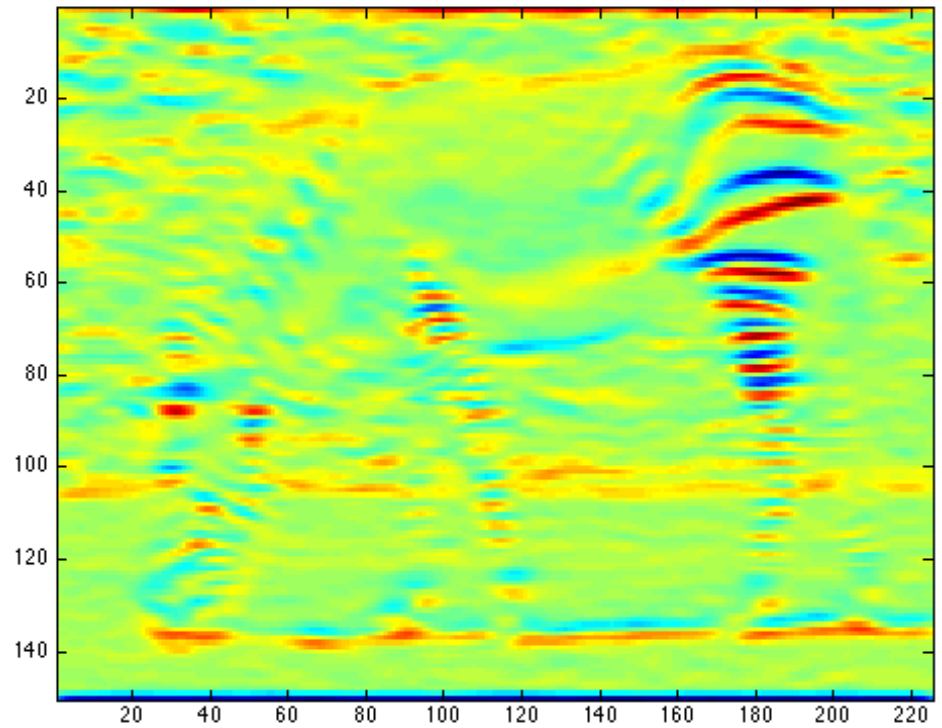
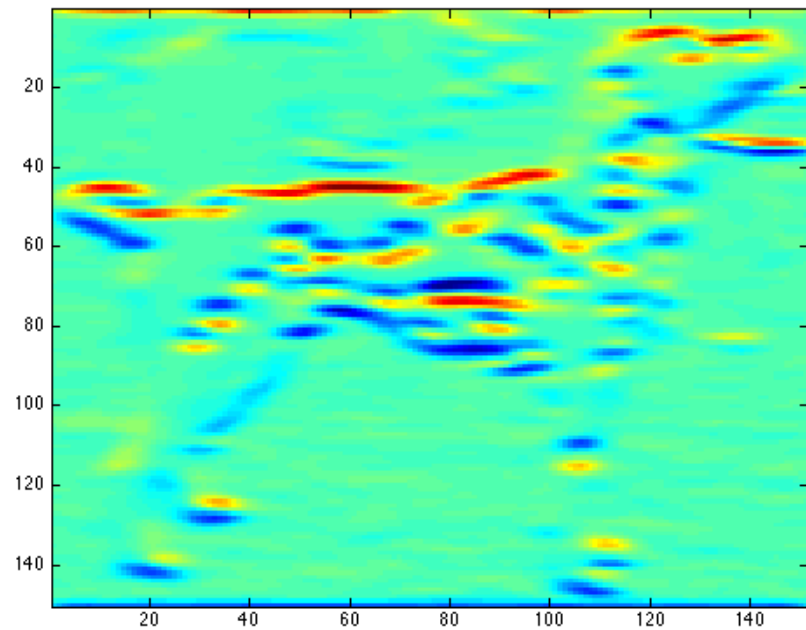
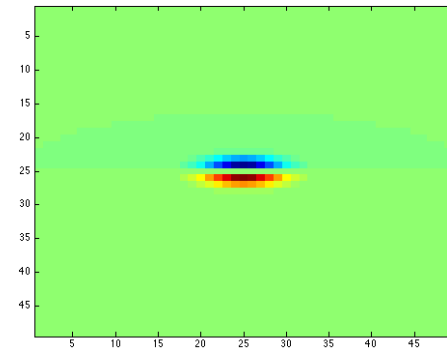
Initial images



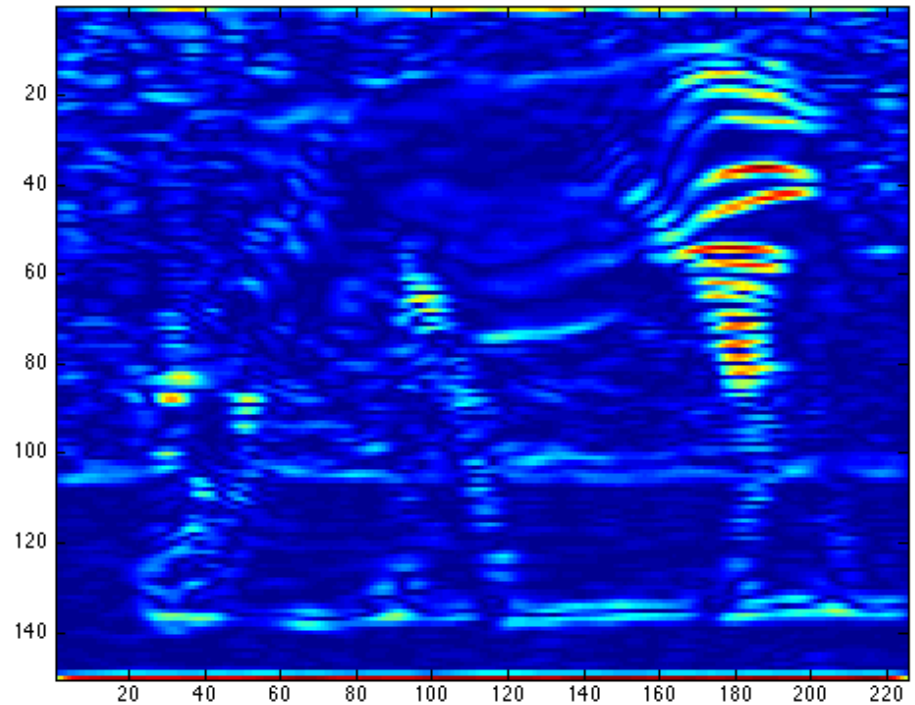
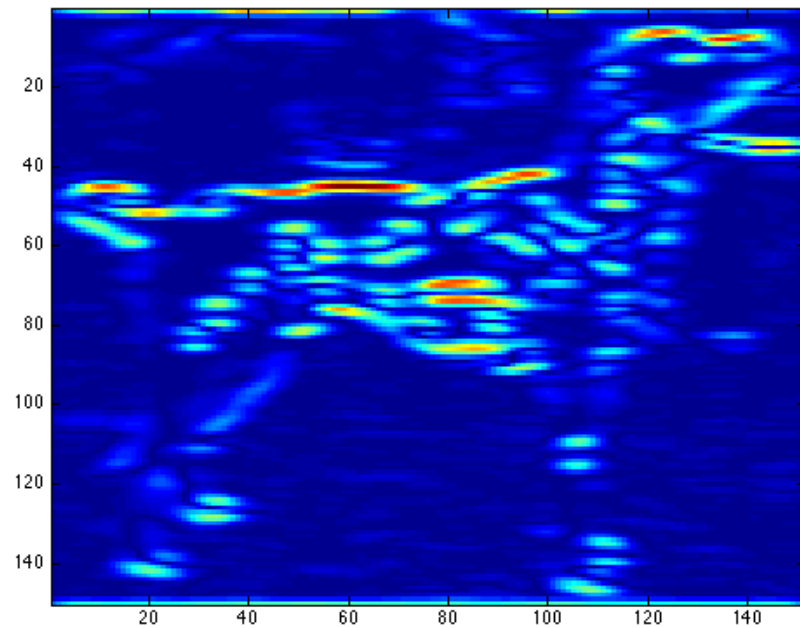
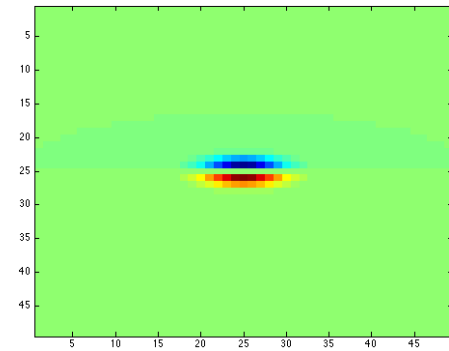
Initial images + filter



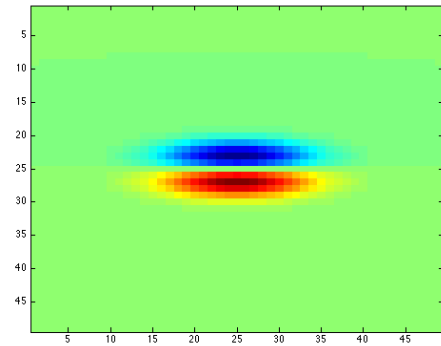
Filtered images



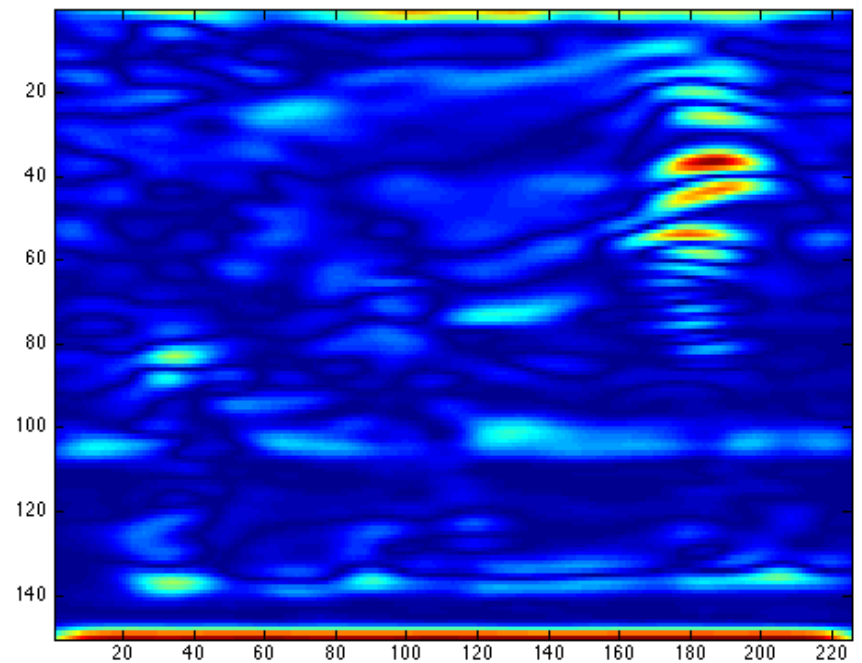
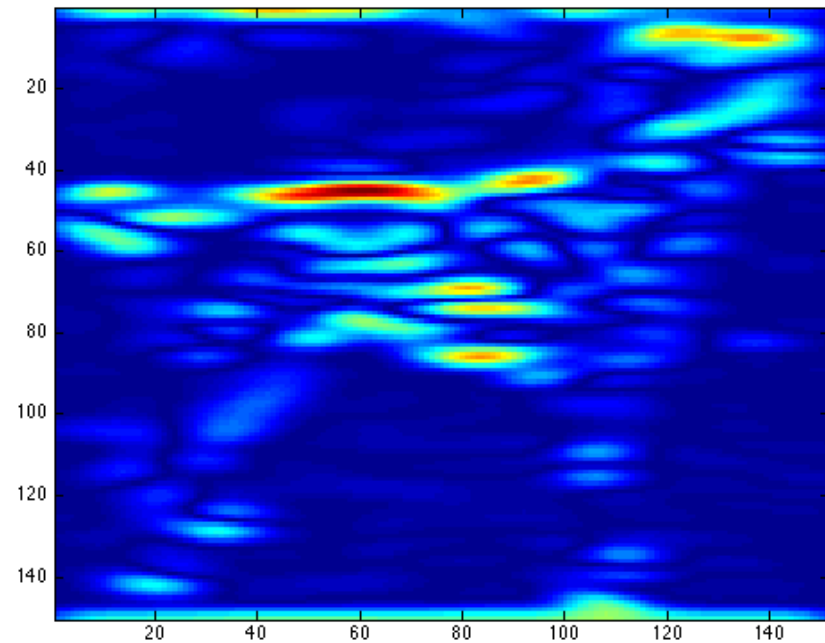
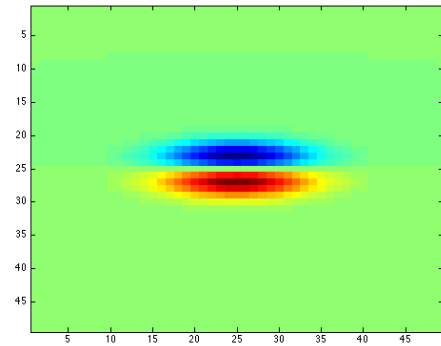
Filtered images with absolute response



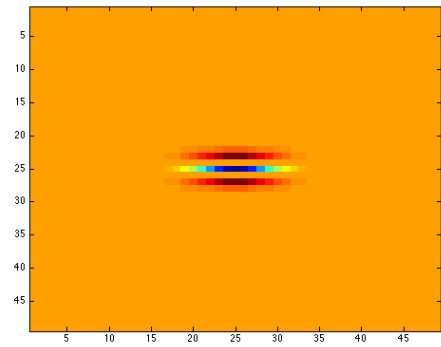
Initial images + filter



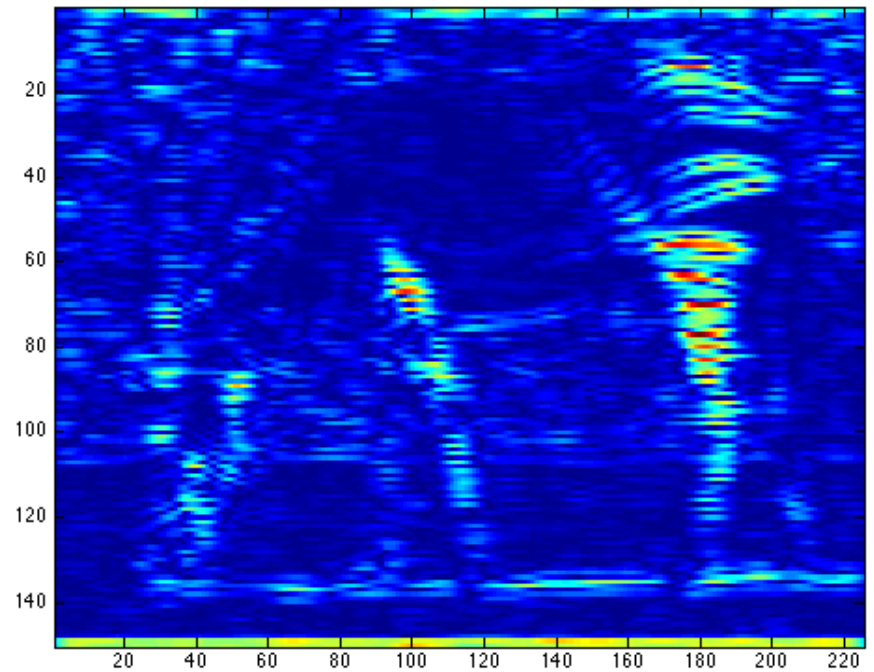
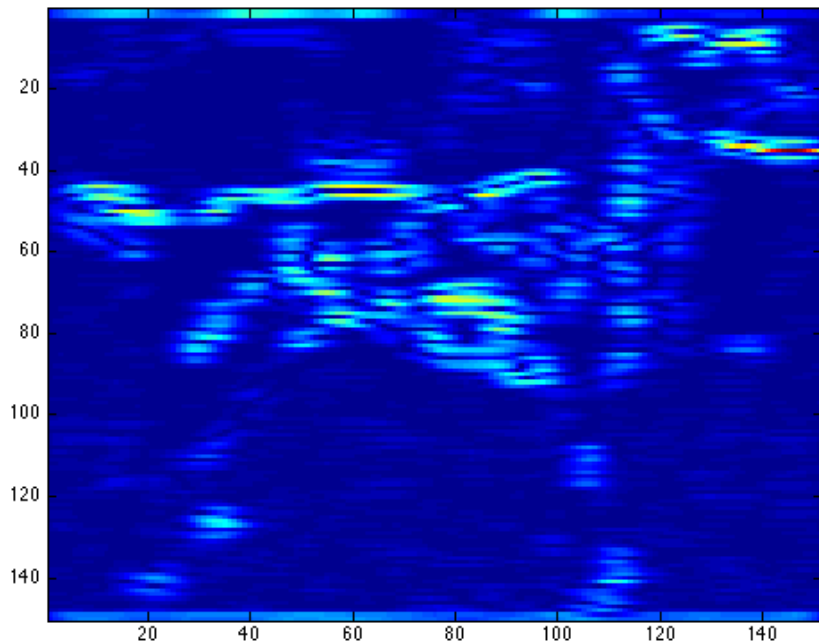
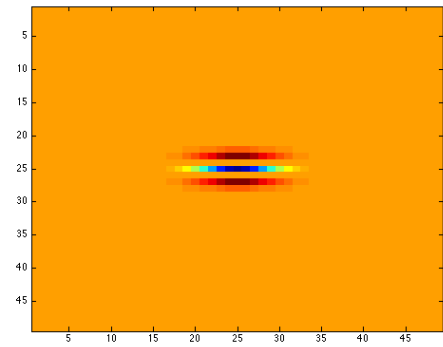
Filtered images with absolute response



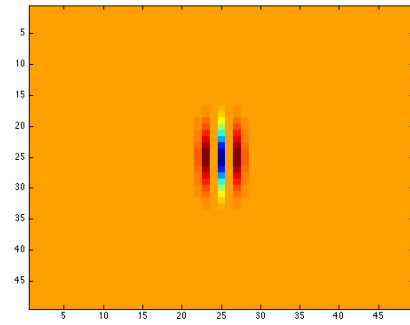
Initial images + filter



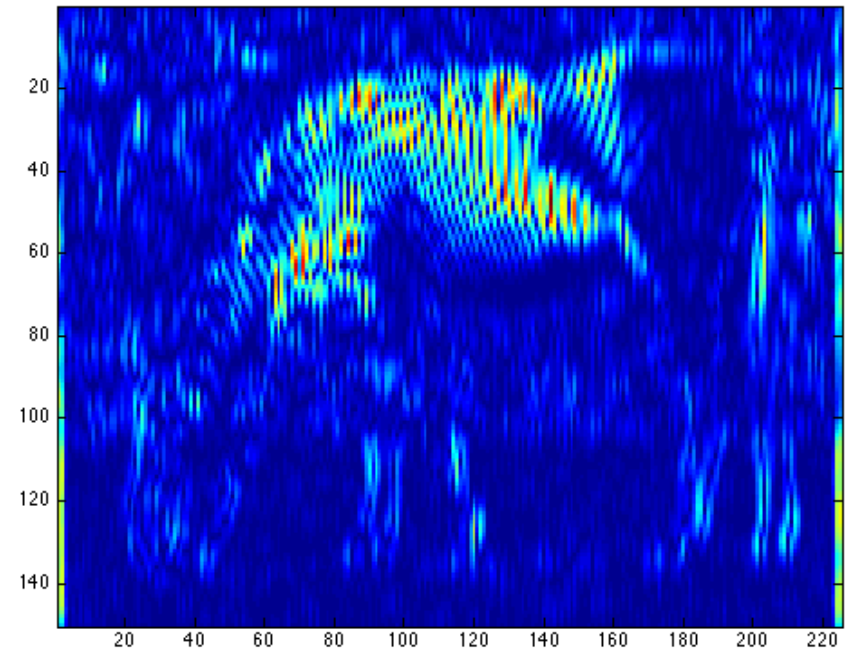
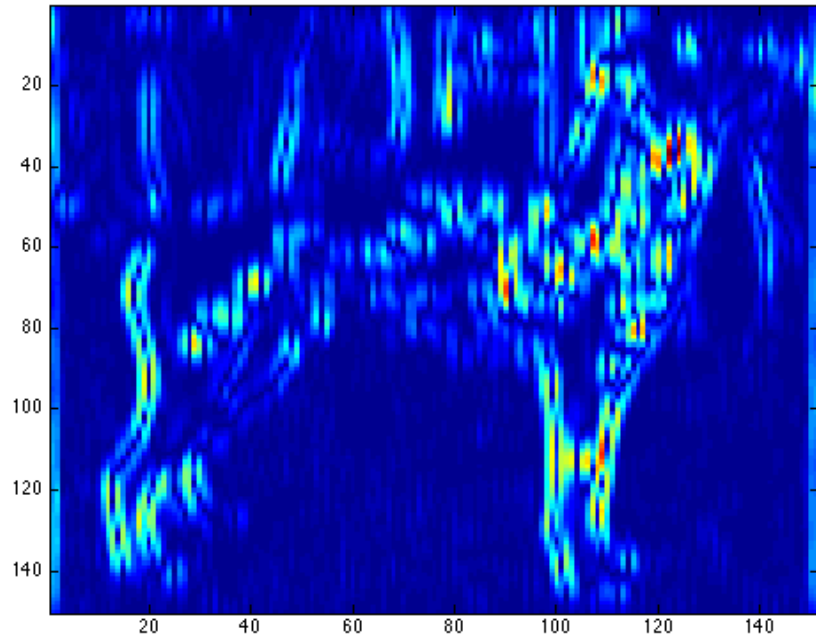
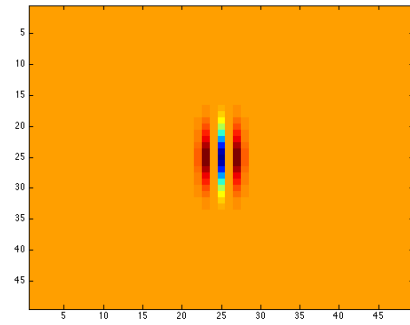
Filtered images with absolute response

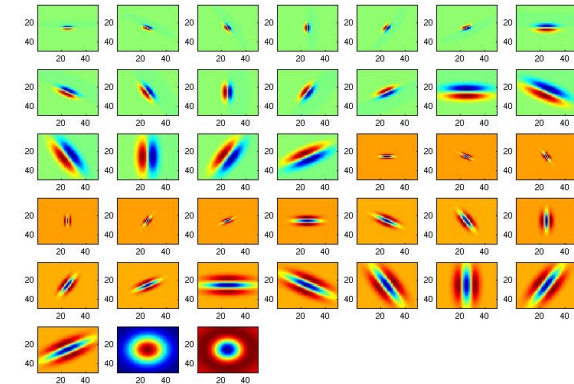
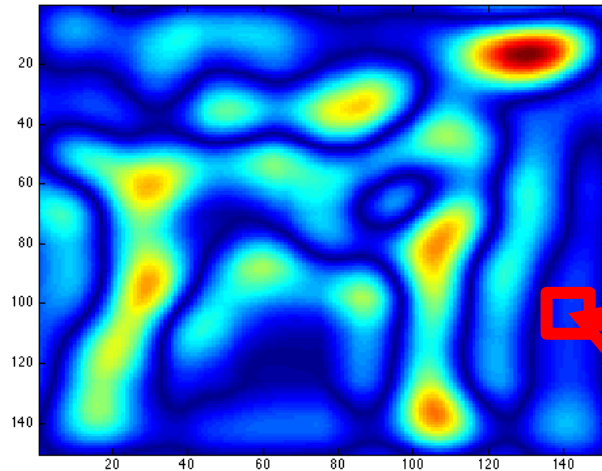


Initial images + filter



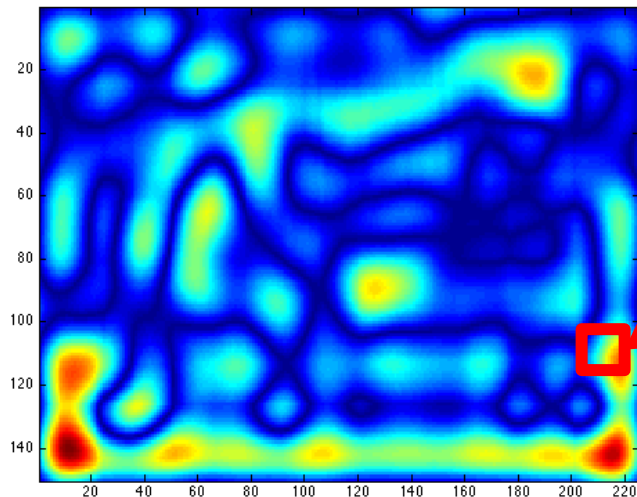
Filtered images with absolute response





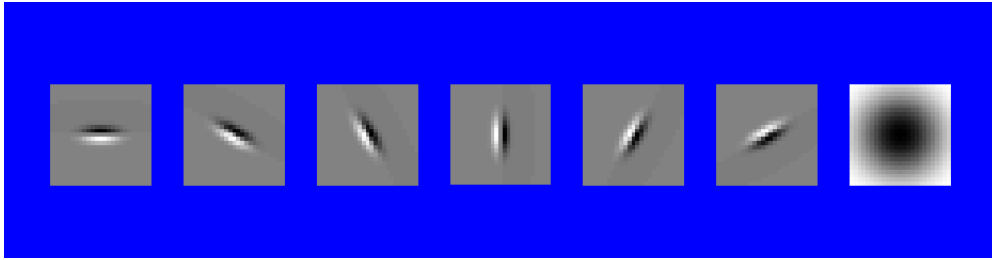
$[r_1, r_2, \dots, r_{38}]$

We can form a
feature vector
from the list of
responses at each
pixel.

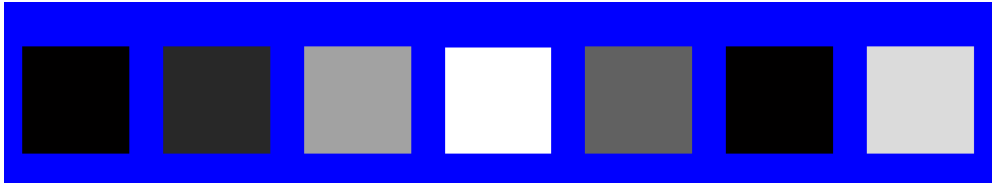


You try: Can you match the texture to the response?

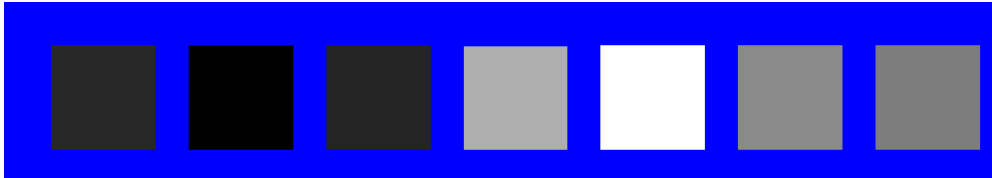
Filters



1



2

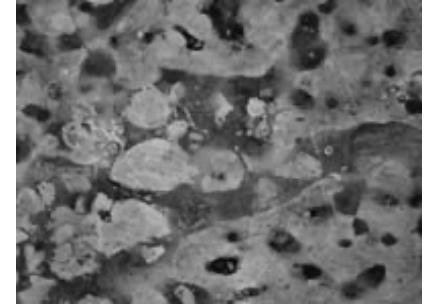


3



Mean abs responses

A



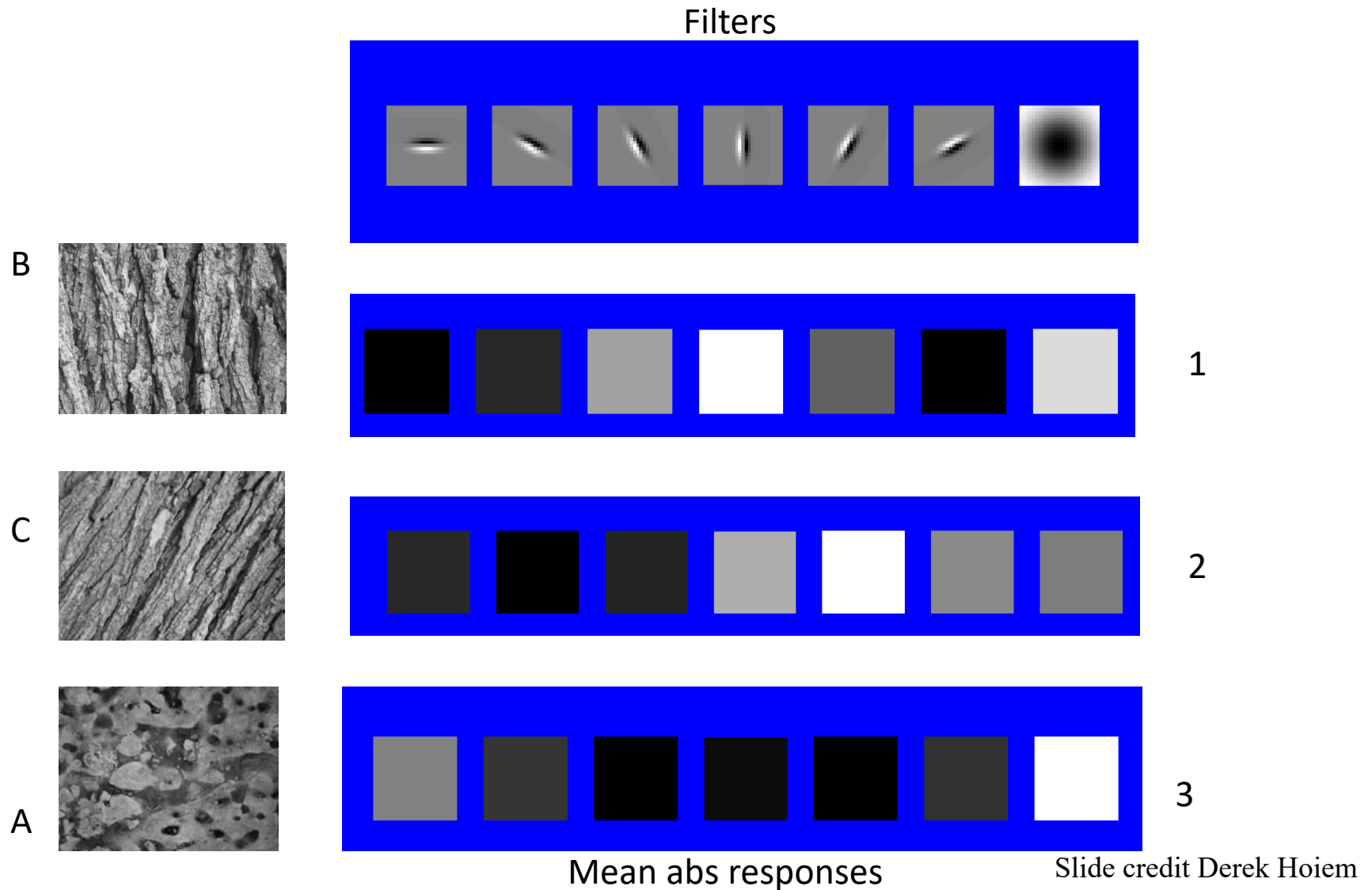
B



C

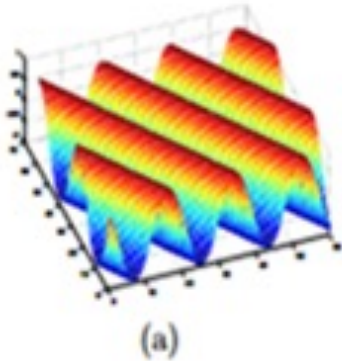


Representing texture by mean abs response

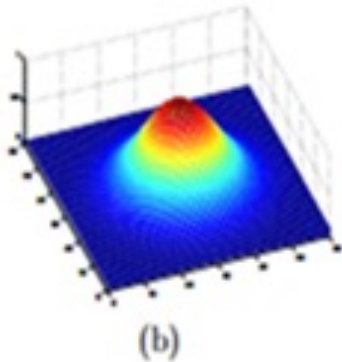


Gabor Filters

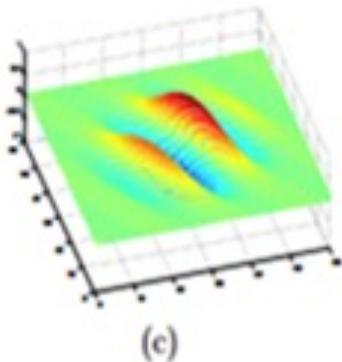
A Gabor filter is a combination of a Gaussian filter and a sinusoidal term.



A Sinusoid oriented 30° with X-axis



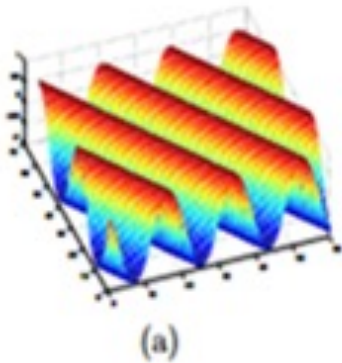
A 2-D Gaussian



The corresponding 2-D Gabor filter

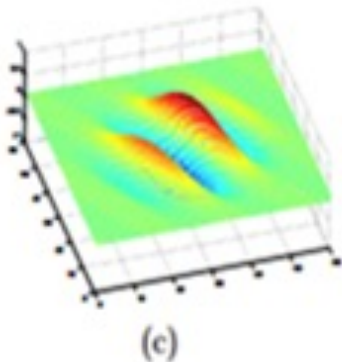
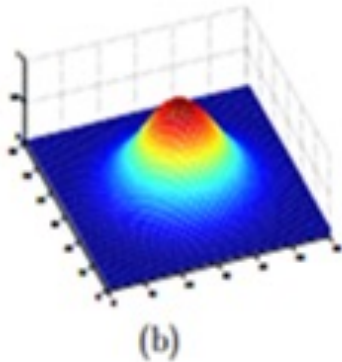
Gabor Filters

A Gabor filter is a combination of a Gaussian filter and a sinusoidal term.



Gabor filter = wave $\times$ Gaussian

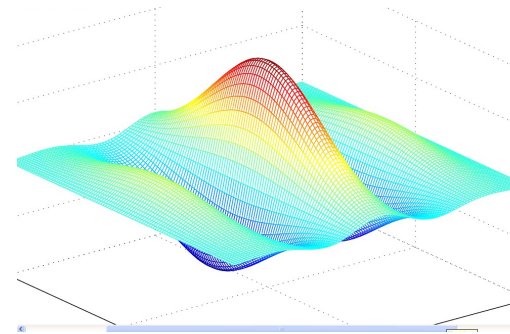
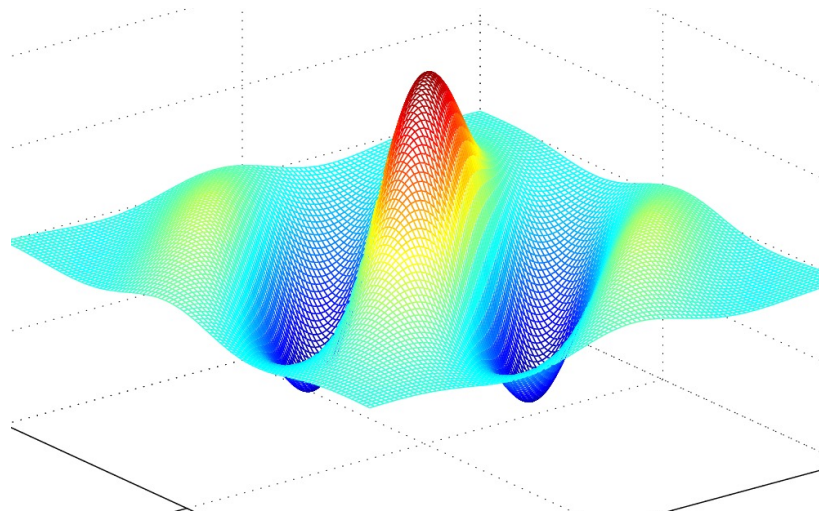
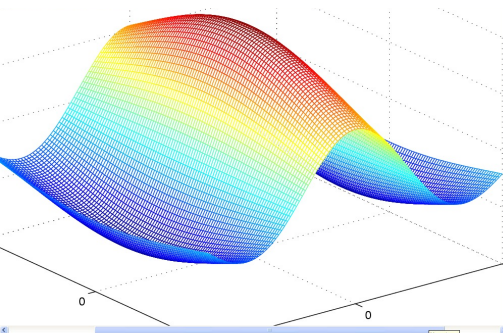
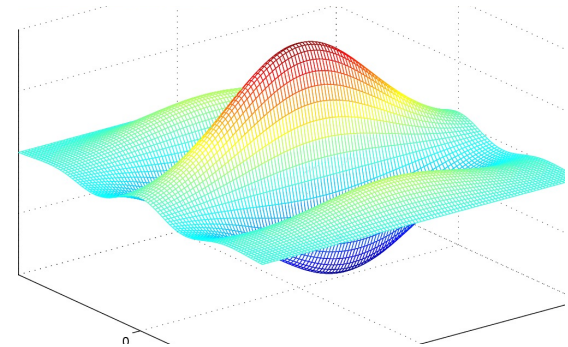
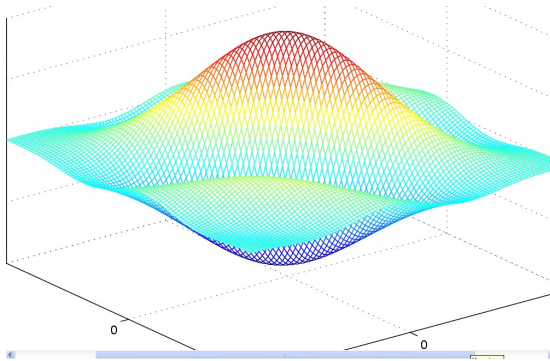
- Wave (sinusoid):
 - detects orientation (direction)
 - detects frequency (scale)
- Gaussian:
 - restricts the filter to a local region



→ detects a specific pattern at a specific location

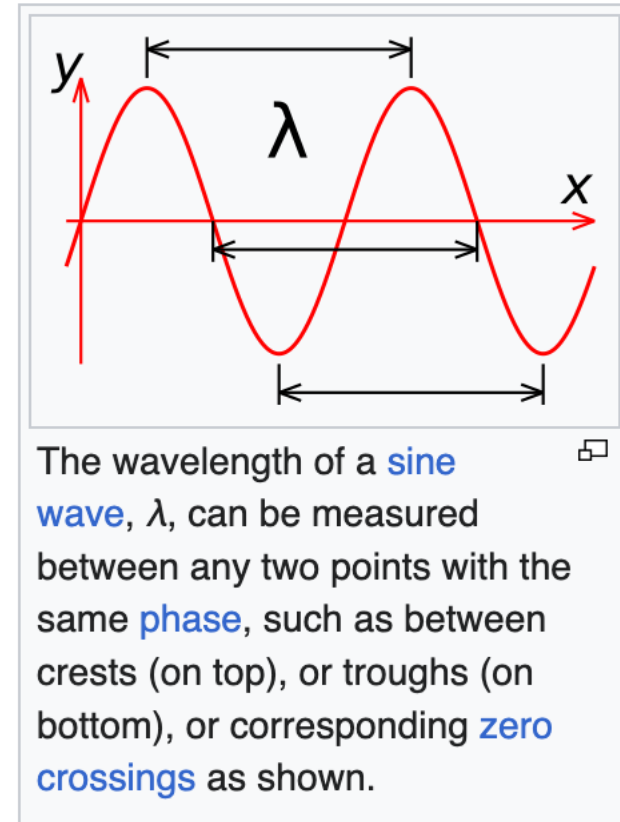
Gabor Filters

- Gabor filters = oriented waves (frequency) $\times$ localized by a Gaussian (position)



Gabor Filters

- Gabor filters = oriented waves (frequency) $\times$ localized by a Gaussian (position)
- The wavelength (λ) controls the frequency ($f = \frac{1}{\lambda}$) of the sin wave:
 λ = distance between peaks of the sine wave
- determines the scale of analysis: larger wavelengths capture coarse structures while smaller ones detect fine details



Gabor Filters

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi\frac{x'}{\lambda} + \psi\right)$$

where $x' = x \cos \theta + y \sin \theta$ and $y' = -x \sin \theta + y \cos \theta$.

The parameters λ, θ, ψ are parameters for the sinusoidal part (or factor).

λ and ψ are basic parameters of a sine wave.

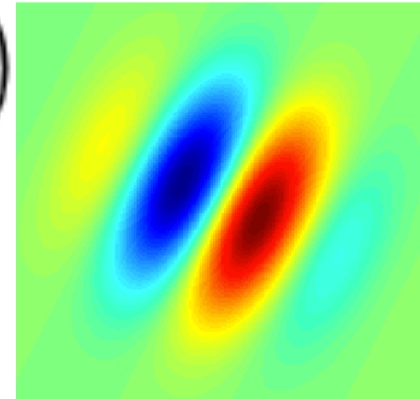
λ - controls the Wavelength (<https://en.wikipedia.org/wiki/Wavelength>) of this sinusoid. A higher λ would "widen" the ellipses you see in the image above, while a lower λ would have the opposite.

θ - controls the rotation of the ellipses you see in the image above. These ellipses are the "parallel stripes". A value of $\theta = 0$ indicates no rotation and would make the ellipses vertical.

ψ - is the phase shift of the sinusoid i.e., how much the ellipses need to be shifted with respect to the center. It's a parameter that has to be specified.

σ - is the standard deviation (i.e., the spread) of the Gaussian part.

γ - controls the aspect ratio of the ellipses (roughly, how elongated the ellipses need to be).



Gabor filter values visualization by Juergen Mueller.

variable naming conventions as used by http://en.wikipedia.org/wiki/Gabor_filter.

Red: positive values. Blue: negative values

1. Wavelength is changed.

1. wavelength (λ) : 5.00

2. orientation (θ) : 0.79

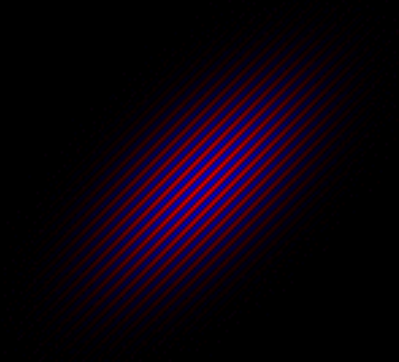
3. gaussvar (σ) : 20.00

4. phaseoffset (ϕ) : 0.00

5. aspectratio (γ) : 0.50

min gabor val : -0.91

max gabor val : 1.00



Color

Important cue for recognition



Important cue for segmentation

“GrabCut” — Interactive Foreground Extraction using Iterated Graph Cuts

Carsten Rother*

Vladimir Kolmogorov[†]
Microsoft Research Cambridge, UK

Andrew Blake[‡]



Figure 1: Three examples of GrabCut. The user drags a rectangle loosely around an object. The object is then extracted automatically.

Rother, Carsten, Vladimir Kolmogorov, and Andrew Blake. "" GrabCut" interactive foreground extraction using iterated graph cuts." *ACM transactions on graphics (TOG)* 23.3 (2004): 309-314.

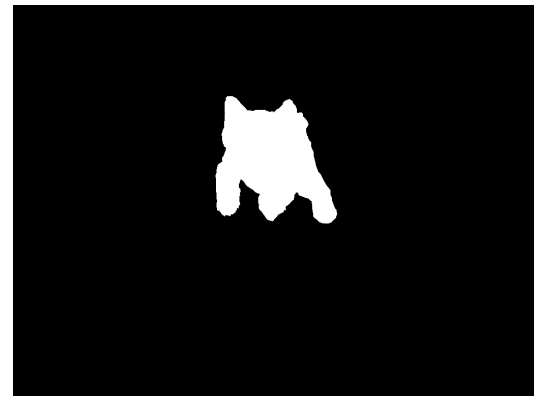
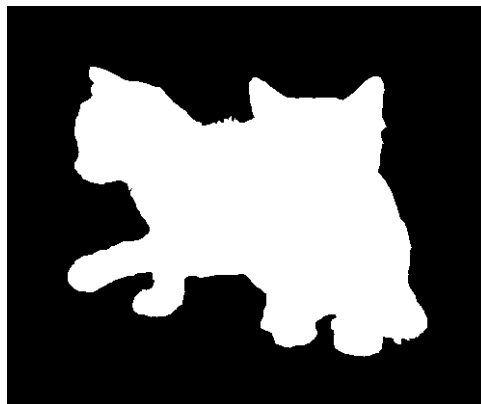
Exercise lab 3

Grabcat - Foreground/background segmentation of cats in images

Input images

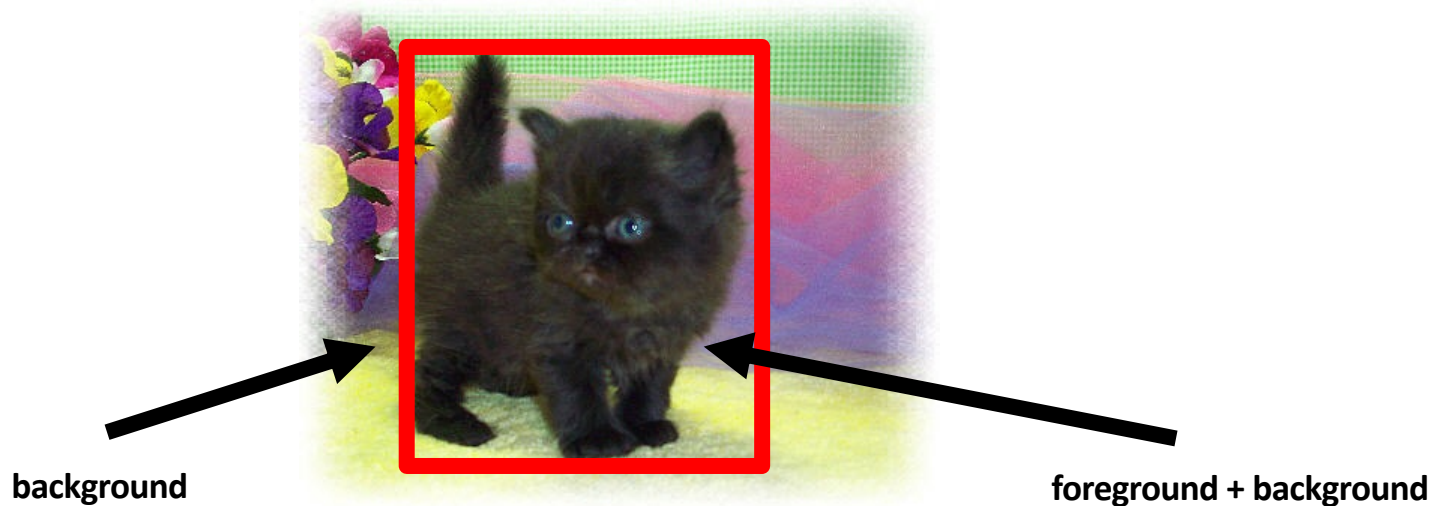


Ground-truth binary masks



Exercise lab 3

Grabcat - Foreground/background segmentation of cats in images

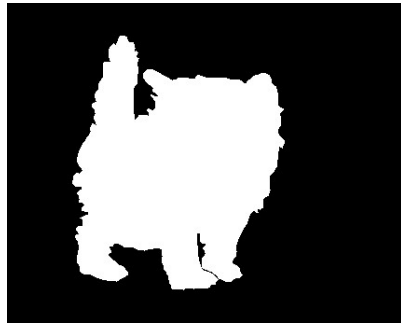


Exercise lab 3

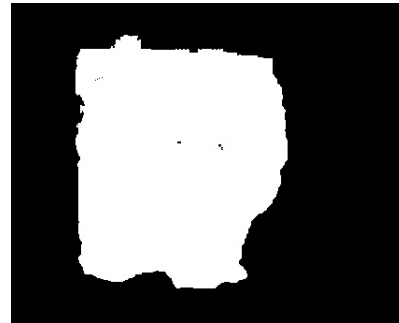
Grabcat - Foreground/background segmentation of cats in images



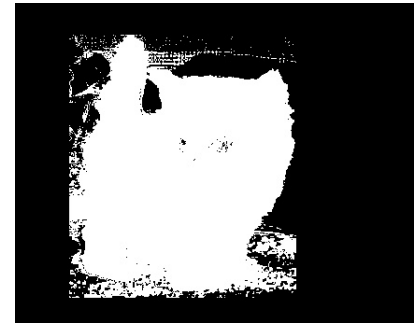
Input image



**Ground-truth
binary mask**



Grabcut



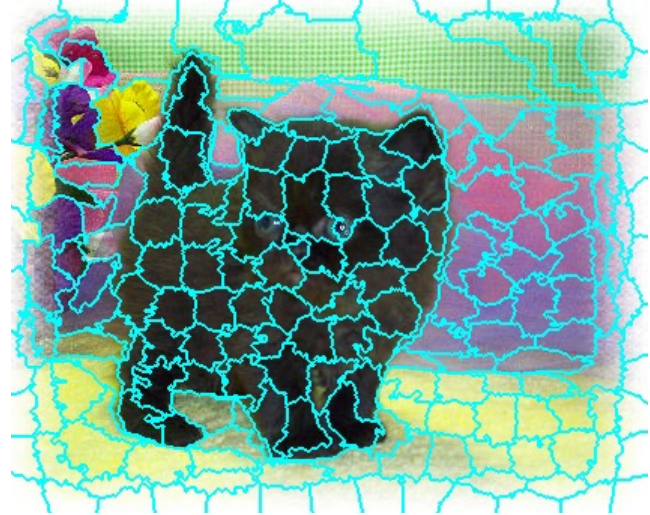
Grabcat

Exercise lab 3 - Grabcat

Input image



Superpixels



Superpixels color labeling



Best achievable mask using superpixels

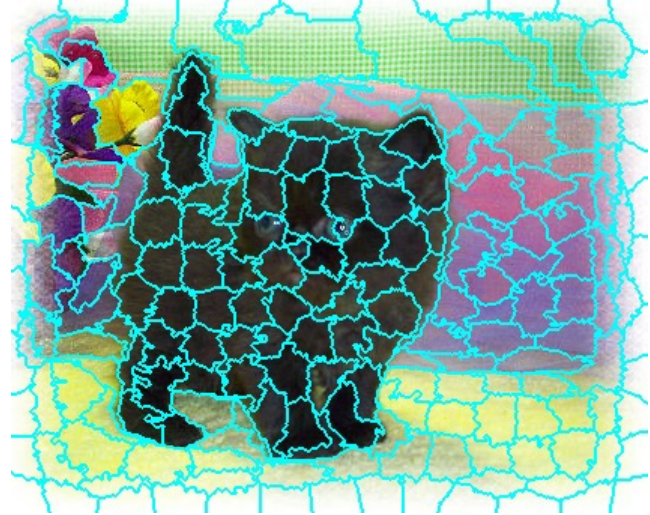


Exercise lab 3 - Grabcat

Input image



Superpixels



Superpixels color labeling



Result obtained using superpixels



SLIC Superpixels Compared to State-of-the-art Superpixel Methods

Radhakrishna Achanta, Appu Shaji, Kevin Smith,
Aurelien Lucchi, Pascal Fua, and Sabine Süsstrunk

Abstract—Computer vision applications have come to rely increasingly on superpixels in recent years, but it is not always clear what constitutes a good superpixel algorithm. In an effort to understand the benefits and drawbacks of existing methods, we empirically compare five state-of-the-art superpixel algorithms for their ability to adhere to image boundaries, speed, memory efficiency, and their impact on segmentation performance. We then introduce a new superpixel algorithm, *simple linear iterative clustering* (SLIC), which adapts a *k*-means clustering approach to efficiently generate superpixels. Despite its simplicity, SLIC adheres to boundaries as well as or better than previous methods. At the same time, it is faster and more memory efficient, improves segmentation performance, and is straightforward to extend to supervoxel generation.

regular lattice, such as [23], is probably a better choice. While it is difficult to define what constitutes an ideal approach for all applications, we believe the following properties are generally desirable:

- 1) Superpixels should adhere well to image boundaries.
- 2) When used to reduce computational complexity as a pre-processing step, superpixels should be fast to compute, memory efficient, and simple to use.
- 3) When used for segmentation purposes, superpixels should both increase the speed and improve the quality of the results.

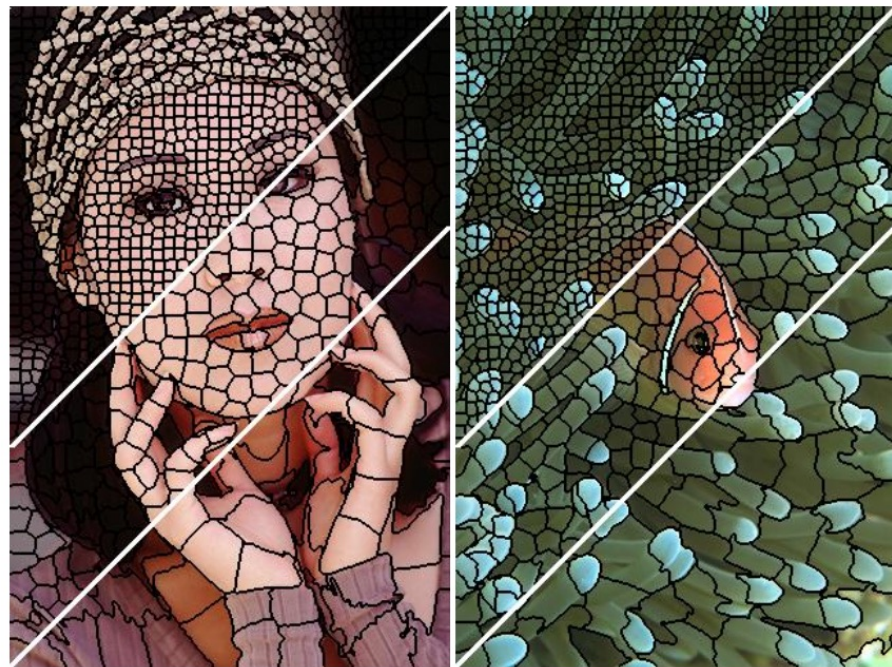


Fig. 1: Images segmented using SLIC into superpixels of size 64, 256, and 1024 pixels (approximately).

and impact on segmentation performance. We also provide

Images can naturally be decomposed in:

pixel → edge → texton → superpixel → part → object

1.Pixel

- the smallest unit of an image, a single point with a specific color (or intensity in grayscale).
- each pixel doesn't carry much information, it's only when grouping pixels together meaningful structures emerge.

2.Edge

- an edge is a boundary where there is a sharp change in color or intensity.
- by detecting edges, we start to see outlines and boundaries of shapes, the first step in for higher-level features.

3.Texton

- a “texton” is a small, fundamental visual pattern (like a local texture element) which often repeats in a region. For example, the basic brick shape in a brick wall, or a small patch of repeated grain in wood texture.
- textons capture local texture statistic. recognizing them help moving beyond just raw edges to textures/patterns.

4.Superpixel

- a cluster of pixels (often hundreds or thousands) that share similar color or texture. Instead of looking at each individual pixel, you group them into a larger, more meaningful “patch.”
- superpixels simplify the image representation. Each superpixel represents a region that's uniform in some way (color, texture, brightness), which makes further processing more efficient and structured.

5.Part

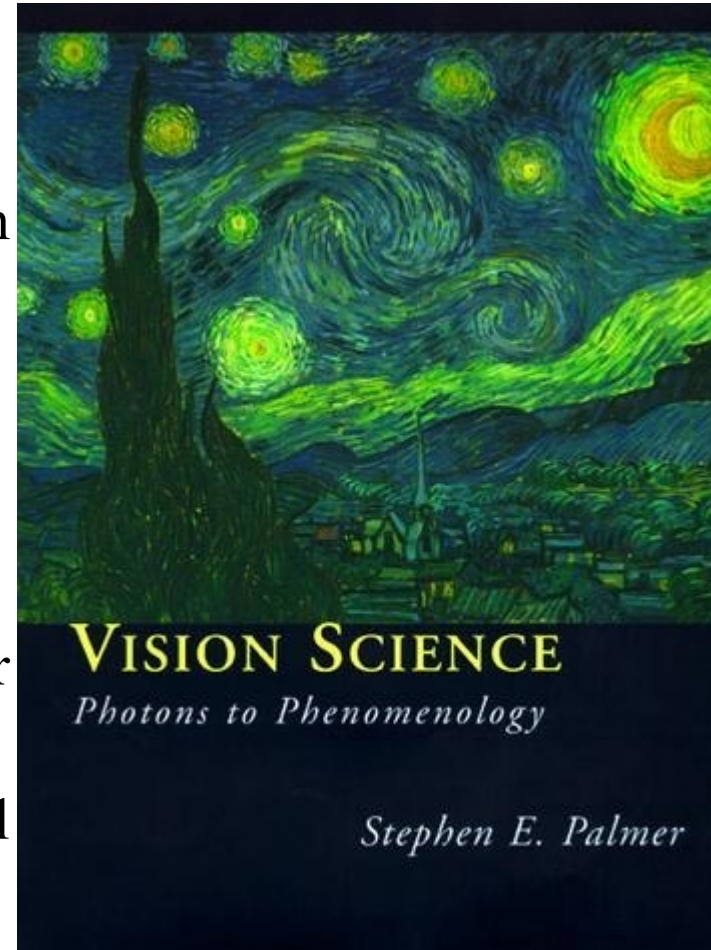
- a significant portion of an object. For instance, if you have a face, “parts” would be the eyes, nose, or mouth; if you have a car, “parts” might be the wheels, doors, or windshield.
- once you group superpixels into larger, functionally relevant segments, these start to represent meaningful “parts” of an object. Parts are closer to what we humans intuitively recognize.

6.Object

- the complete, recognizable entity in the scene (e.g., a person, a car, a tree).
- combining all the parts (which are themselves made up of superpixels, which in turn come from pixels) yields the full, identifiable object.

What is (perceived) color?

- Perceived color is NOT a property of objects.
 - Perceived color is the result of interaction between:
 - illumination (light spectrum)
 - surface reflectance
 - the visual system (eye + brain)
- The same object can appear different under different lighting
- Different spectra can appear identical (metamers)



How does the human eye work?

Light → Eye → Brain

1. Light enters the eye

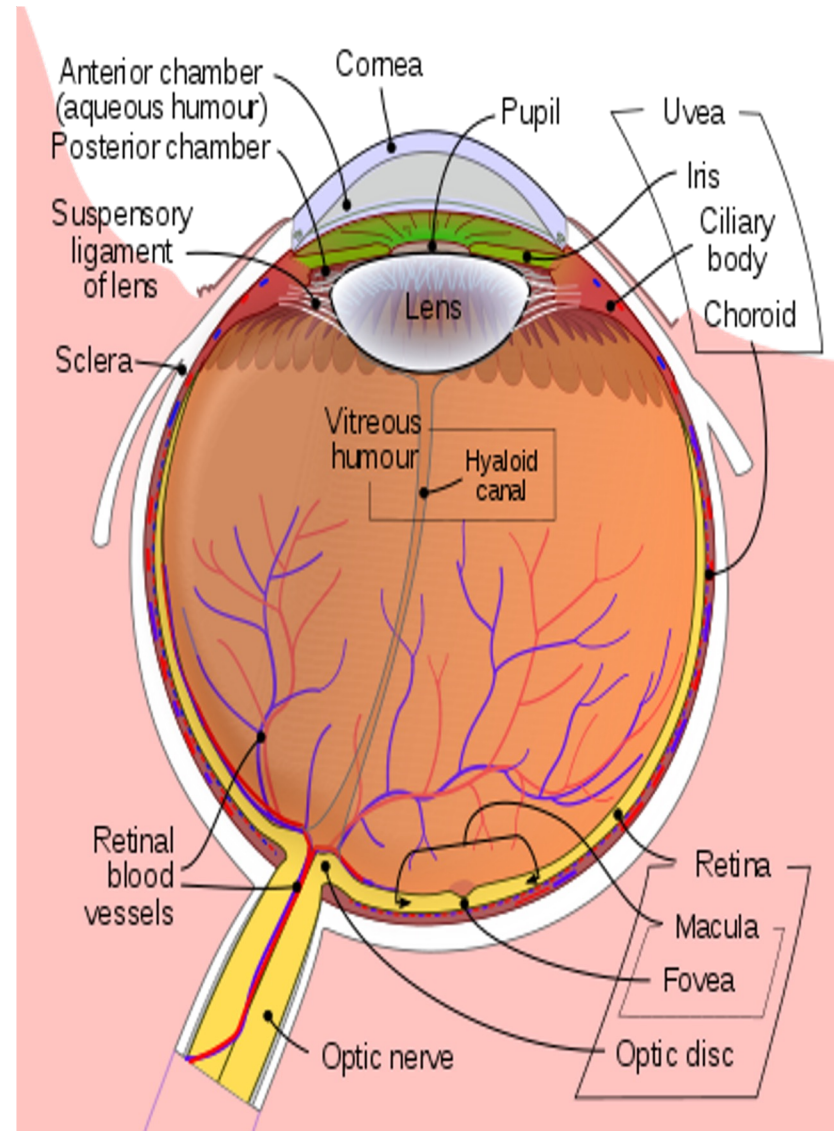
- passes through cornea, pupil, and lens
- lens focuses light onto the retina

2. Retina (sensor layer)

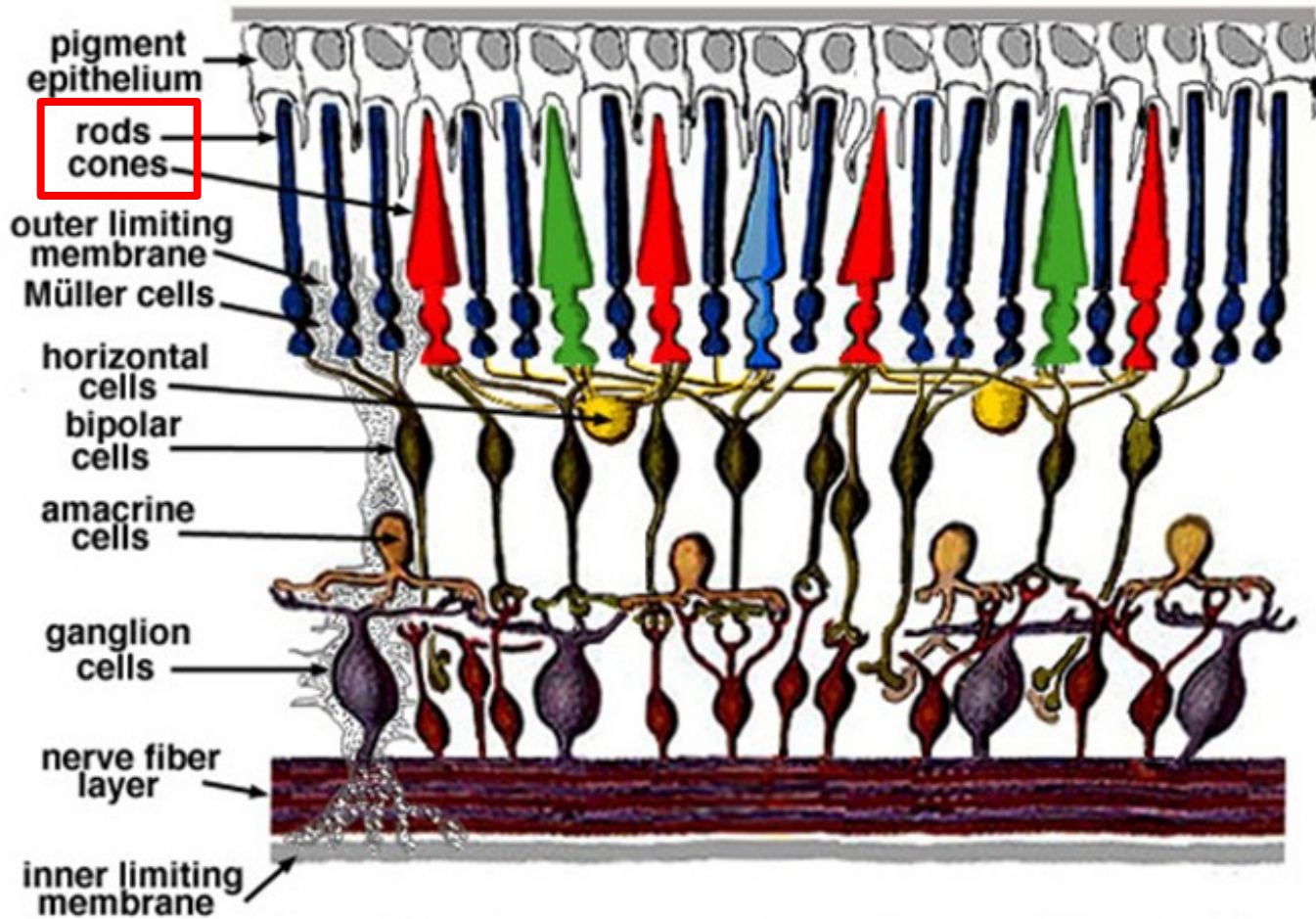
- contains photoreceptors:
 - rods (intensity)
 - cones (color)
- converts light into electrical signals

3. Neural processing

- signals travel through the optic nerve
- processed in the visual cortex (brain)



Retina

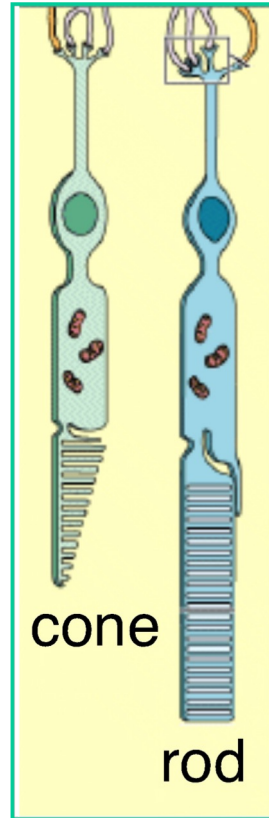


Simple diagram of the organization of the retina.

Photoreceptors: Cones vs Rods

Cones (RO: conuri)

- ~6 million
- Active in bright light
- Low sensitivity (need many photons)
- Fast response
- High spatial resolution (fine details)
- Color vision (S, M, L cones)
- Concentrated in the fovea



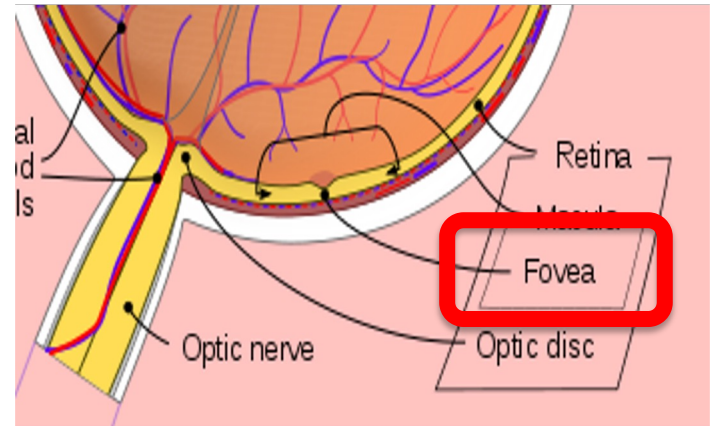
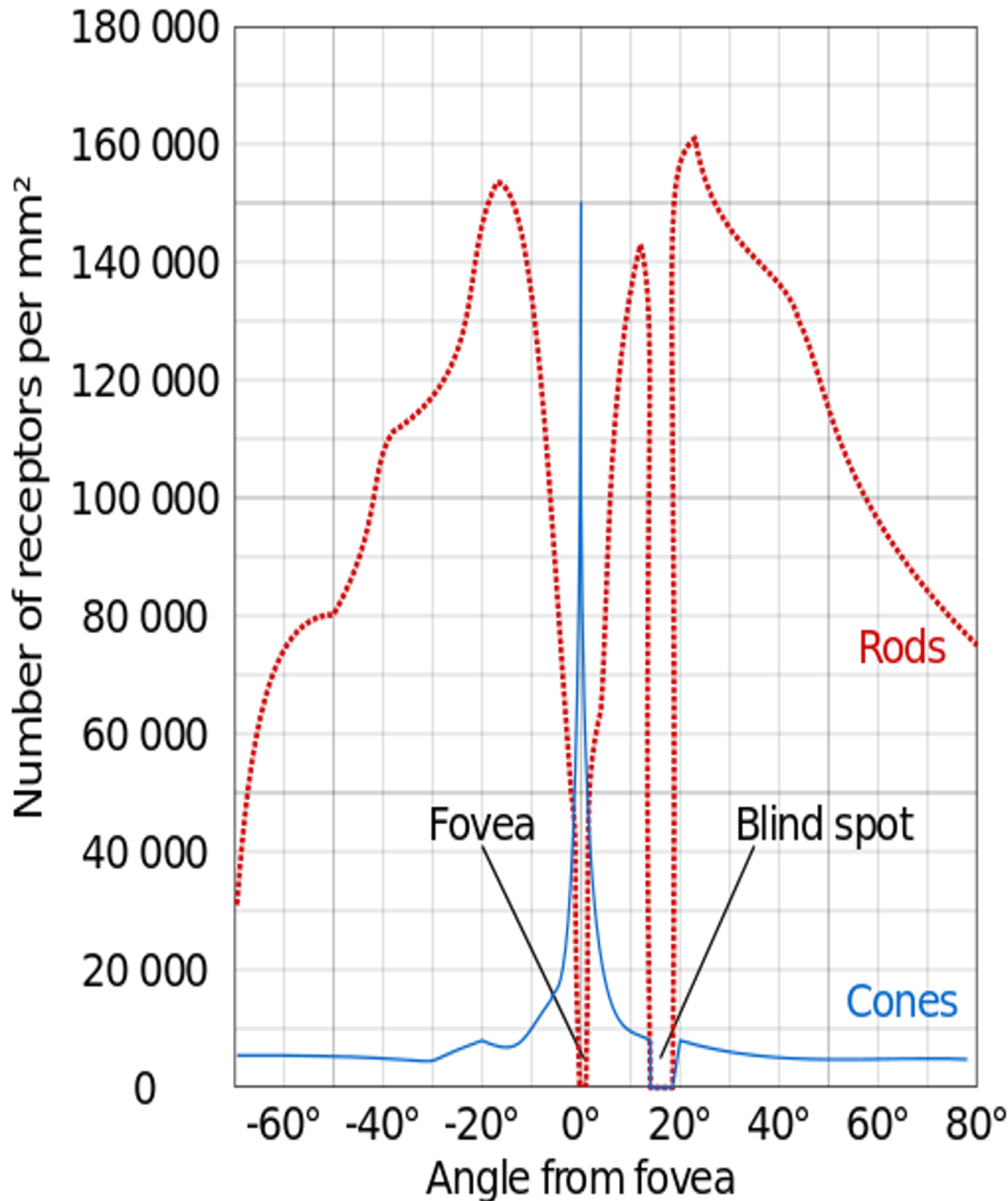
Rods (RO: bastonaş)

- ~120 million
- Active in low light
- Very high sensitivity (can detect single photons)
- Slow response
- Low spatial resolution (pooled signals)
- No color (grayscale vision)
- Located in the peripheral retina

→ **Cones = color & detail (day vision)**

→ **Rods = sensitivity (night vision)**

Fovea: where it's all happening



Fovea: where it's all happening

- Small central region of the retina (~1.5 mm)
- Packed with cones (almost no rods)
→ highest spatial resolution
- Enables:
 - sharp vision
 - color perception
- We actively move our eyes to place objects on the fovea (e.g., when reading)

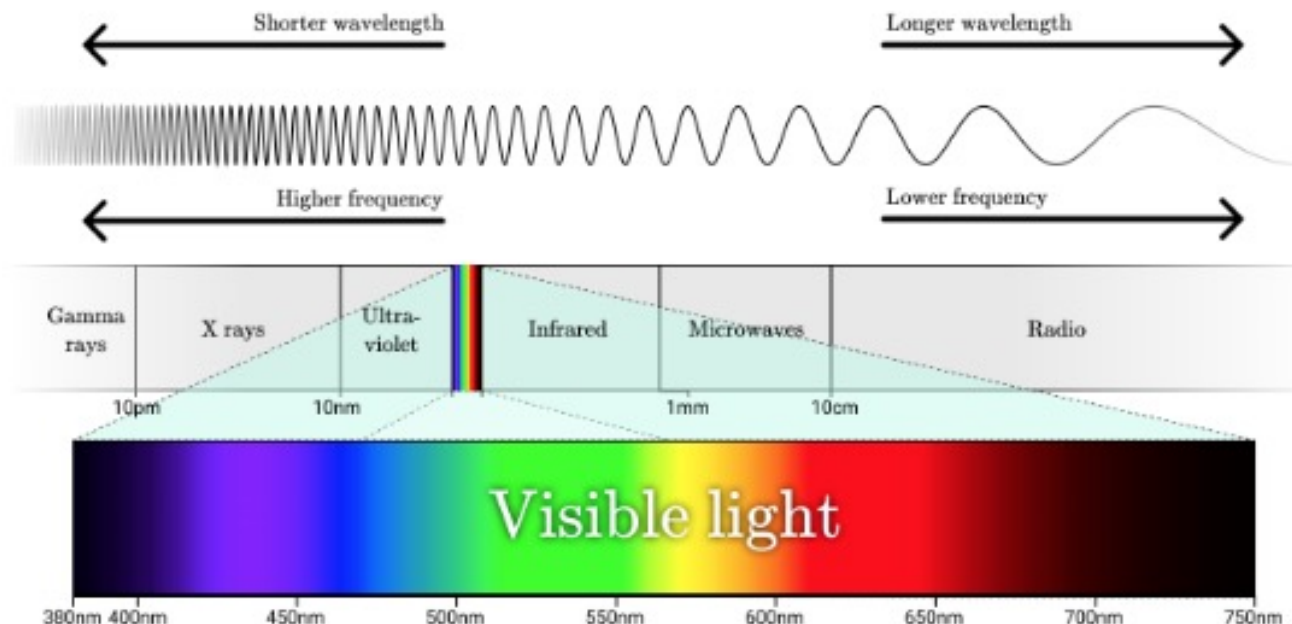
What is perceived color?

Perceived color is a perceptual interpretation of light. It depends on:

- the spectral distribution of illumination
- the reflectance of surfaces
- the response of the visual system

Visible light:

- wavelengths $\approx 380 - 750$ nm



The electromagnetic spectrum

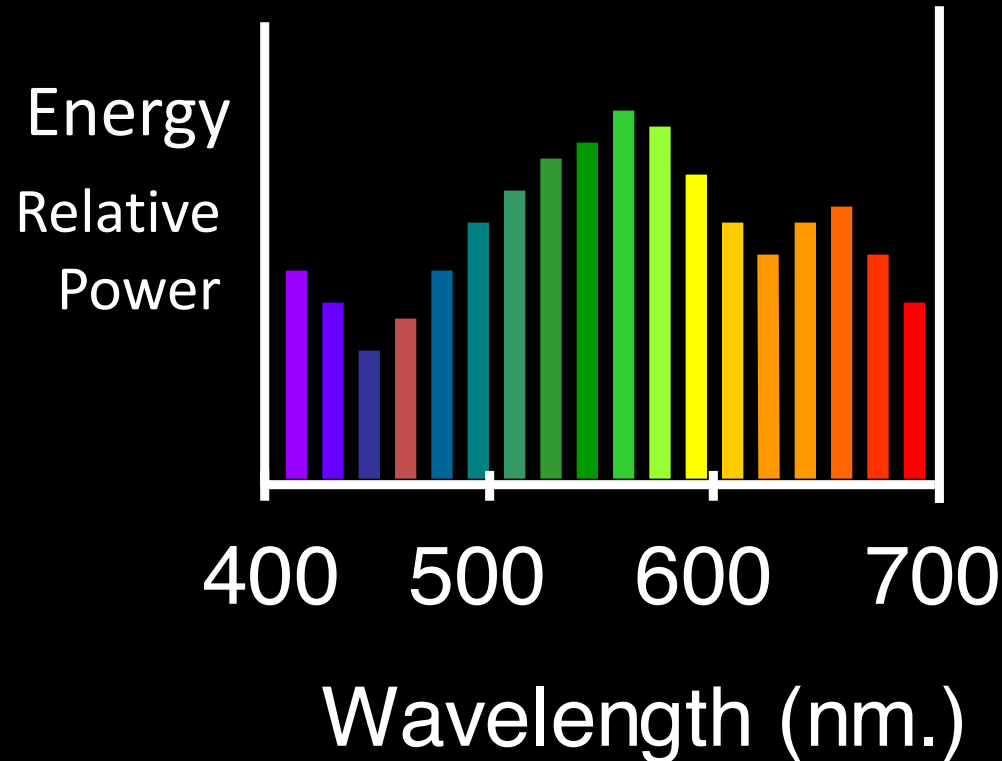
Light as a Physical Signal

Light is described by its spectrum:

- For each wavelength $\lambda \rightarrow$ how much energy is present
- Spectrum = energy distribution over wavelengths

Different spectra $\rightarrow$ different perceived colors

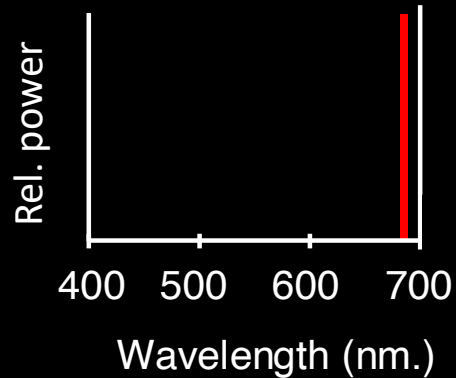
Same perceived color can come from different spectra (metamers)



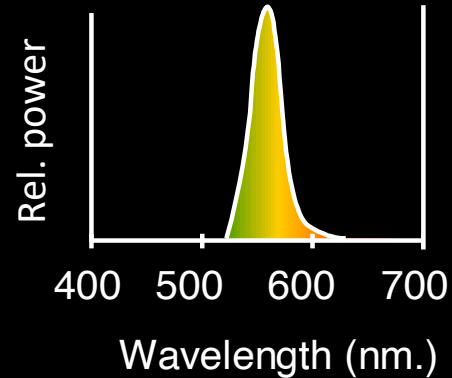
Spectra of Light Sources

Some examples of the spectra of light sources

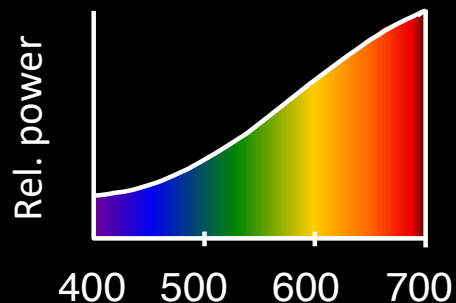
A. Ruby Laser



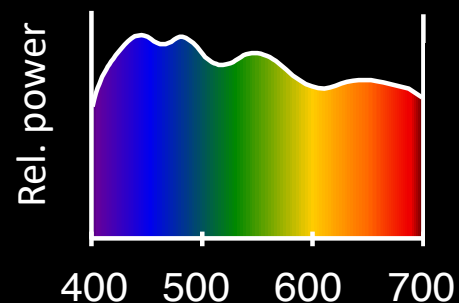
B. Gallium Phosphide Crystal



C. Tungsten Lightbulb

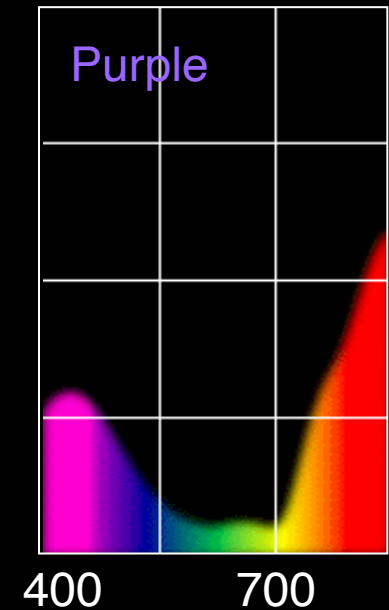
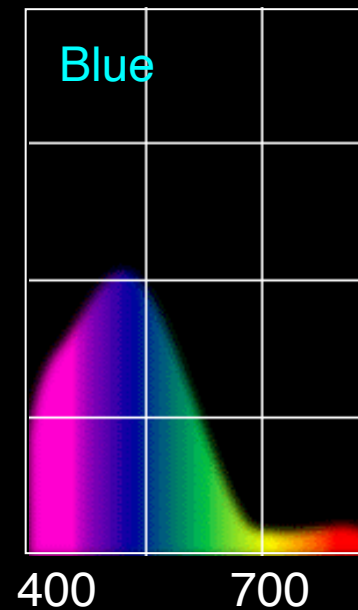
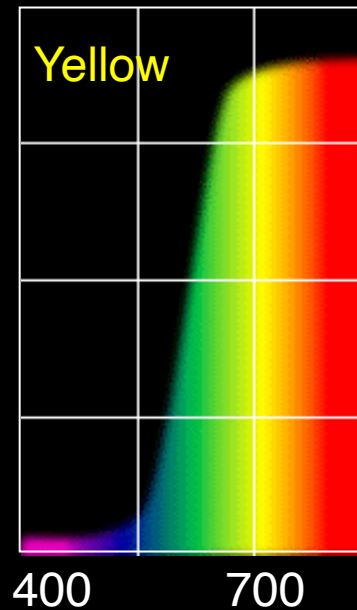
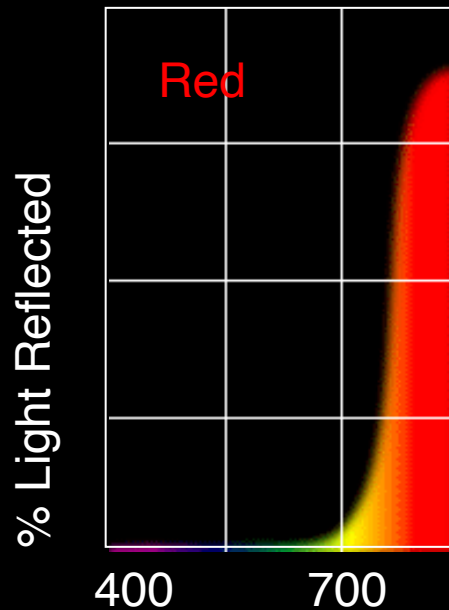


D. Normal Daylight



Reflectance Spectra of Surfaces

Some examples of the reflectance spectra of surfaces

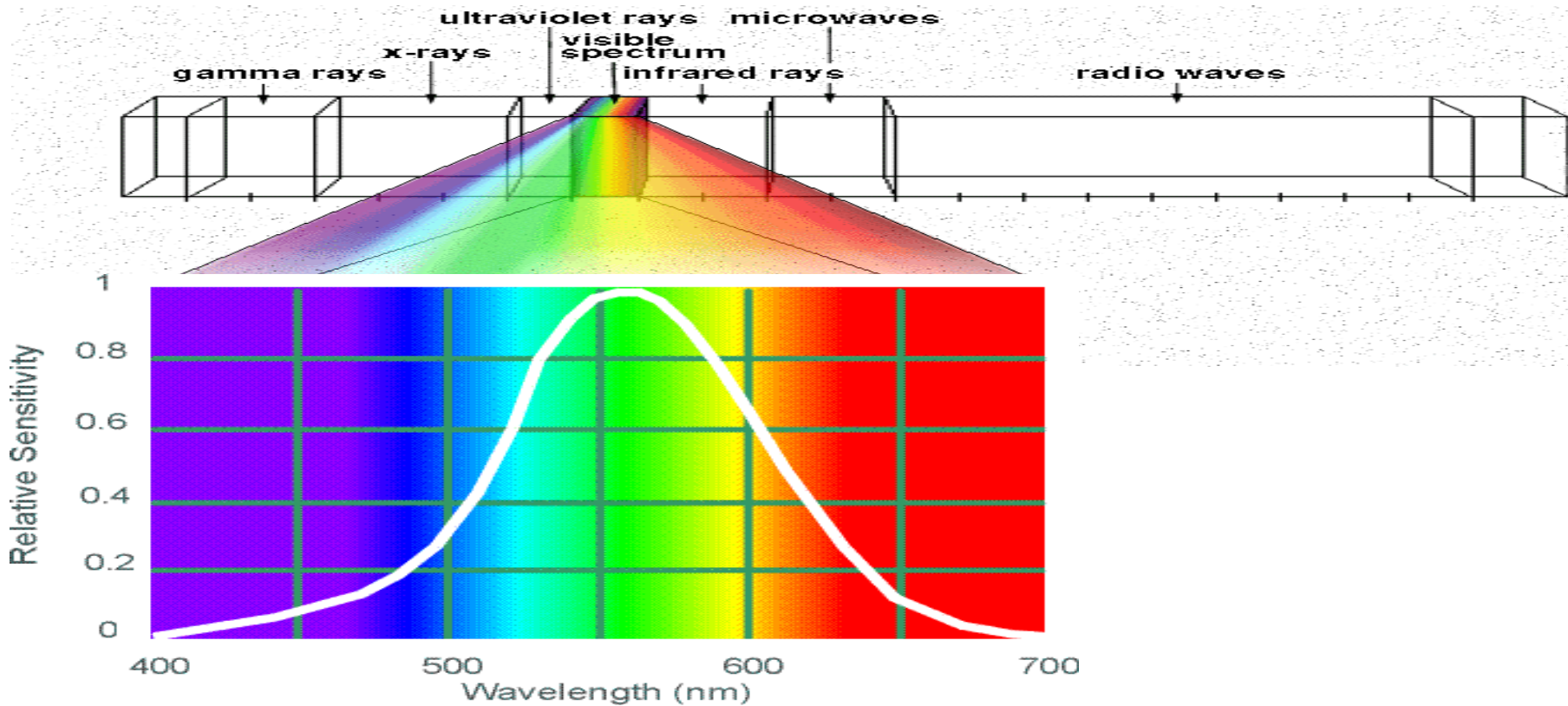


Wavelength (nm)

Human sensitivity to light

The human eye is not equally sensitive to all wavelengths.

- Highest sensitivity $\approx$ green (~ 550 nm)
- Lower sensitivity to blue and red

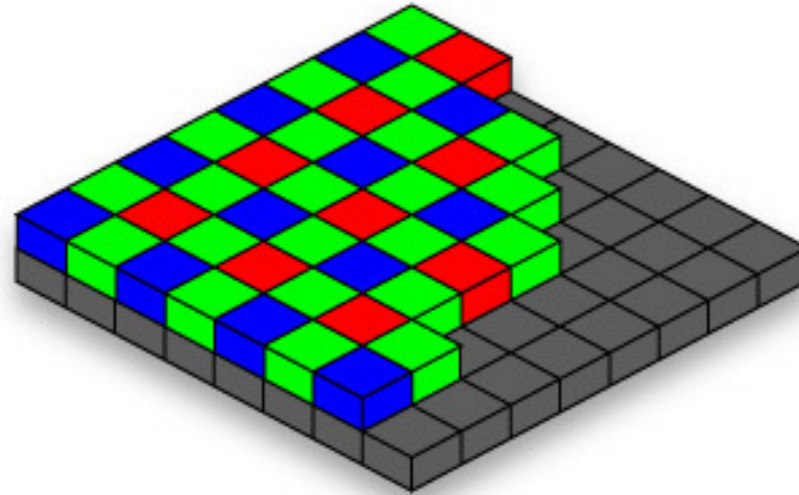


Human Luminance Sensitivity Function

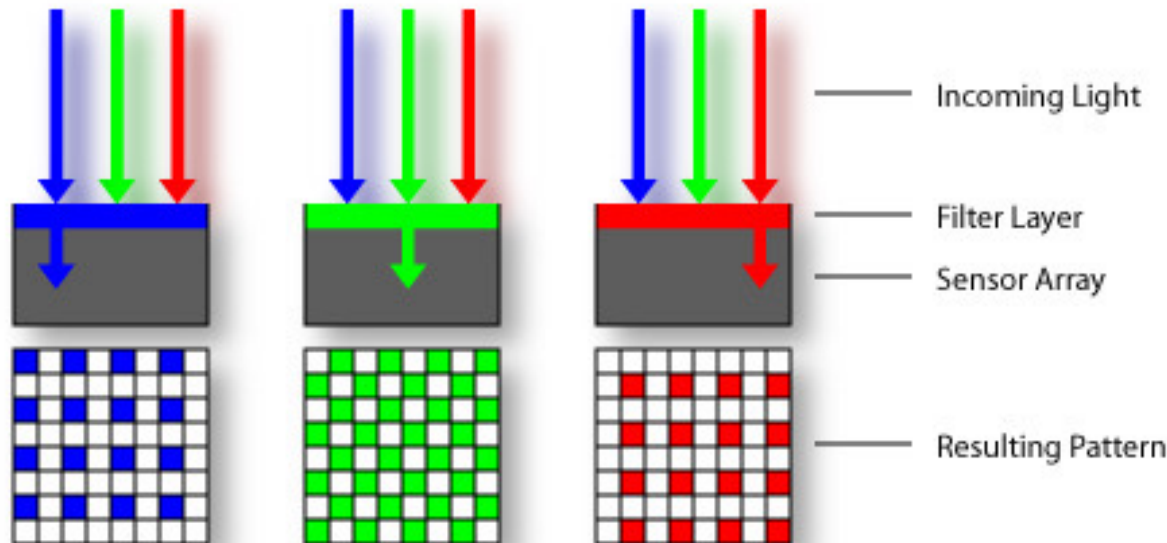
Why RGB?

If light is a spectrum, why are images RGB?

Color Sensing: Bayer Grid



- Estimate RGB at each cell from neighboring values



How the eye encodes light

Photoreceptors have different sensitivity curves

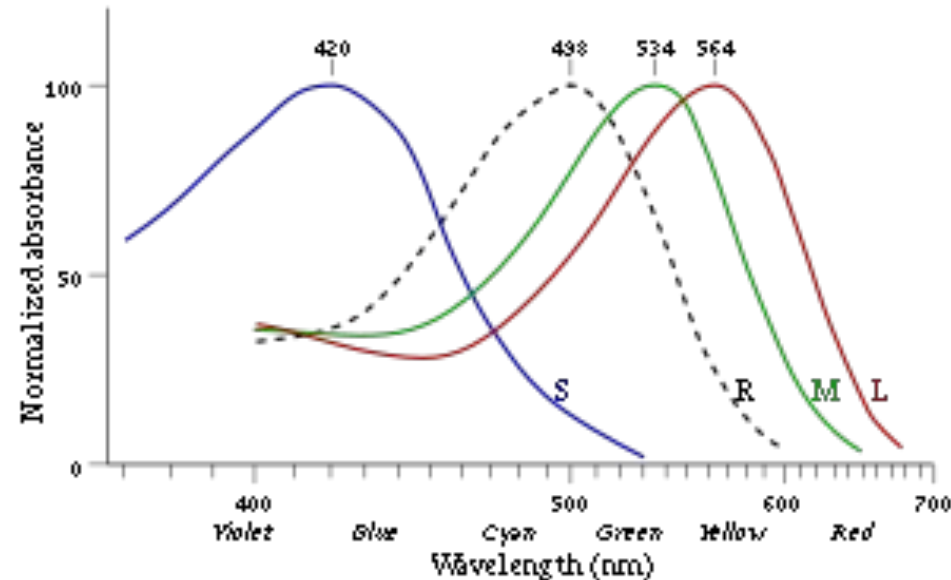
- Each receptor responds differently across wavelengths

Rods:

- Peak sensitivity ≈ 498 nm
- Responsible for low-light vision

Cones (3 types):

- S (short) $\rightarrow$ peak ≈ 420 nm
- M (medium) $\rightarrow$ peak ≈ 530 nm
- L (long) $\rightarrow$ peak ≈ 560 nm



Cones and color perception

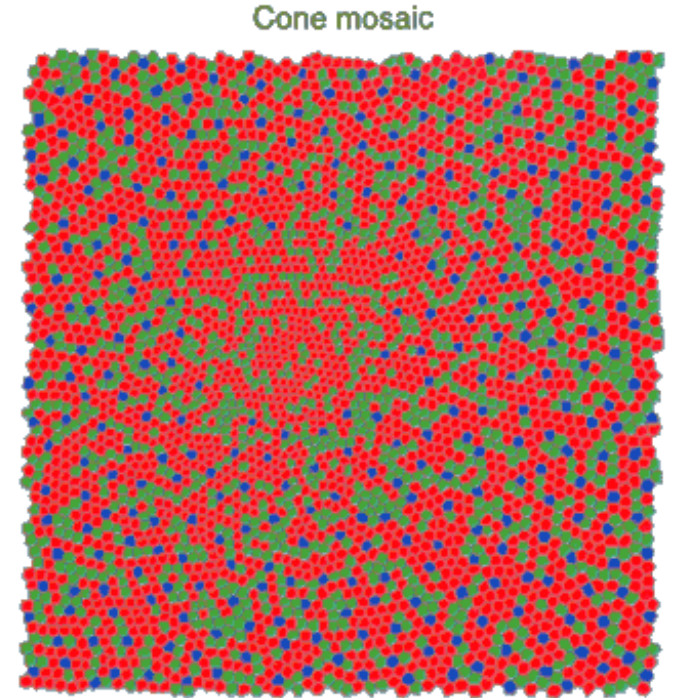
- Light is a continuous spectrum
- Cones sample this spectrum differently

Each cone produces one value:

→ S (blue), M (green), L (red)

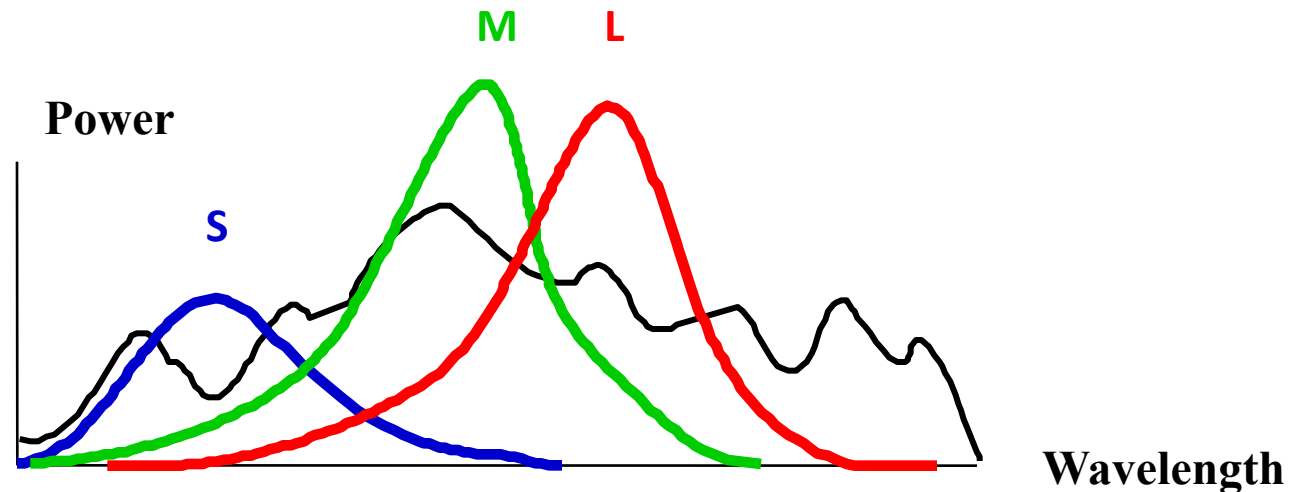
Color = (S, M, L) responses

Different spectra → same responses
(metamers)



- Ratio of Long to Medium to Short cones: approx. 10:5:1
- Almost no S cones in the center of the fovea

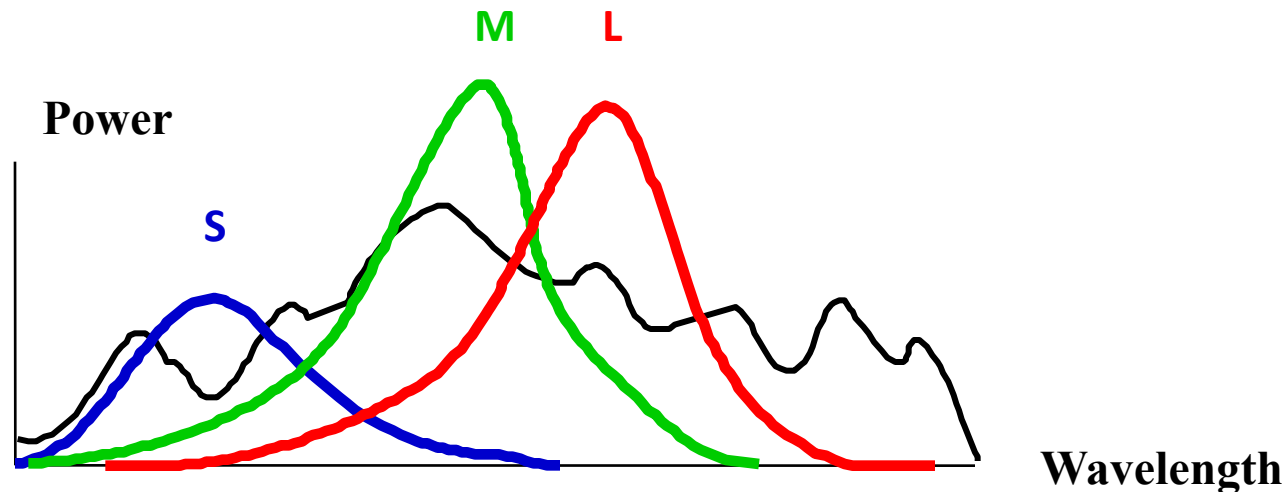
Color perception



Rods and cones act as *filters* on the spectrum

- To get the output of a filter, multiply its response curve by the spectrum, integrate over all wavelengths
 - each cone yields one number
- The camera captures light through 3 filters (R, G, B)
 - these filters are like coarse samplers over the spectrum,
 - the full spectrum of the light is compressed into 3 values (the RGB values obtained)

Color perception

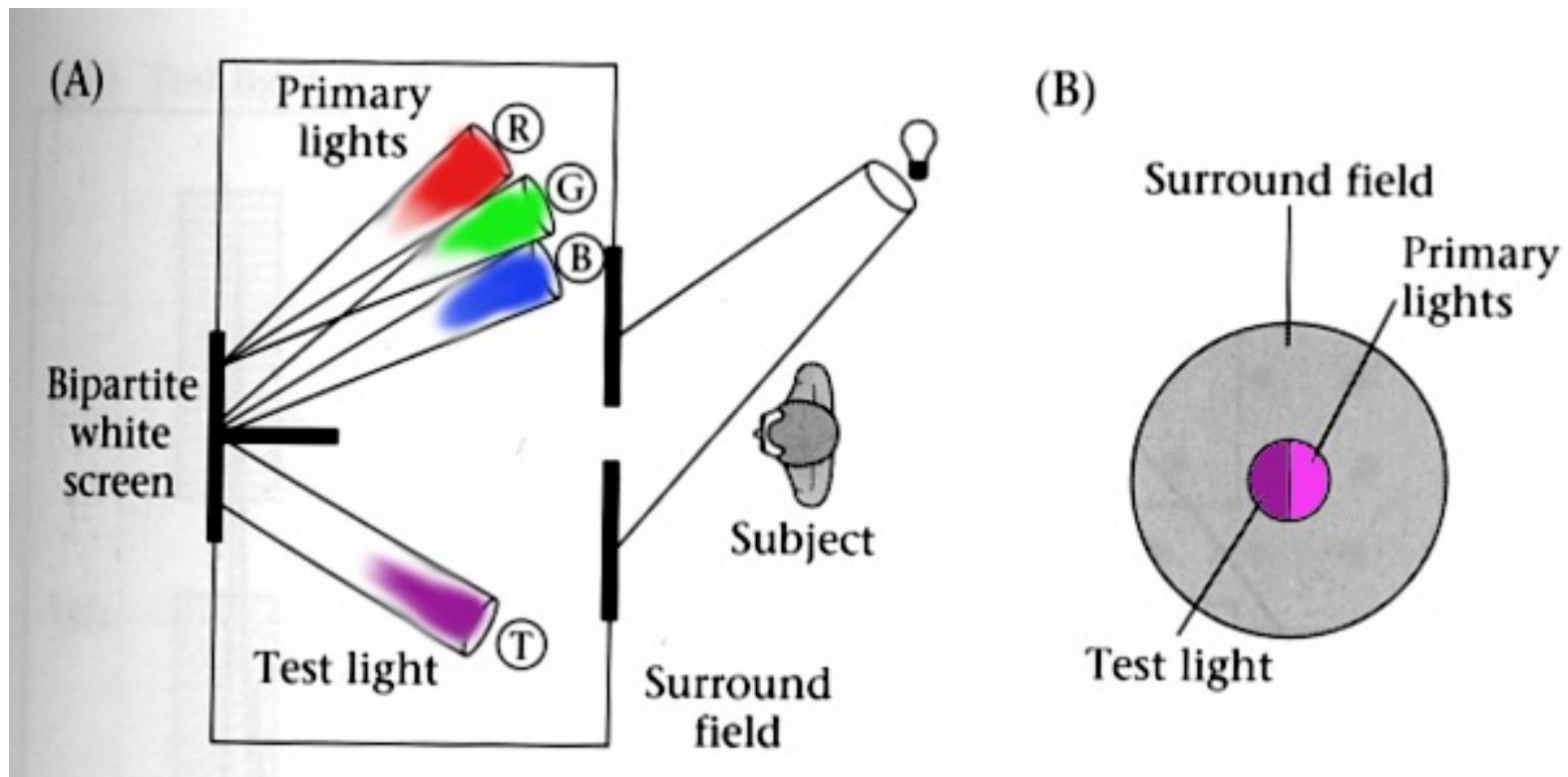


Rods and cones act as *filters* on the spectrum

- To get the output of a filter, multiply its response curve by the spectrum, integrate over all wavelengths
 - each cone yields one number
- How can we represent an entire spectrum with three numbers? We can't!
Most of the information is lost
 - as a result, two different spectra may appear indistinguishable (such spectra are known as **metamers** = **lights that appear visually the same**)

Color matching experiments

- Late 1920s experimented with colors! (and people)
 - Subjects get controls to 3 “primary” lights
 - Show them a light
 - Subject adjusts their lights to match the given light



Trichromatic color theory

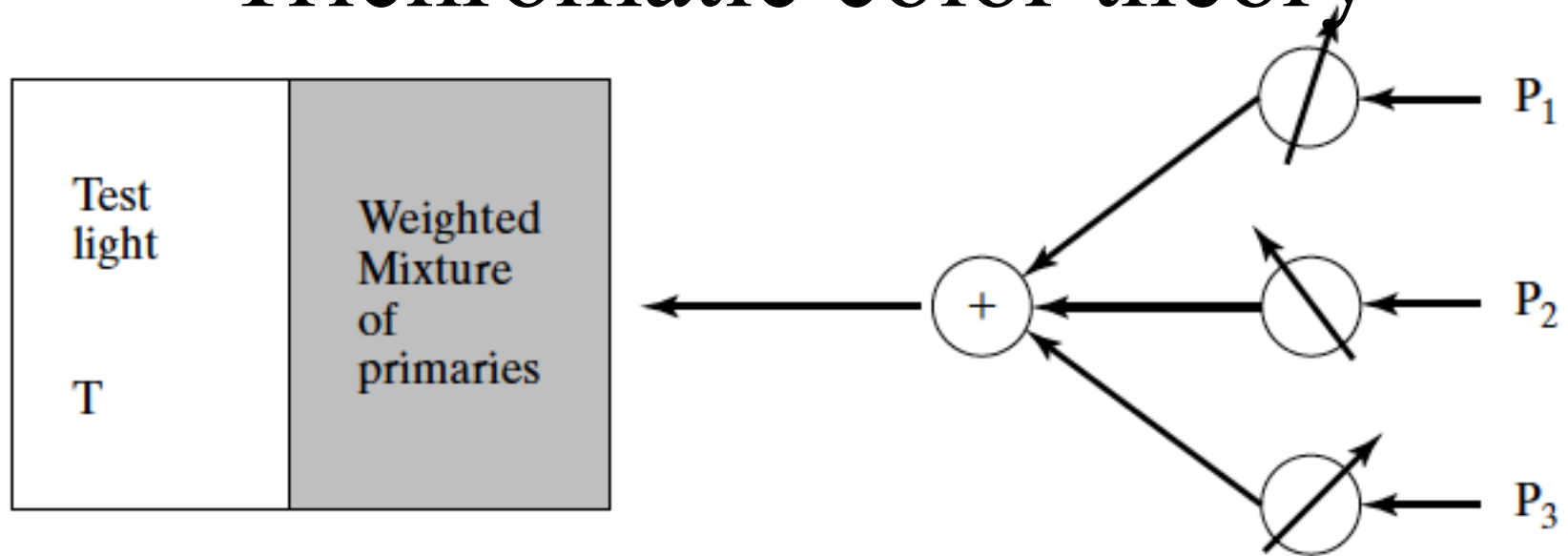
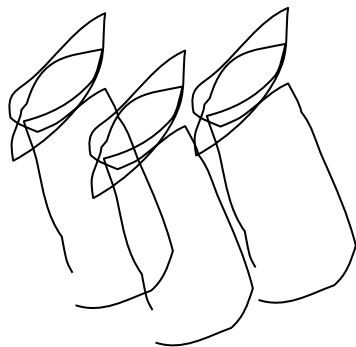
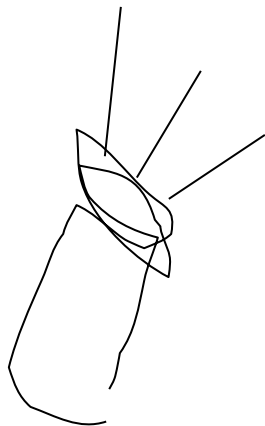
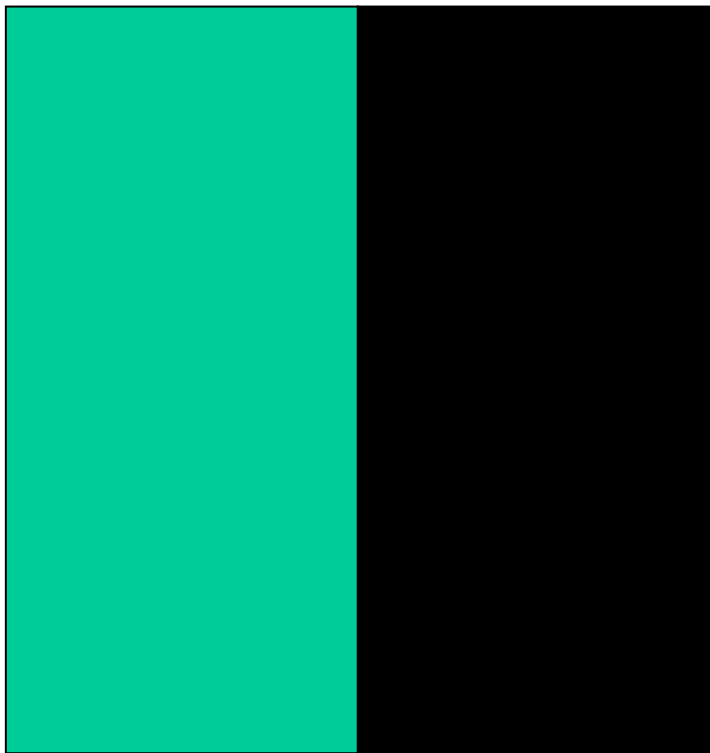
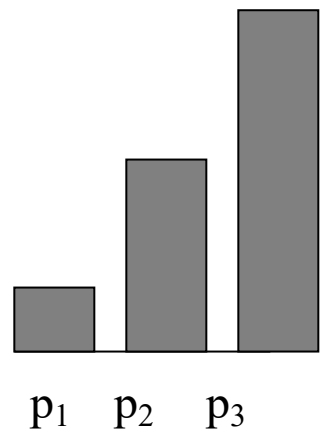
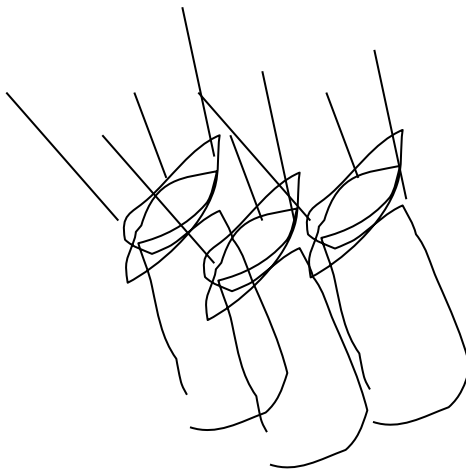
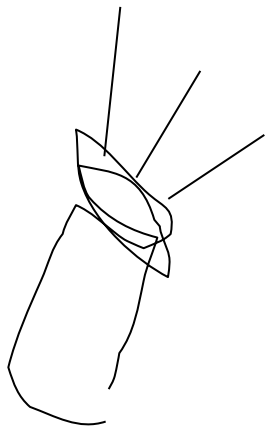
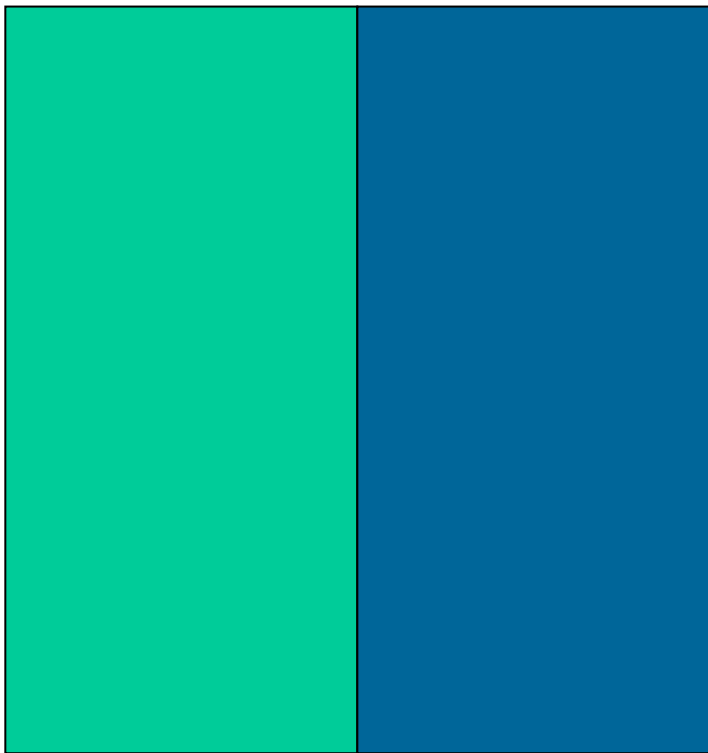


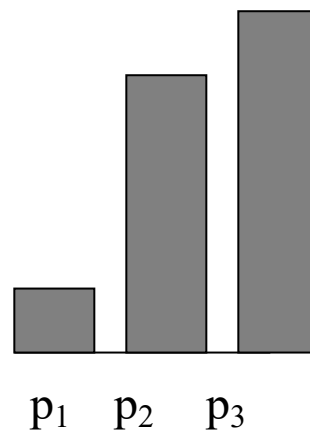
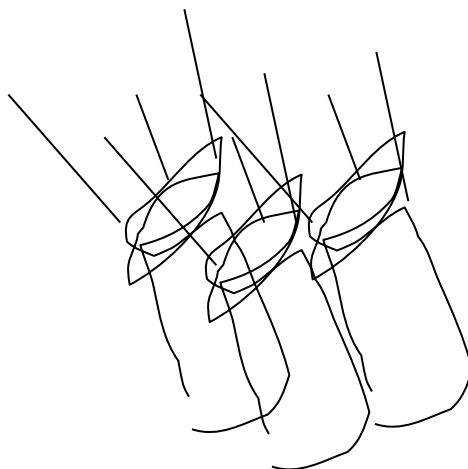
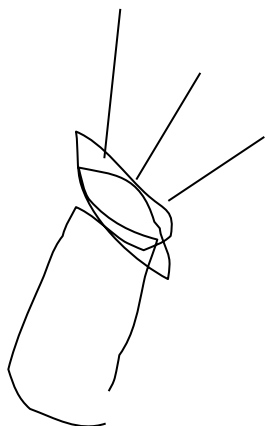
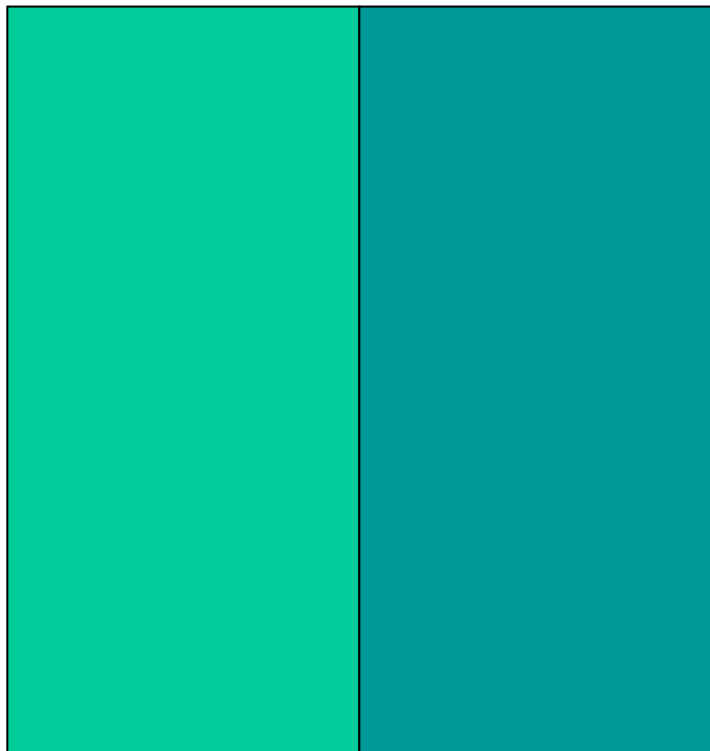
FIGURE 3.2: Human perception of color can be studied by asking observers to mix colored lights to match a test light shown in a split field. The drawing shows the outline of such an experiment. The observer sees a test light T and can adjust the amount of each of three primaries in a mixture displayed next to the test light. The observer is asked to adjust the amounts so that the mixture looks the same as the test light. The mixture of primaries can be written as $w_1P_1 + w_2P_2 + w_3P_3$; if the mixture matches the test light, then we write $T = w_1P_1 + w_2P_2 + w_3P_3$. It is a remarkable fact that for most people three primaries are sufficient to achieve a match for many colors, and three primaries are sufficient for all colors if we allow subtractive matching (i.e., some amount of some of the primaries is mixed with the test light to achieve a match). Some people require fewer primaries. Furthermore, most people choose the same mixture weights to match a given test light.

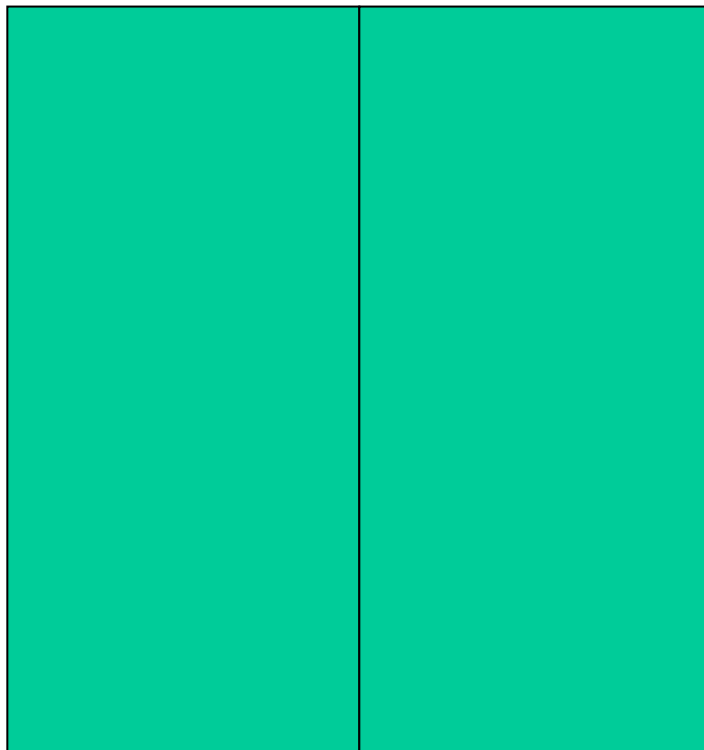
Trichromatic color theory

- In color matching experiments, most people can match any given light with three primaries
 - Primaries must be *independent*
- For the same light and same primaries, most people select the same weights
 - Exception: color blindness
- Trichromatic color theory
 - Three numbers seem to be sufficient for encoding color
 - Dates back to 18th century (Thomas Young)

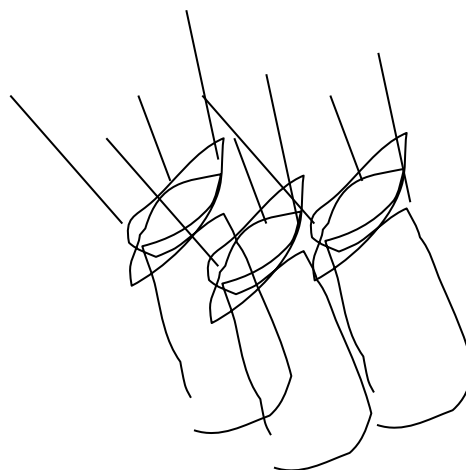
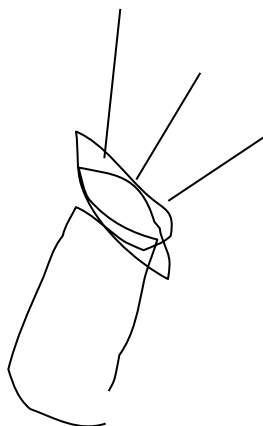
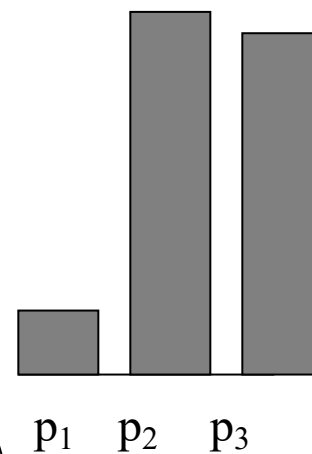


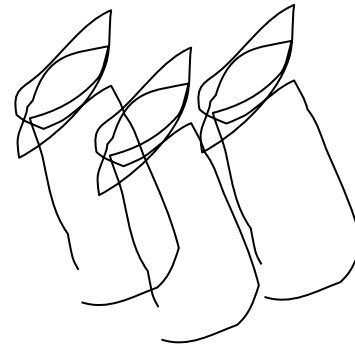
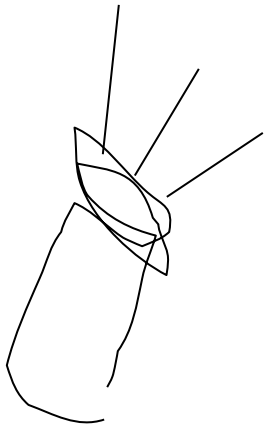


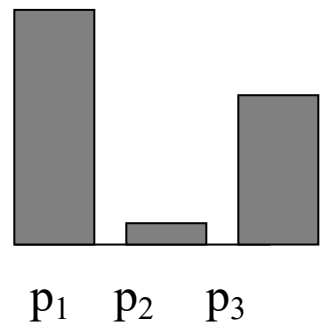
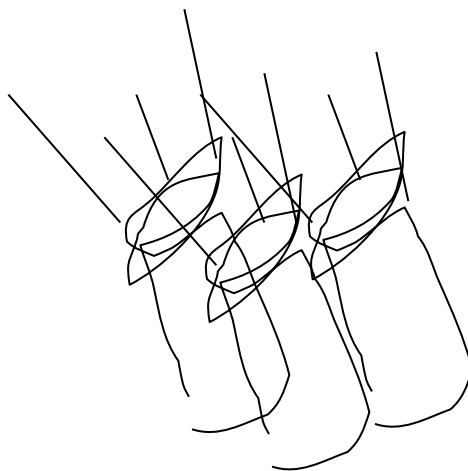
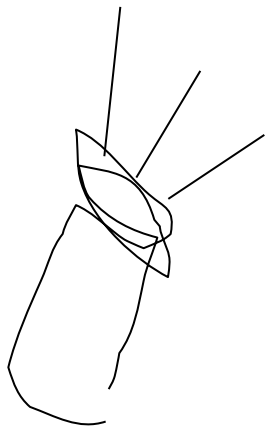
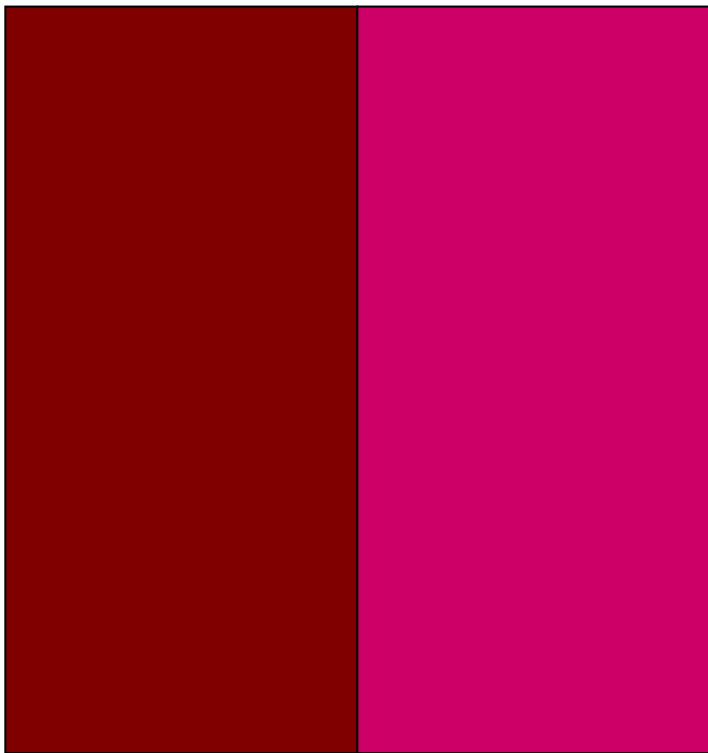


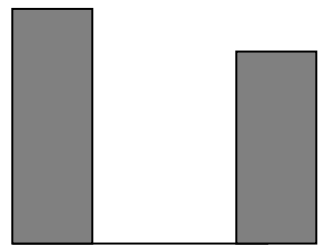
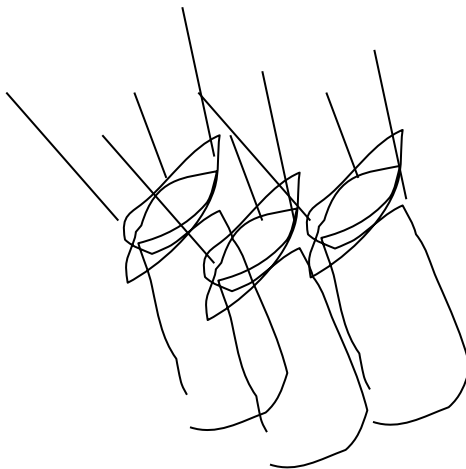
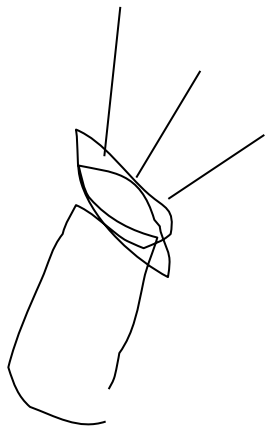
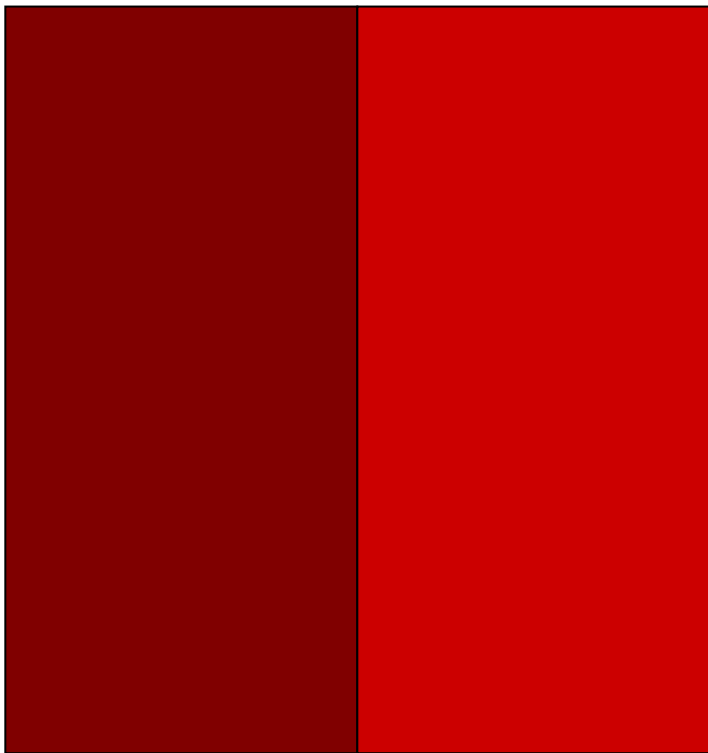


**The primary color
amounts needed
for a match**



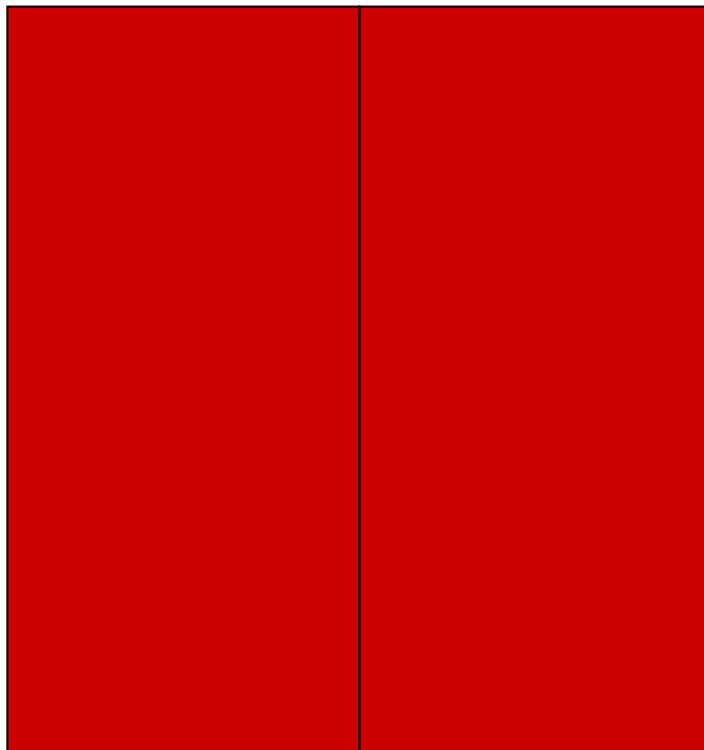




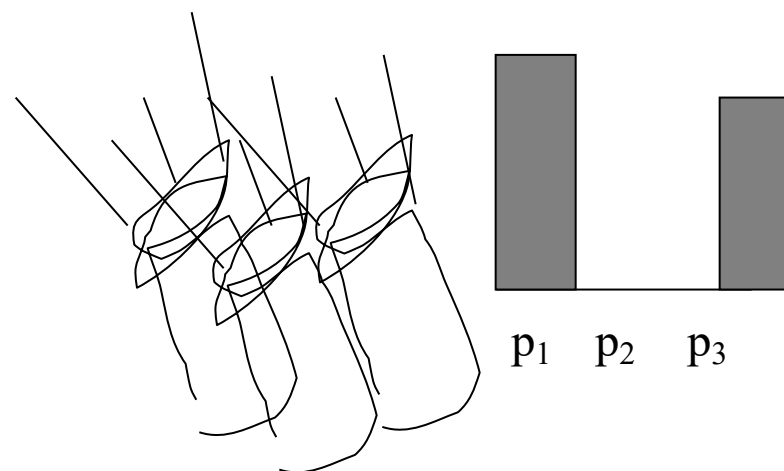
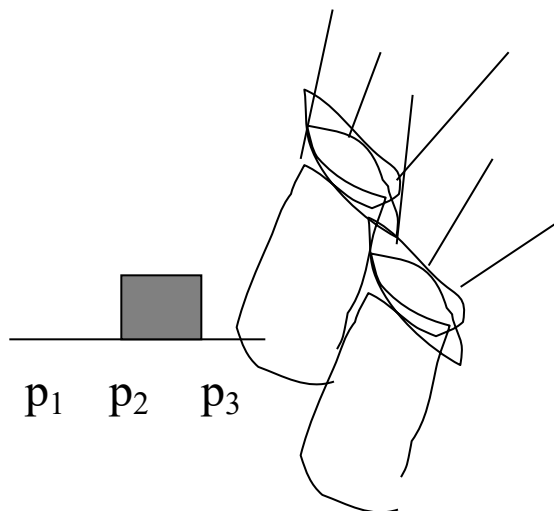
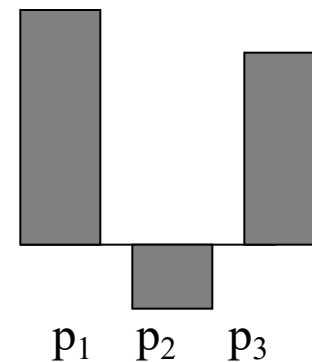


p_1 p_2 p_3

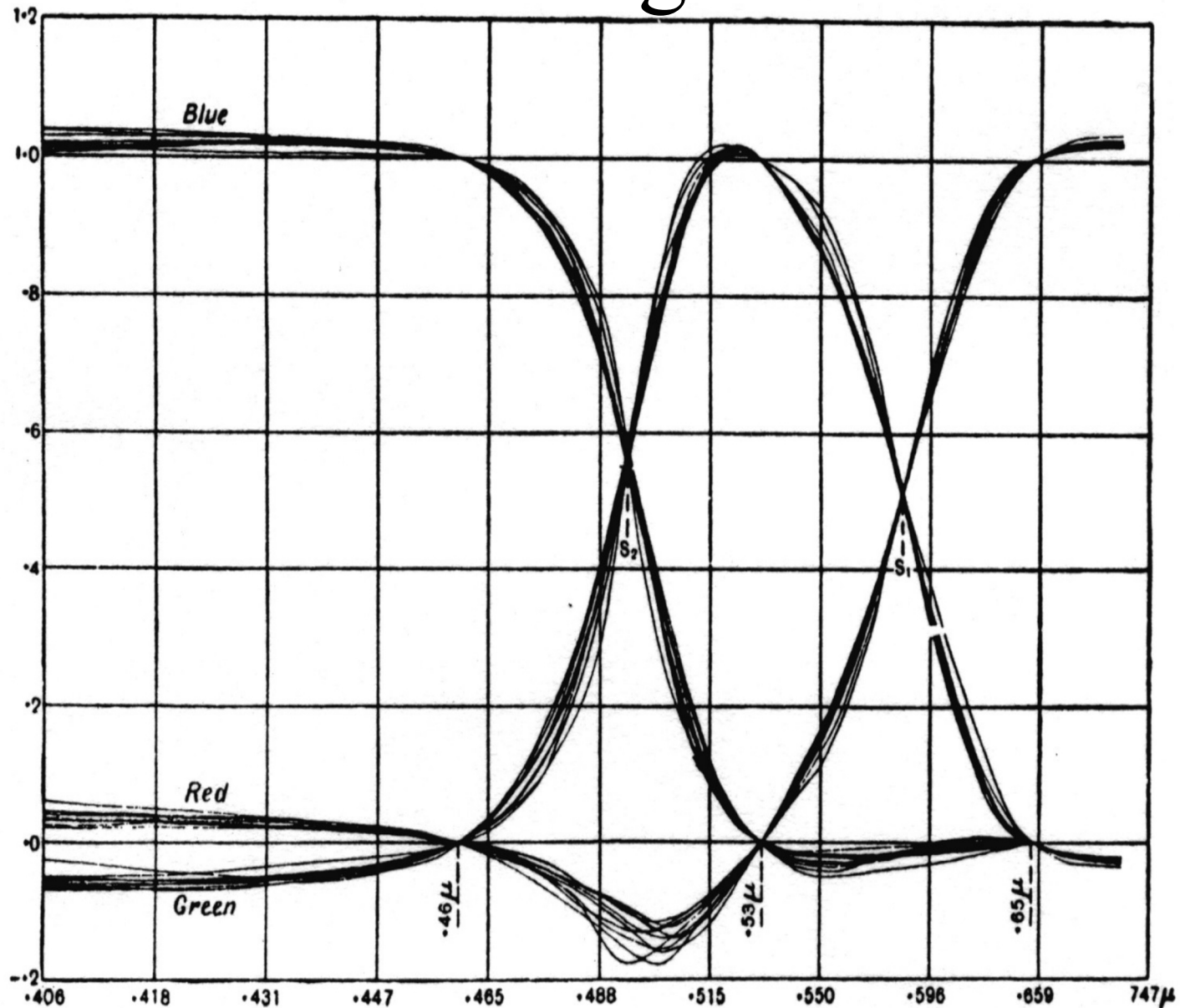
We say a “negative” amount of p_2 was needed to make the match, because we added it to the test color’s side.



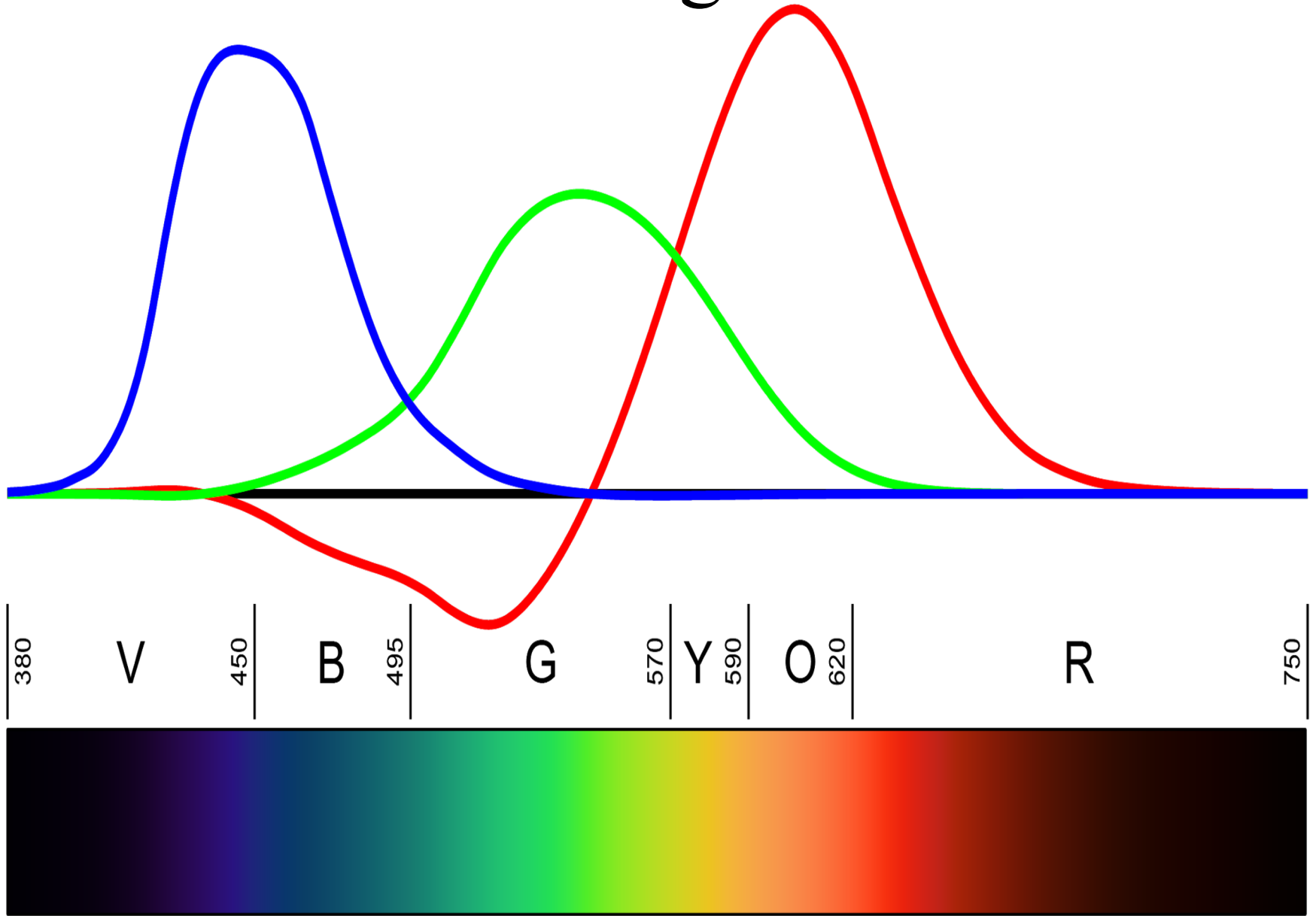
The primary color amounts needed for a match:



RGB matching functions



RGB matching functions

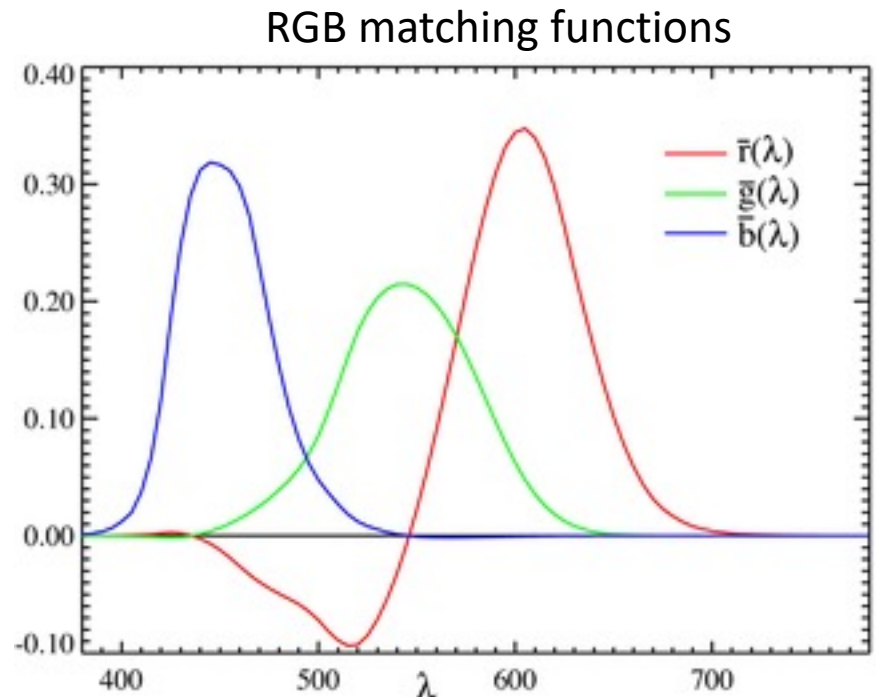
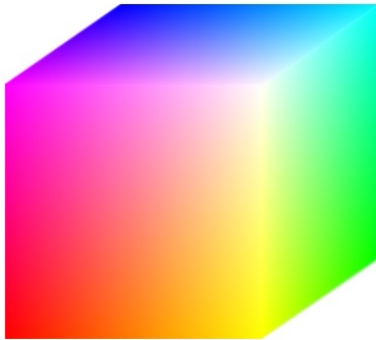


Linear color spaces

- Choose 3 primaries
- Any color = linear combination of these primaries
→ Color = weights of the primaries
- Matching functions:
for each wavelength → how much of each primary is needed

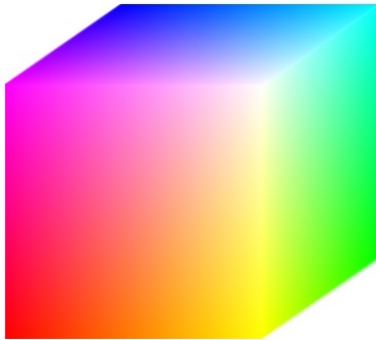
Linear color spaces - RGB

- Choose 3 primaries (R, G, B)
- Any color = linear combination of these primaries
→ Color = weights of the primaries
- Matching functions:
for each wavelength → how much of each primary is needed

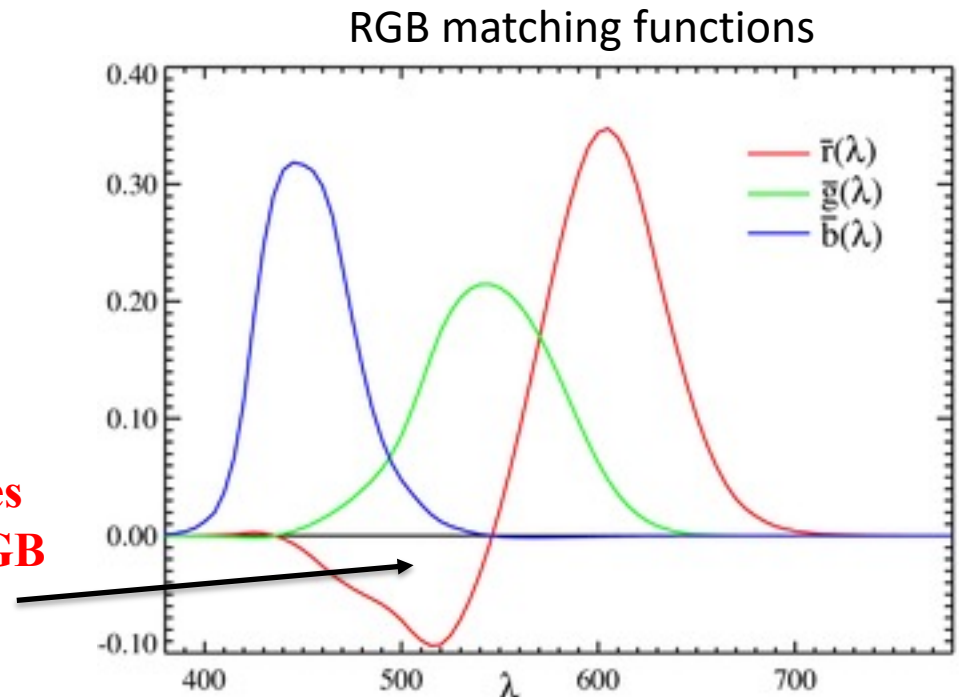


Linear color spaces - RGB

- Choose 3 primaries (R, G, B)
- Any color = linear combination of these primaries
→ Color = weights of the primaries
- Matching functions:
for each wavelength → how much of each primary is needed



**Some colors require negative RGB values
→ These cannot be displayed on an RGB monitor (we cannot subtract light)**



CIE XYZ color space

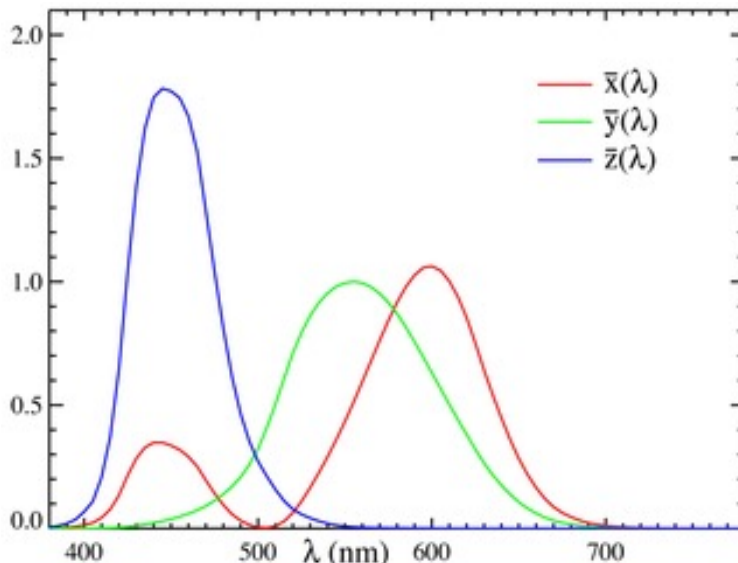
Problem: RGB matching functions can be negative

Solution: Redefine the color space (XYZ)

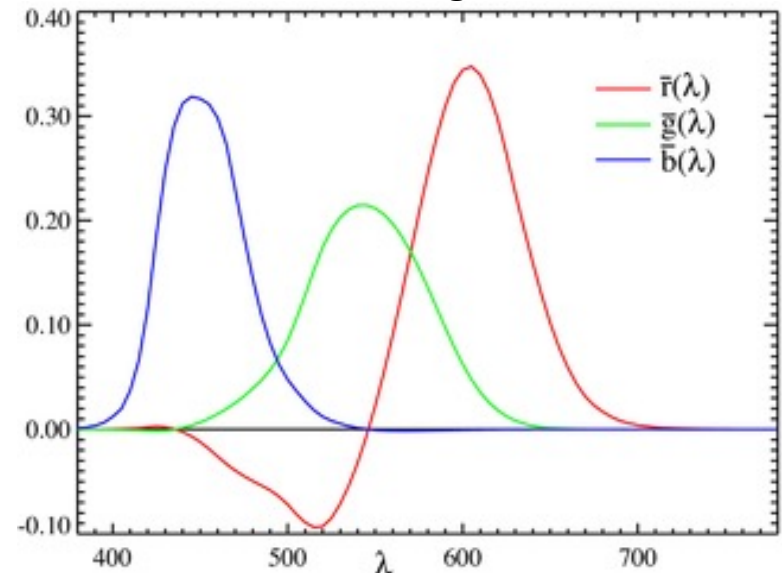
Properties:

- No negative values
 - Y corresponds to brightness, X, Z $\rightarrow$ encode chromatic information
- XYZ is a mathematical basis for representing all colors

XYZ matching functions



RGB matching functions



The xy Chromaticity Diagram

Color = brightness + chromaticity

- Chromaticity = color independent of brightness

- Same chromaticity:
→ white, grey, black (different brightness)

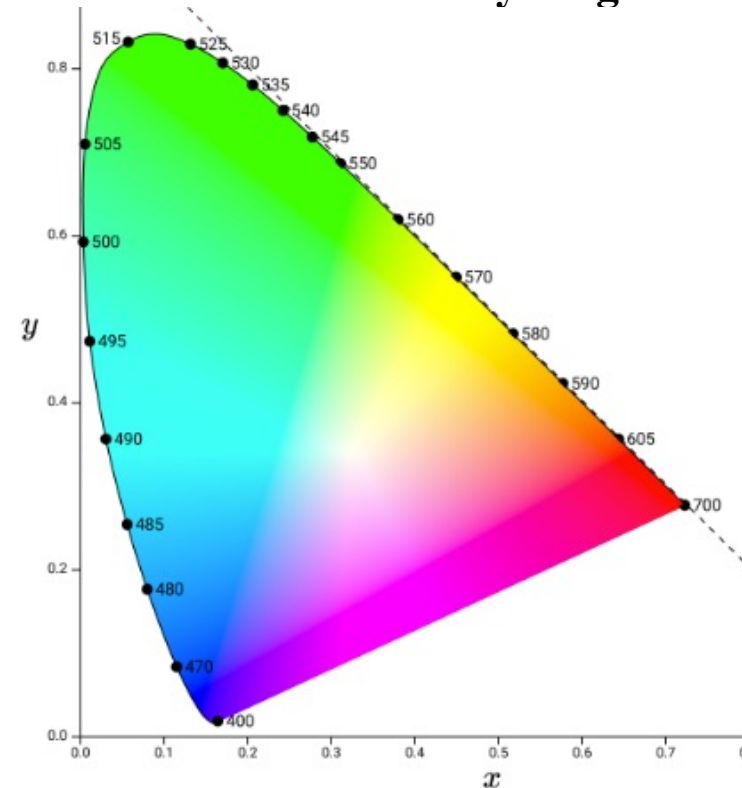
- Reduce from 3D → 2D (remove brightness)

- $x = X / (X + Y + Z)$

- $y = Y / (X + Y + Z)$

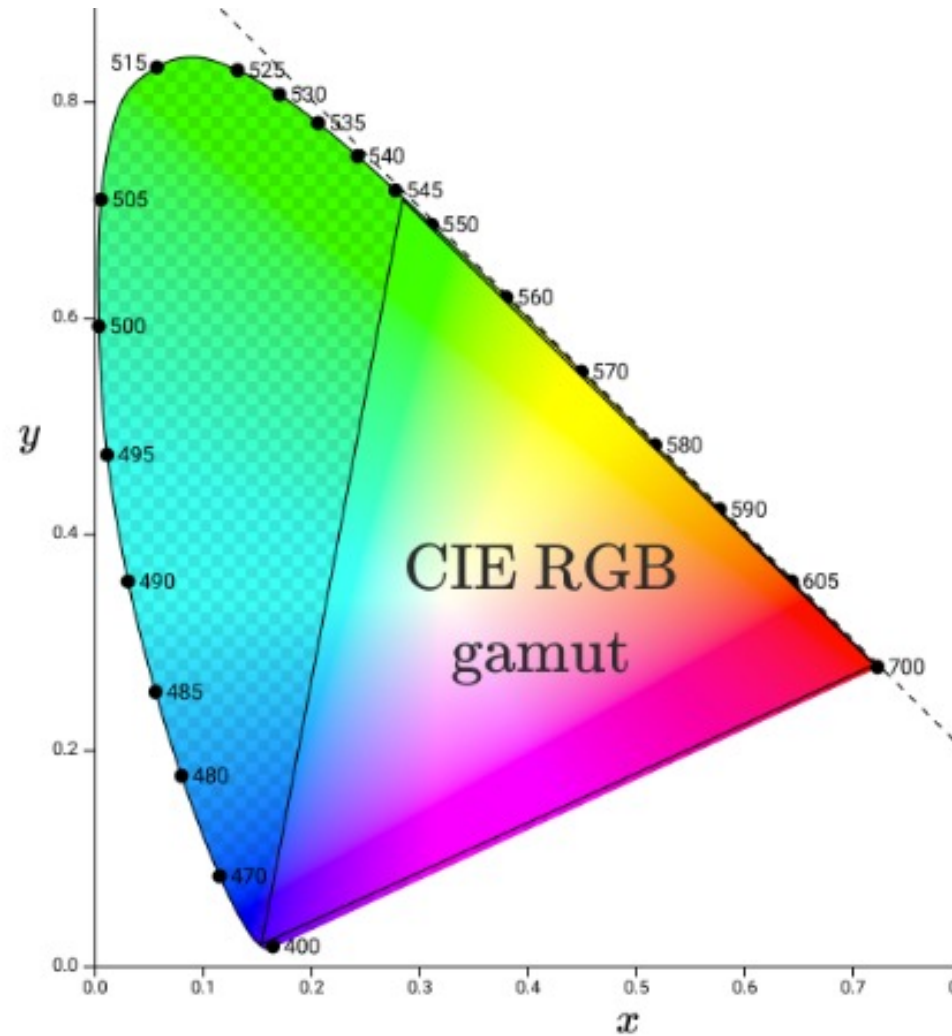
→ Chromaticity = color robust to intensity

Chromaticity diagram



The RGB Gamut (Limits of RGB)

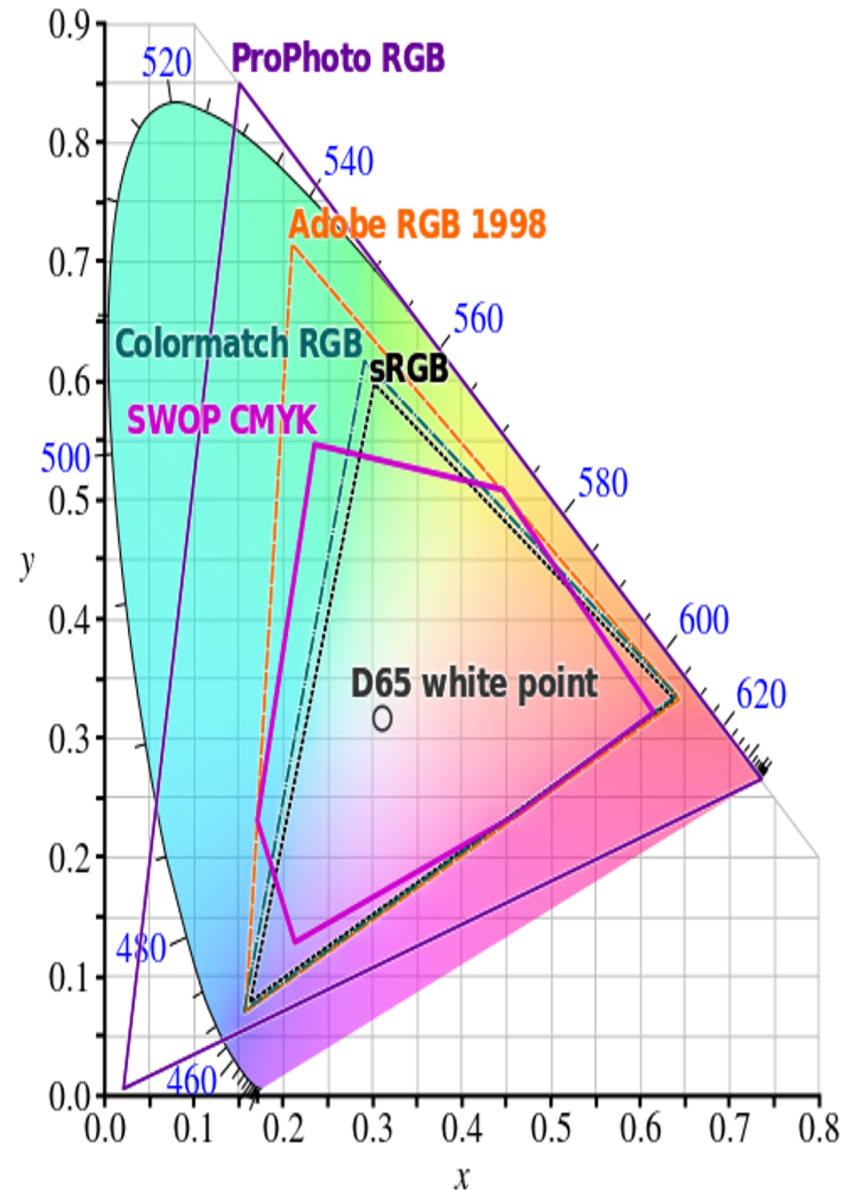
- The chromaticity diagram shows all visible colors
 - The triangle = colors representable with RGB
 - Colors inside → can be represented
 - Colors outside → cannot be represented
- RGB covers only a subset of visible colors
- Some real colors cannot be reproduced accurately on screens



Different Color Spaces = Different Gamuts

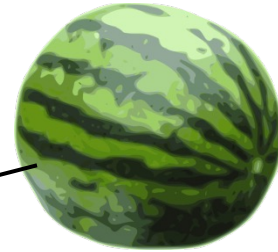
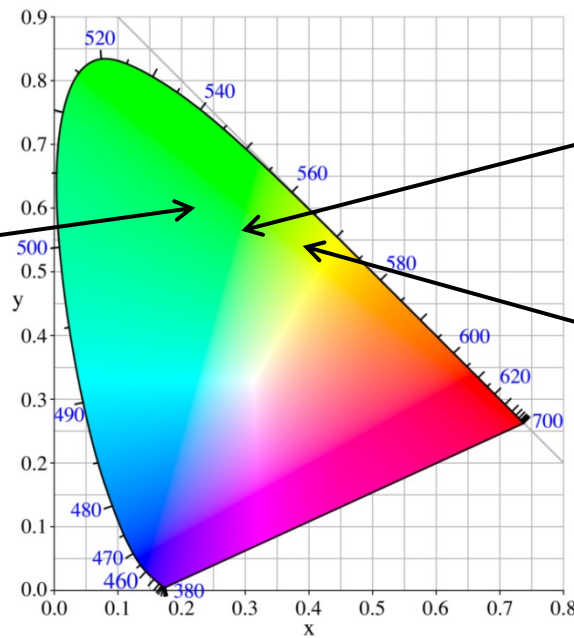
- Different color spaces = different gamuts
- Larger gamut → more colors
- Smaller gamut → fewer colors
- sRGB (small, standard)
- Adobe RGB (larger)
- ProPhoto RGB (very large)
- CMYK (subtractive color model used in printing)

→ Choice of color space matters



Are distances in color space meaningful?

- Do equal distances in a color space correspond to equal perceived differences?

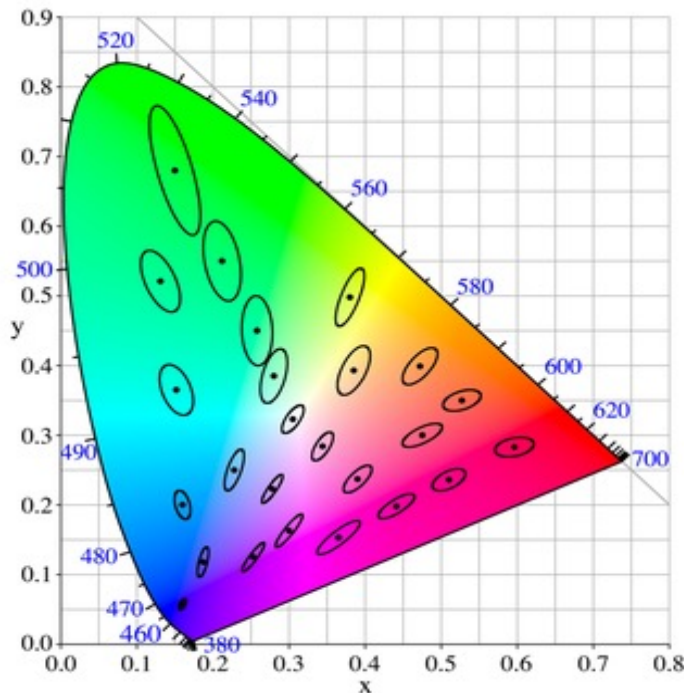


- Not necessarily: CIE XYZ is **not** a *uniform* color space, so magnitude of differences in coordinates are poor indicator of color “distance”.

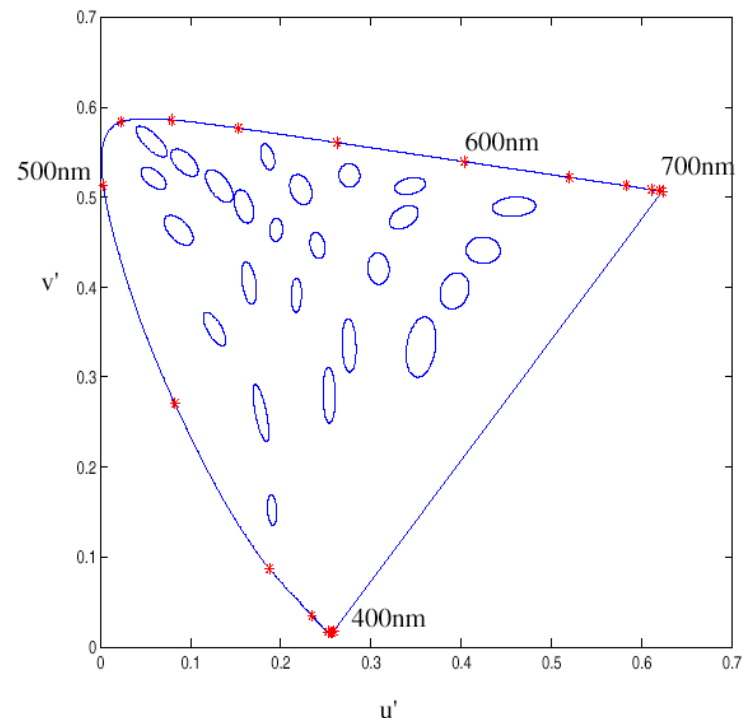
→ Euclidean distance $\neq$ perceptual difference

Uniform color spaces

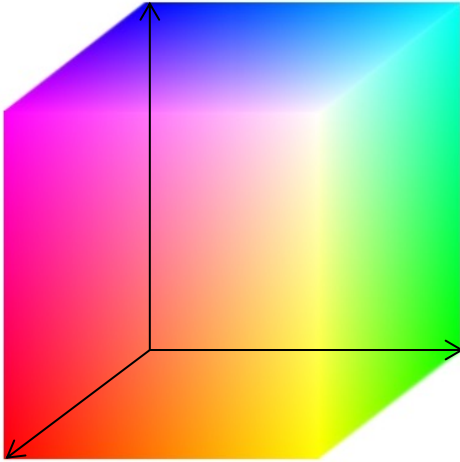
- Unfortunately, differences in x , y coordinates do not reflect perceptual color differences
- CIE $u'v'$ is a projective transform of x,y to make the ellipses more uniform (another color space is CIELAB)



McAdam ellipses: Just noticeable differences in color

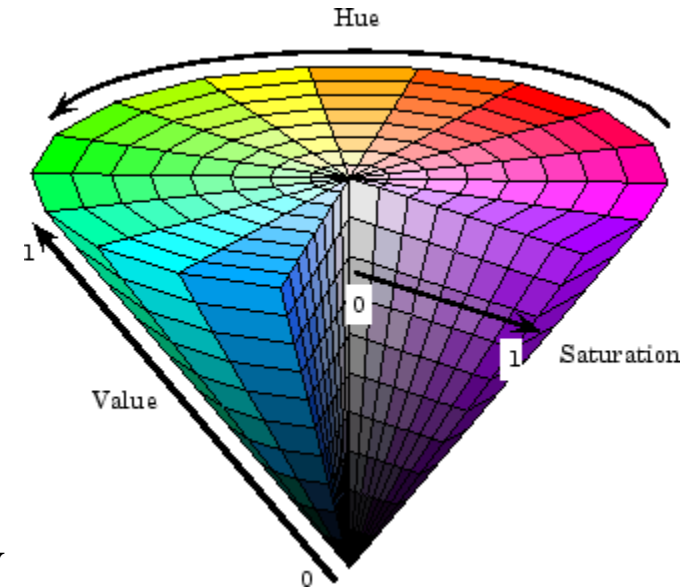


RGB vs HSV



RGB

- Additive model (used by displays)
- Not intuitive:
 - mixes color + brightness
 - hard to isolate “just color”
- Good for image representation
- Not ideal for analysis



HSV

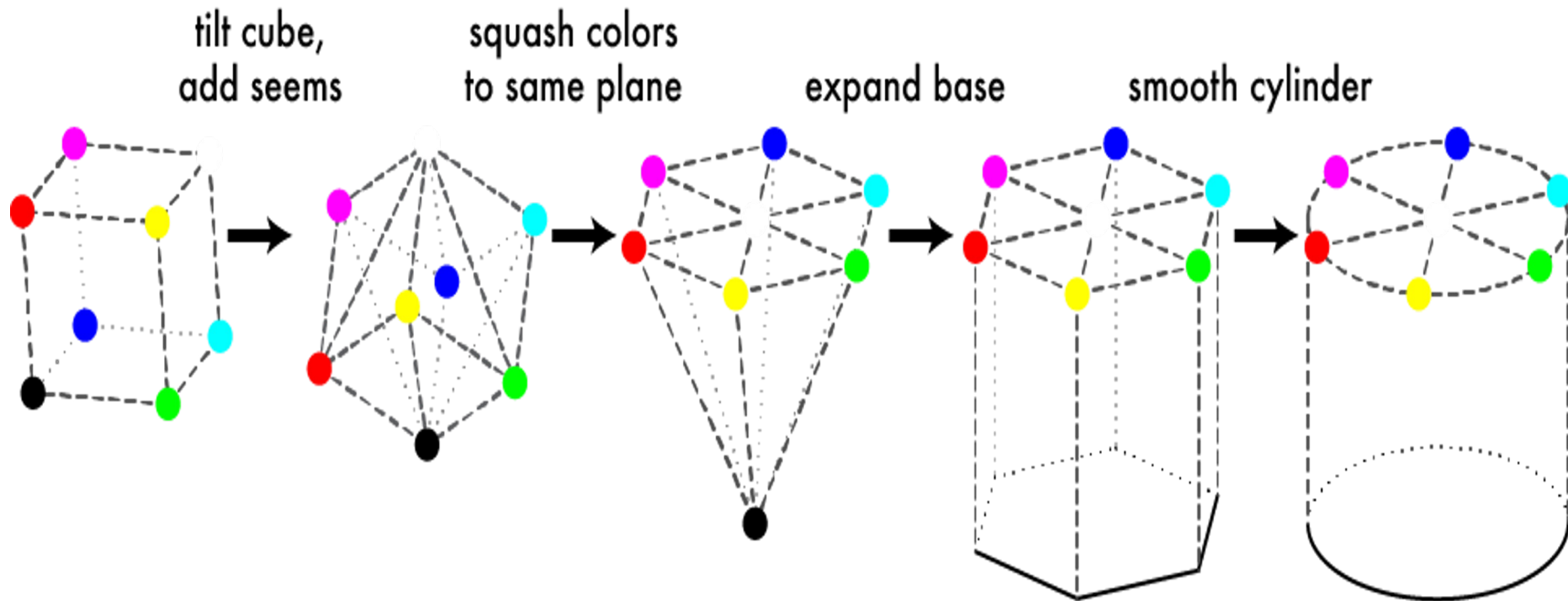
- More intuitive (human perception)
- Separates components:
 - Hue = color (red, green, blue)
 - Saturation = purity
 - Value = brightness
- Easier to manipulate and analyze colors

Demo: <https://colorizer.org/>

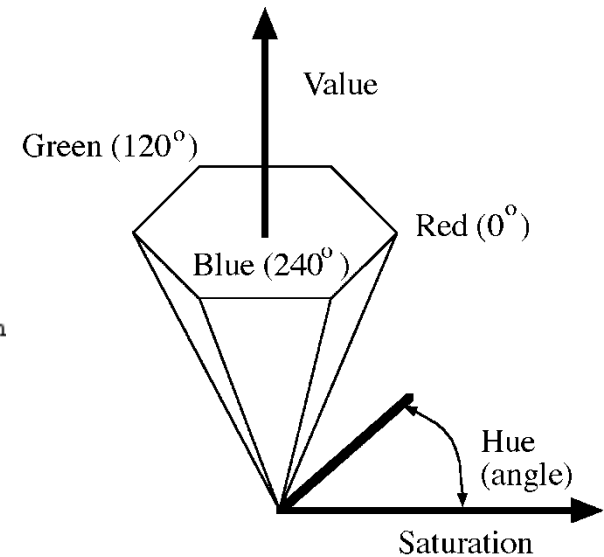
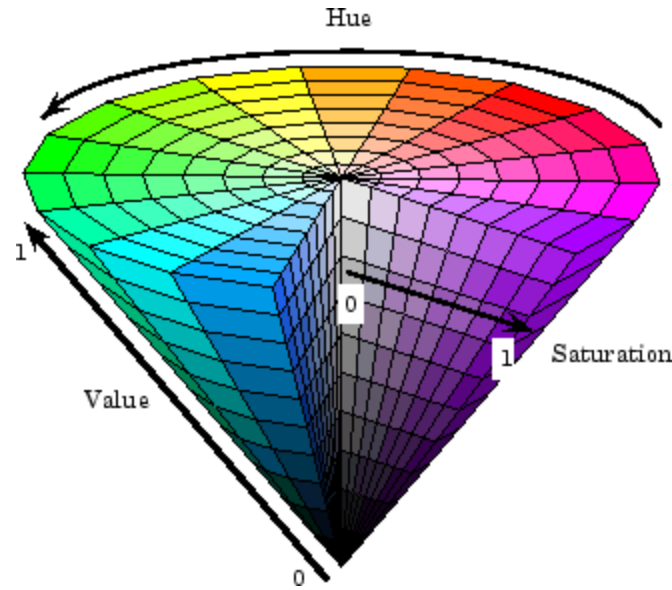
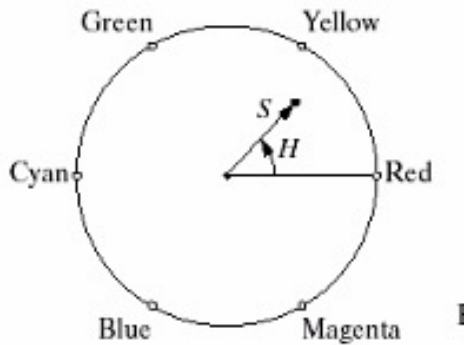
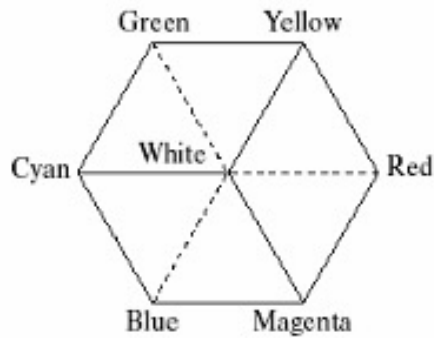
From RGB cube to HSV space

HSV = geometric transformation of RGB

- Value $\rightarrow$ brightness (vertical axis)
 - Hue $\rightarrow$ angle (color)
 - Saturation $\rightarrow$ distance from center
- $\rightarrow$ reorganizes RGB into perceptual components

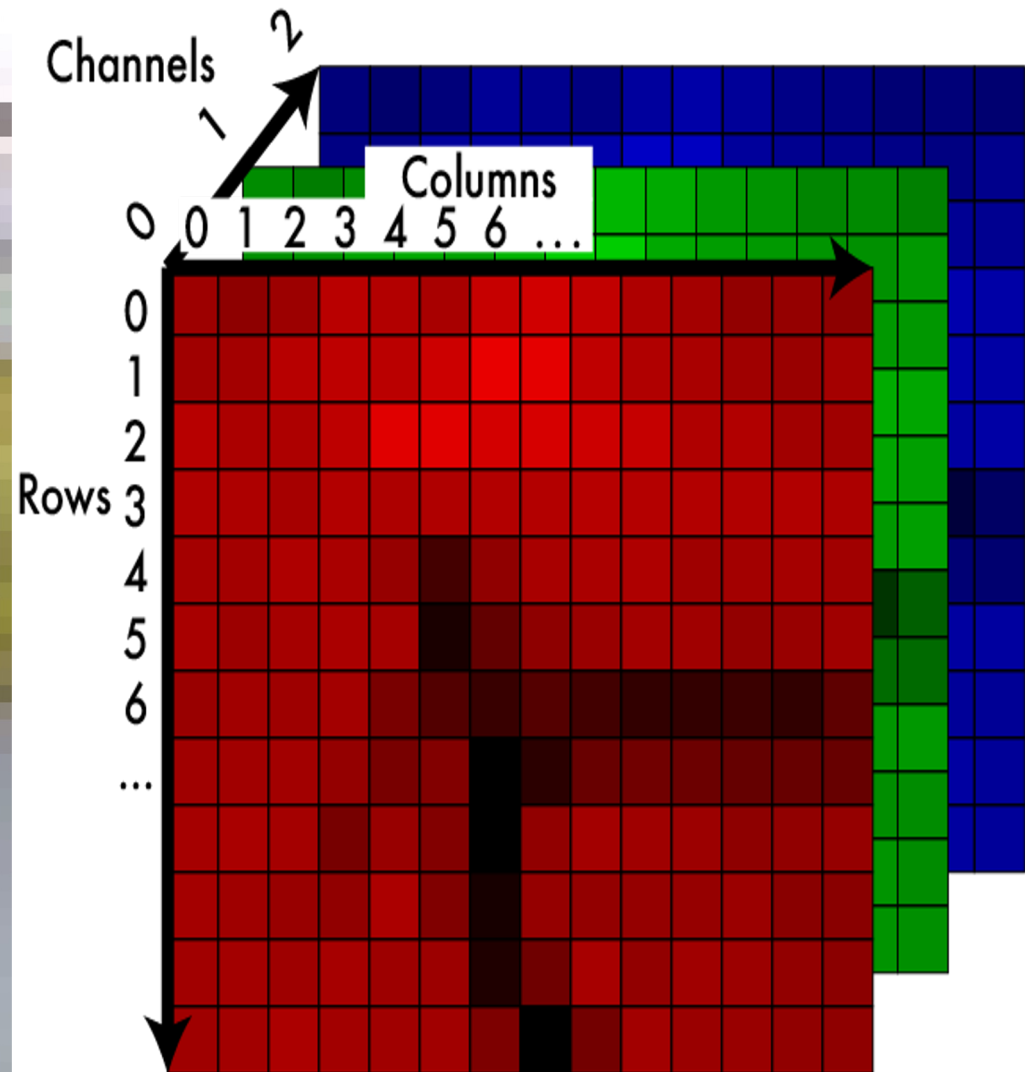
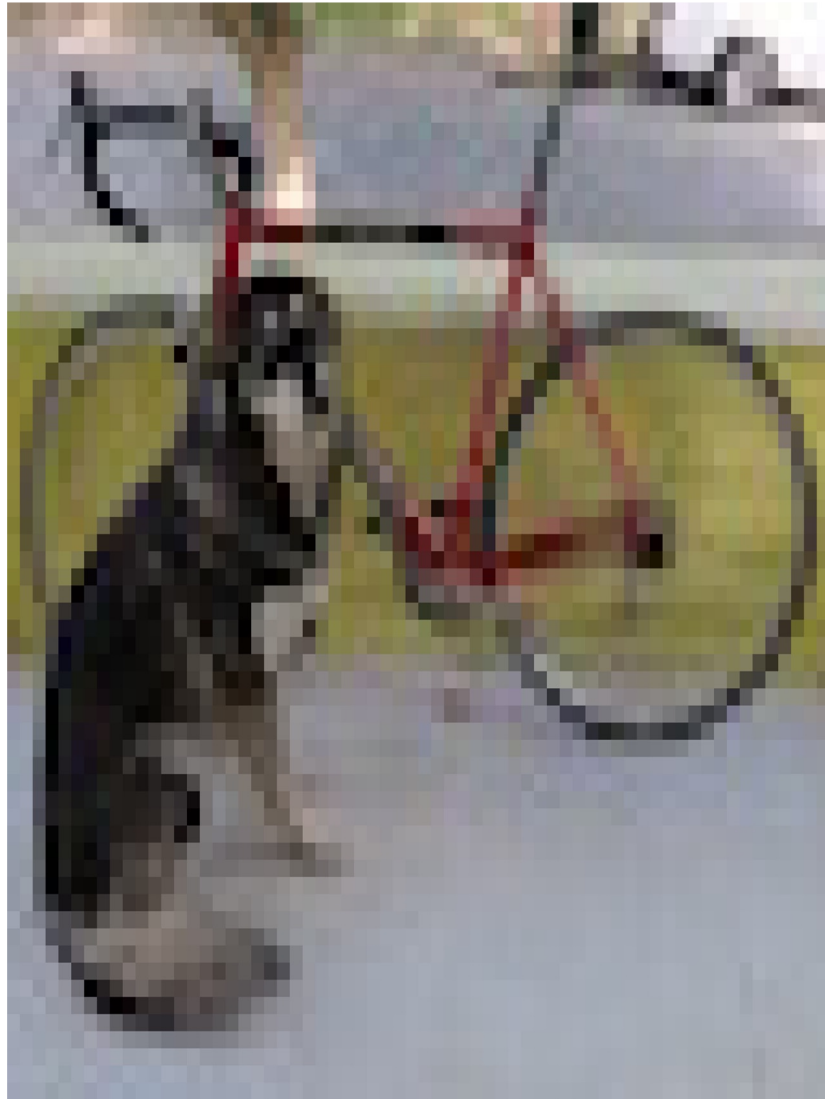


Nonlinear color spaces: HSV

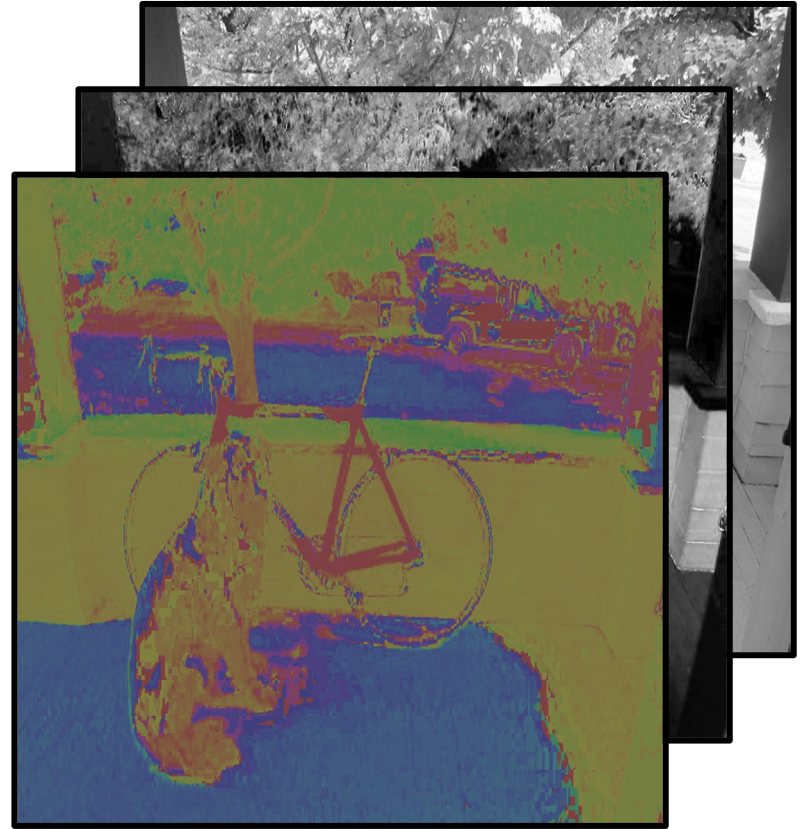
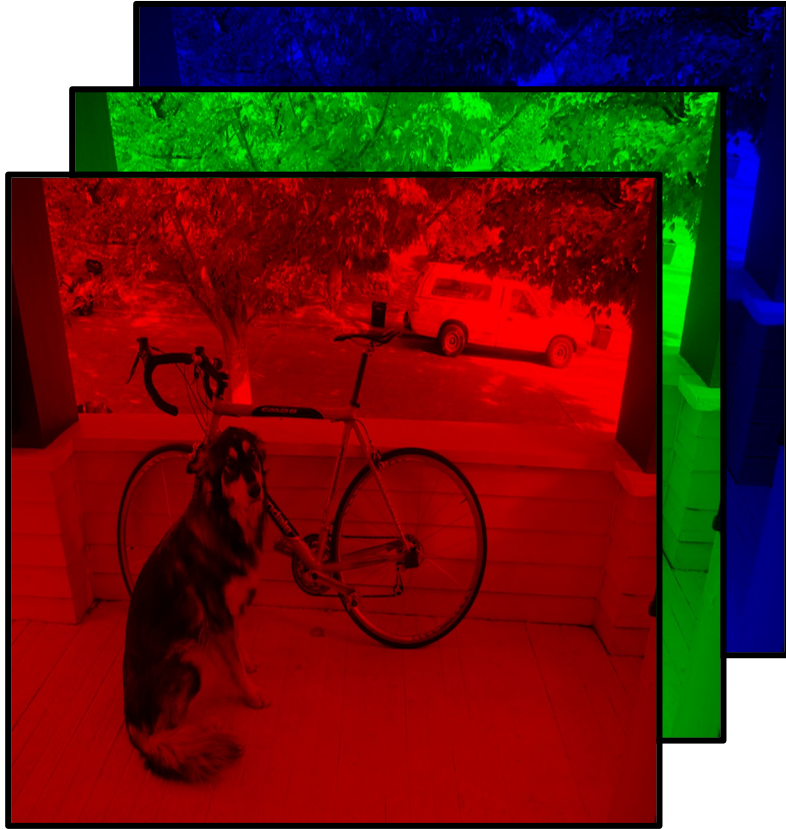


- Perceptually meaningful dimensions:
Hue, Saturation, Value (Intensity)

RGB Color image: 3d tensor in colorspace

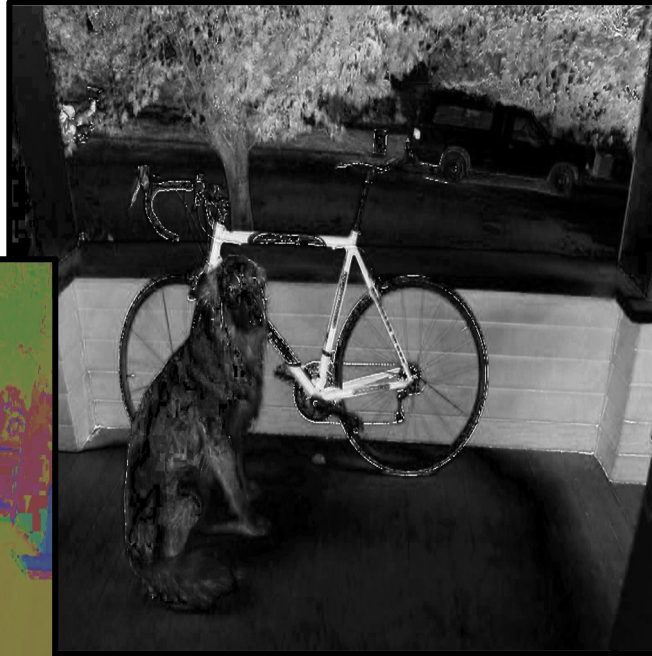
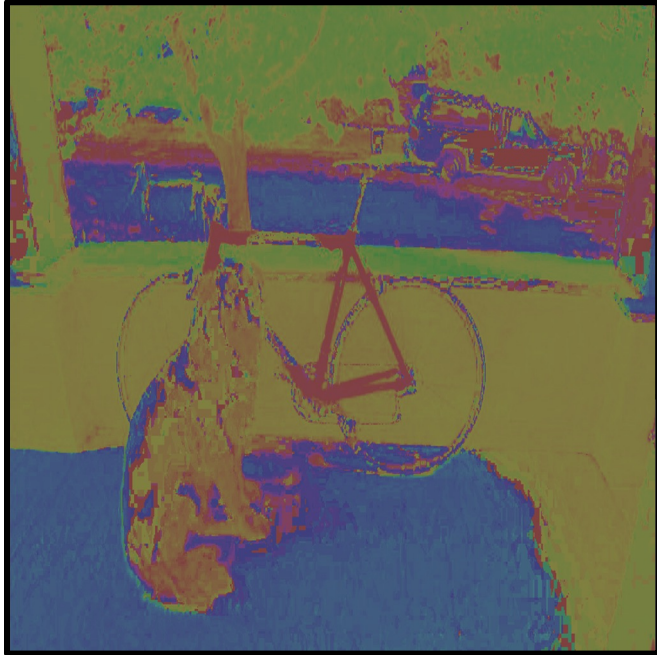


HSV Color image: 3d tensor, different info



Saturation

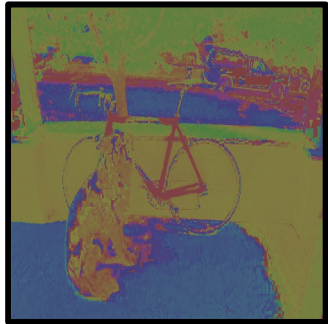
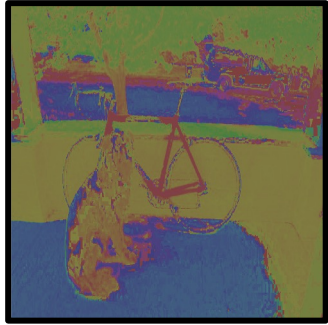
Hue



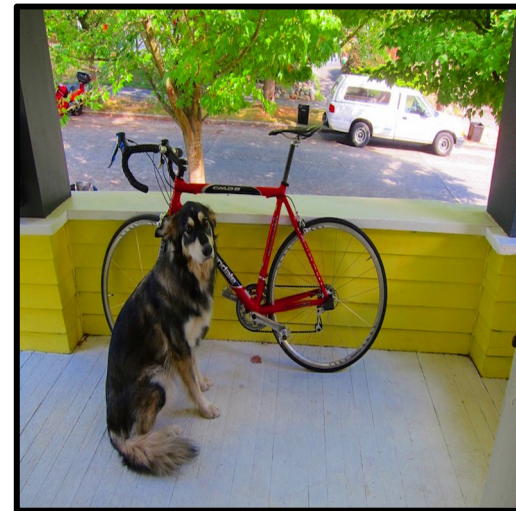
Value



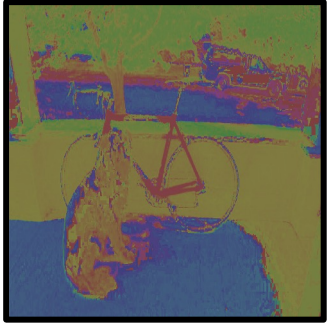
More saturation = intense colors



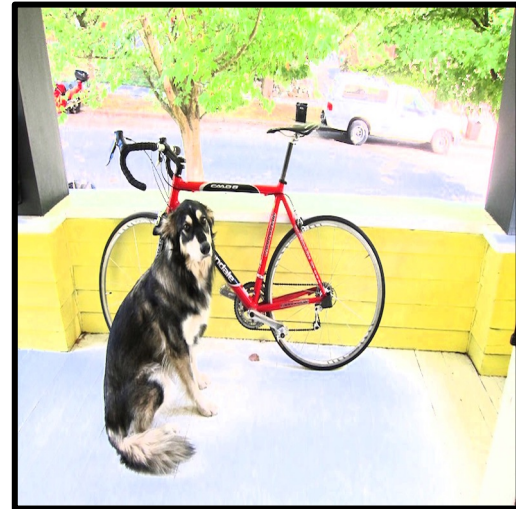
2x



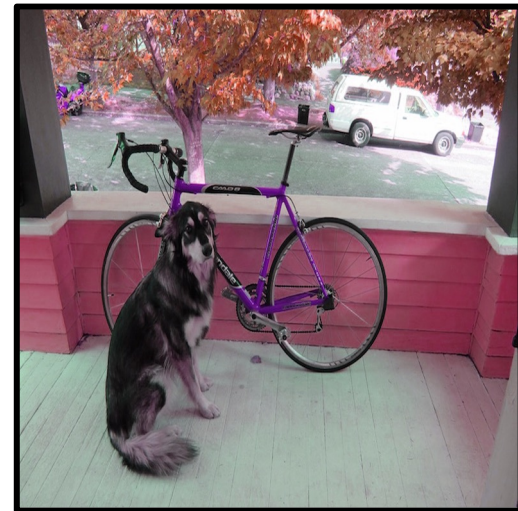
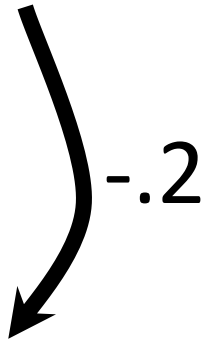
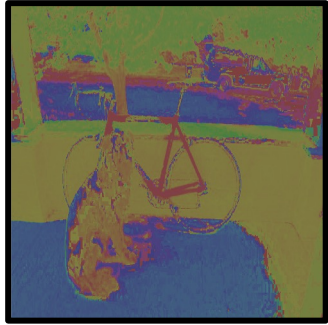
More value = lighter image



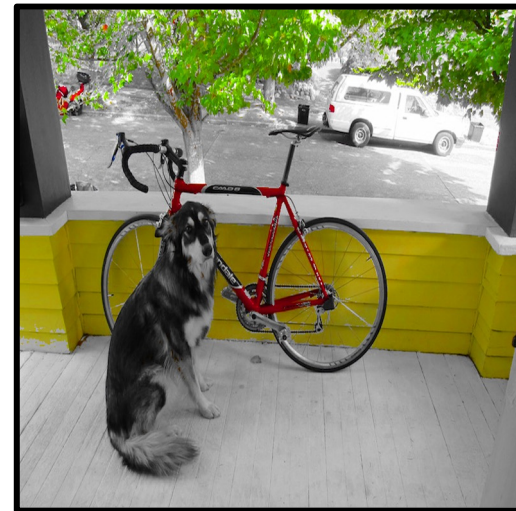
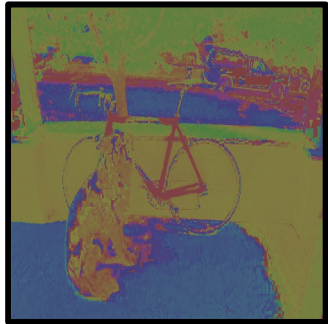
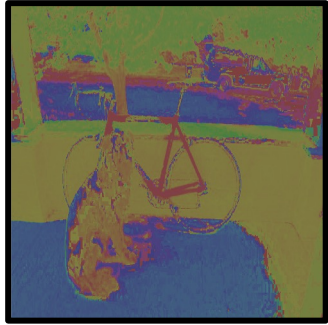
2x



Shift hue = shift colors



Increase and threshold saturation





Course structure

1. Low-level vision: Features and filters

Linear filters, color, texture, edge detection, template matching

2. Mid-level vision: Grouping and fitting

Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. Mid-level vision: Multiple views

Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. High-level vision: Object recognition

Object classification, object detection, part based models, bovw models

5. High-level vision: Video understanding

Object tracking, background subtraction, motion descriptors, optical flow

Local features

A local feature = keypoint (location + scale) + descriptor
(local appearance)

Good features: distinctive, repeatable, invariant to geometry and illumination

→ used for matching and recognition



Local features

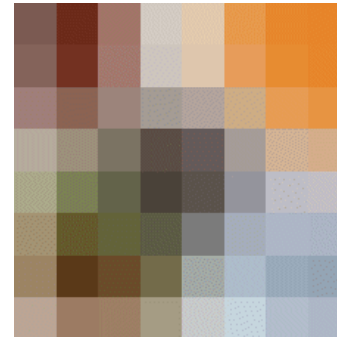
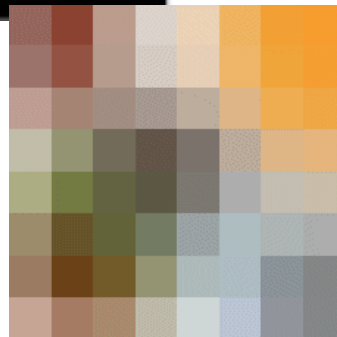
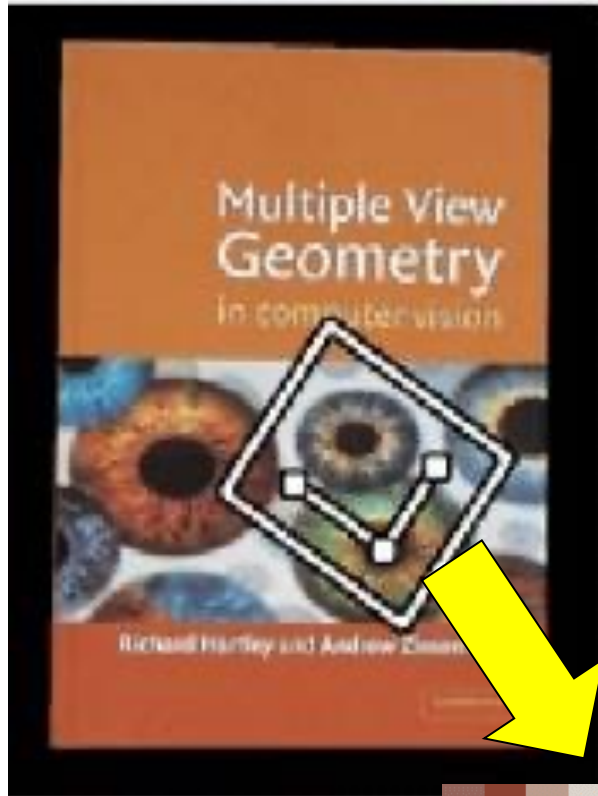
A local feature = keypoint (location + scale) + descriptor
(local appearance)

Good features: distinctive, repeatable, invariant to geometry and illumination

→ used for matching and recognition

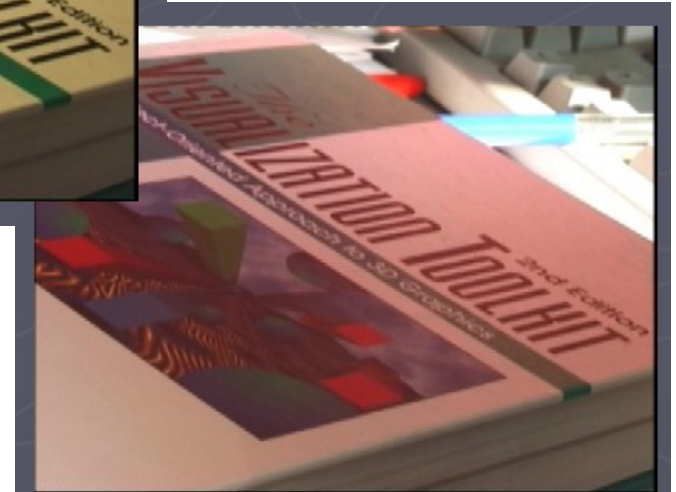
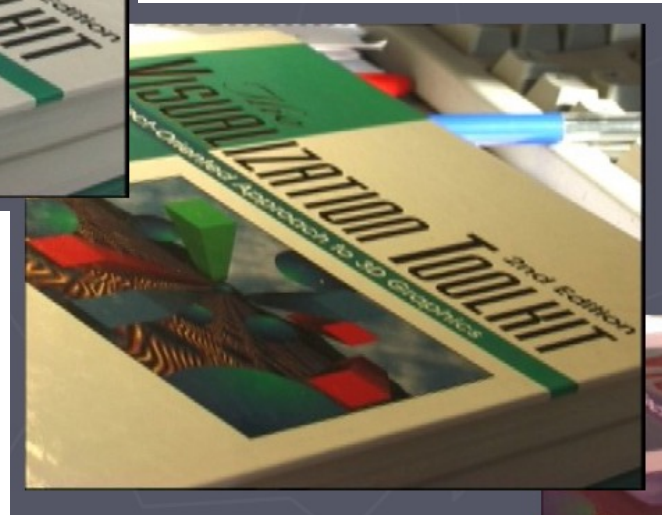
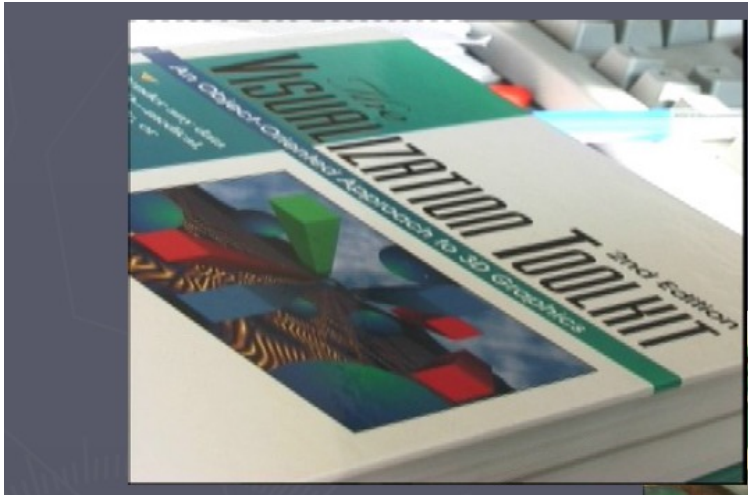


Geometric transformations



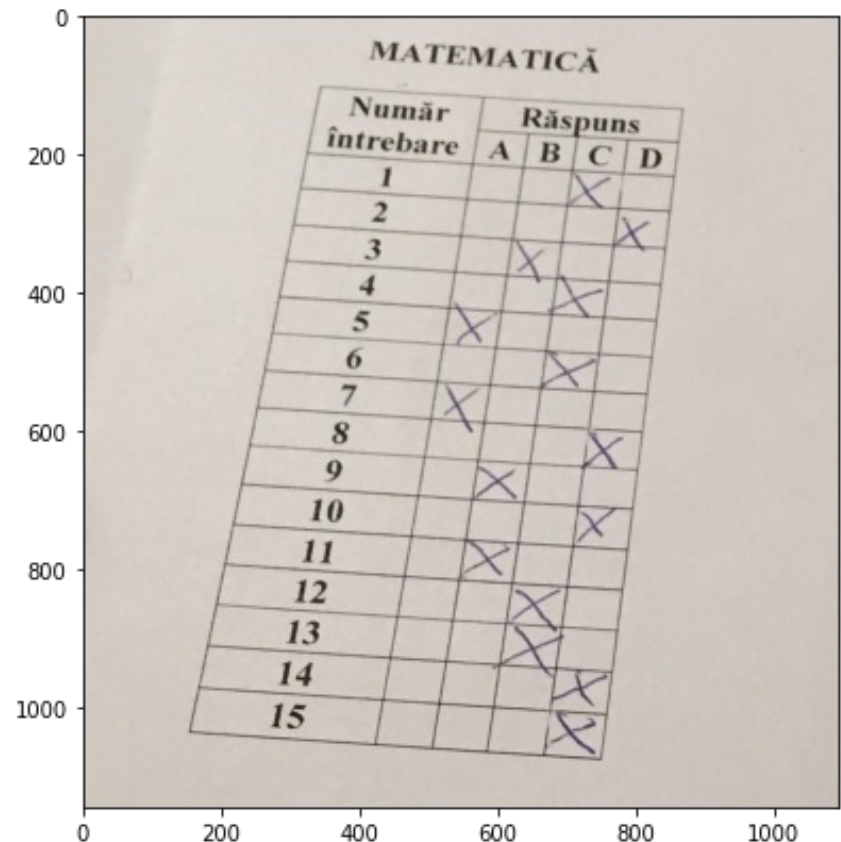
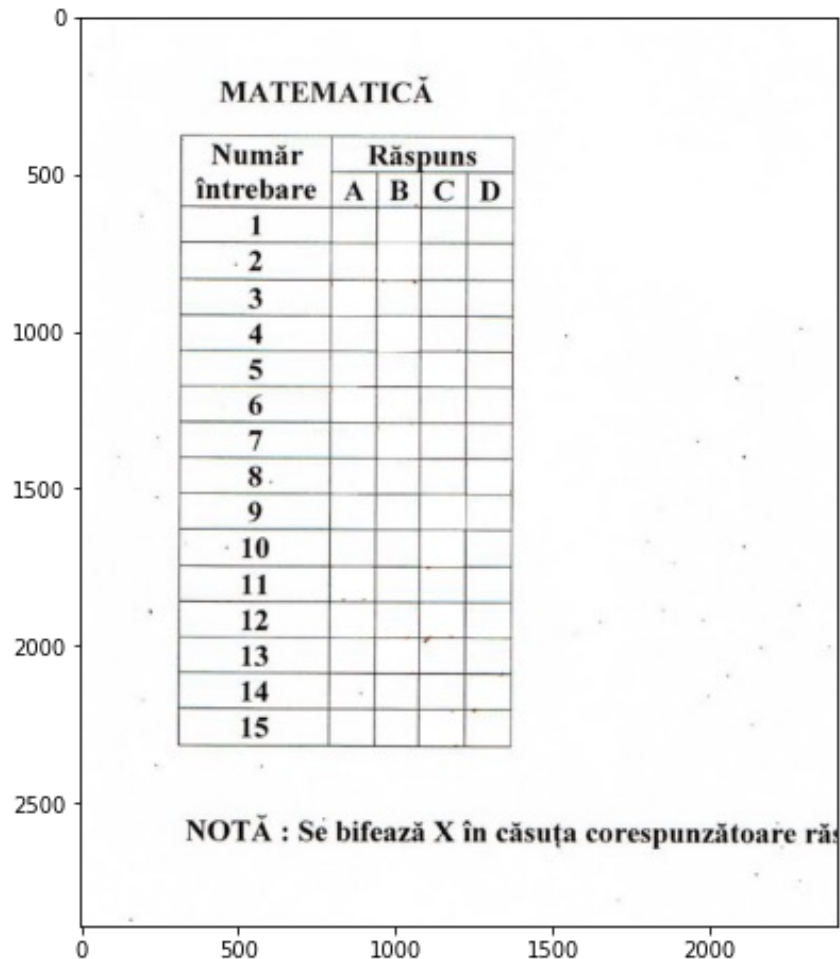
Exemples:
translation,
rotation,
scaling

Photometric transformations



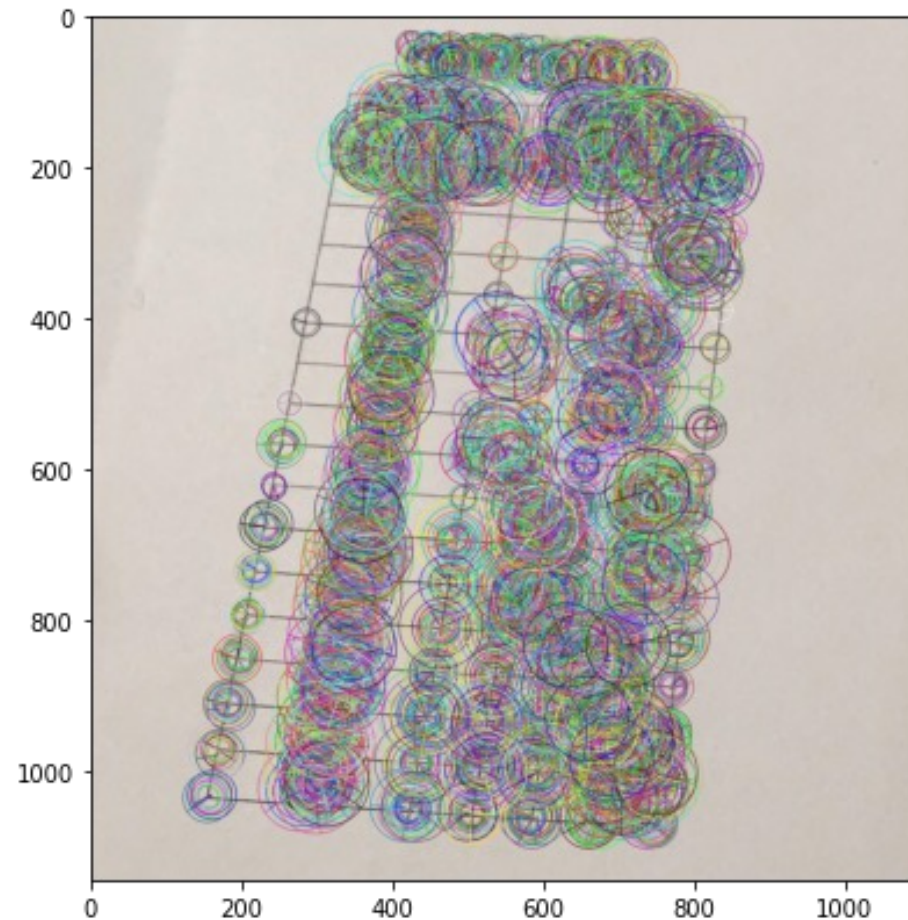
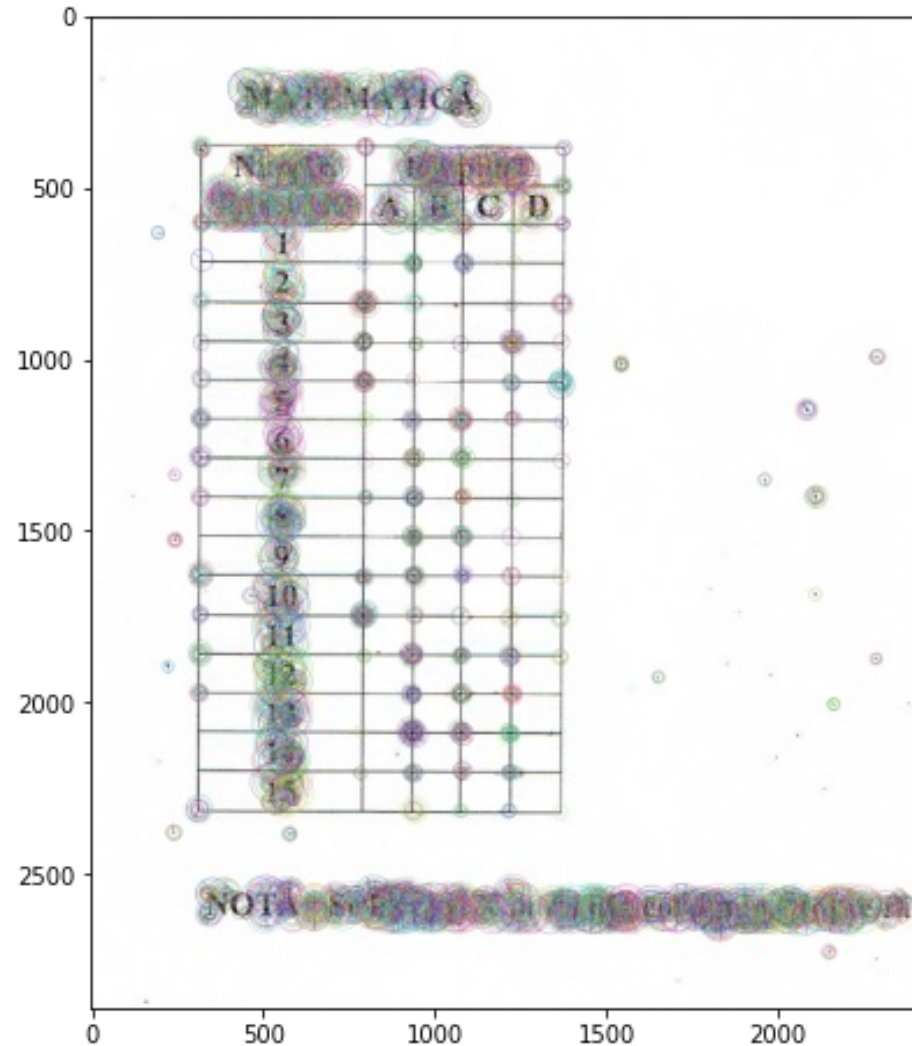
Matching local features – past P1

- matching local image feature = find correspondence points between images based on local features that look the same



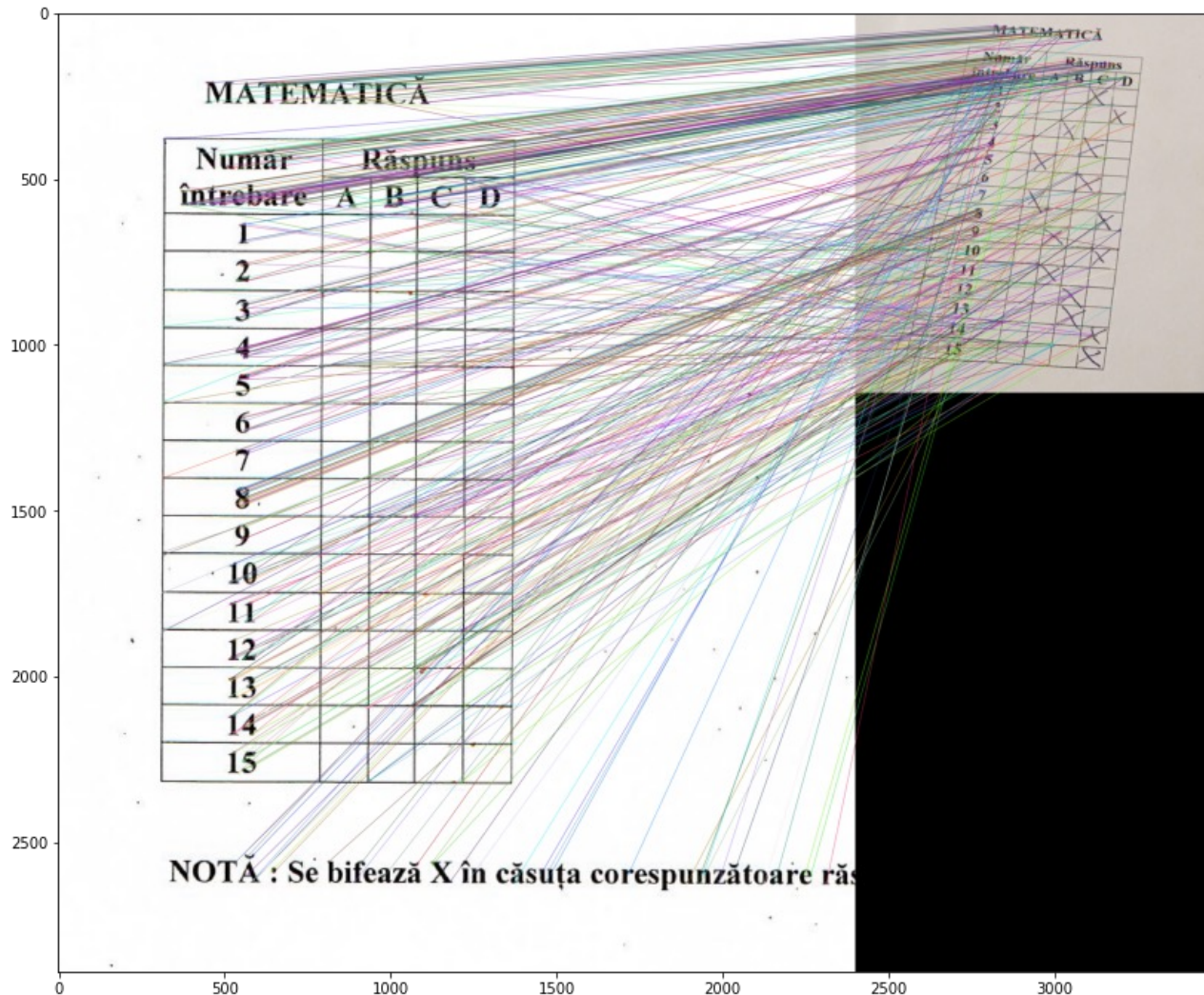
Matching local features – past P1

- matching local image feature = find correspondence points between images based on local features that look the same



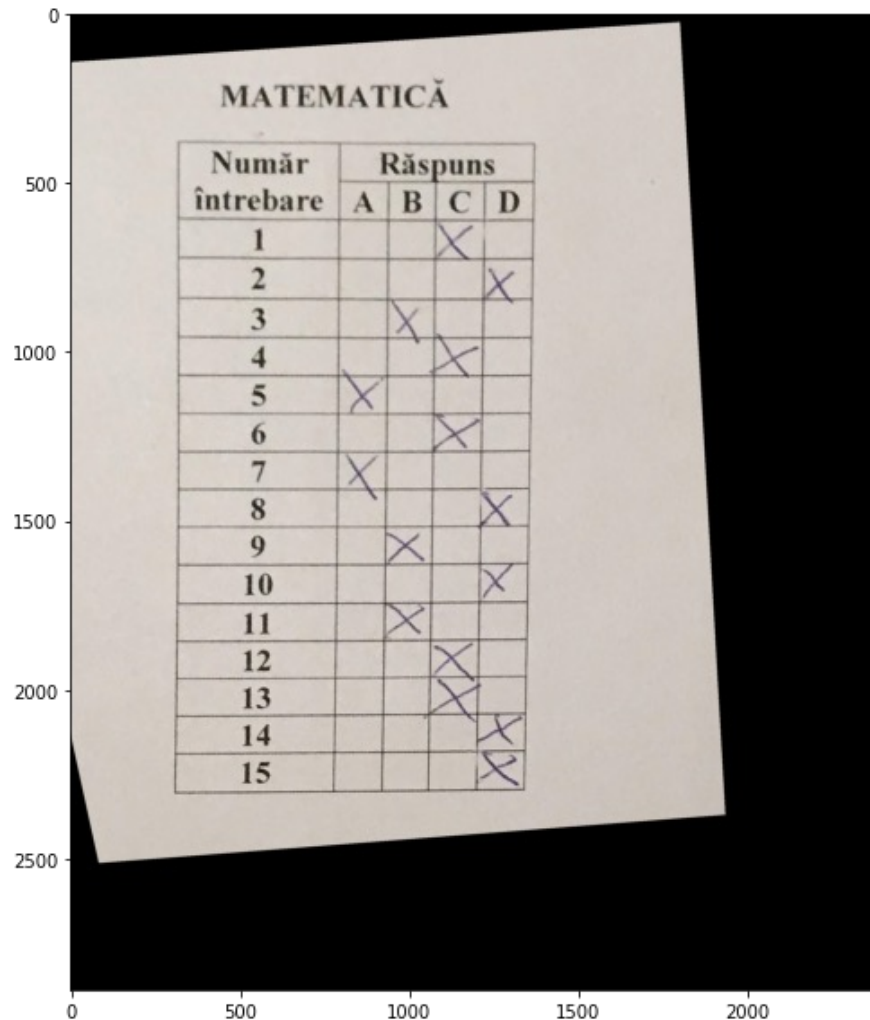
Matching local features – past P1

- matching local image feature = find correspondence points between images based on local features that look the same



Finding the homography transformation

- Find the matrix of the homography (perspective) transformation and use it to warp one image



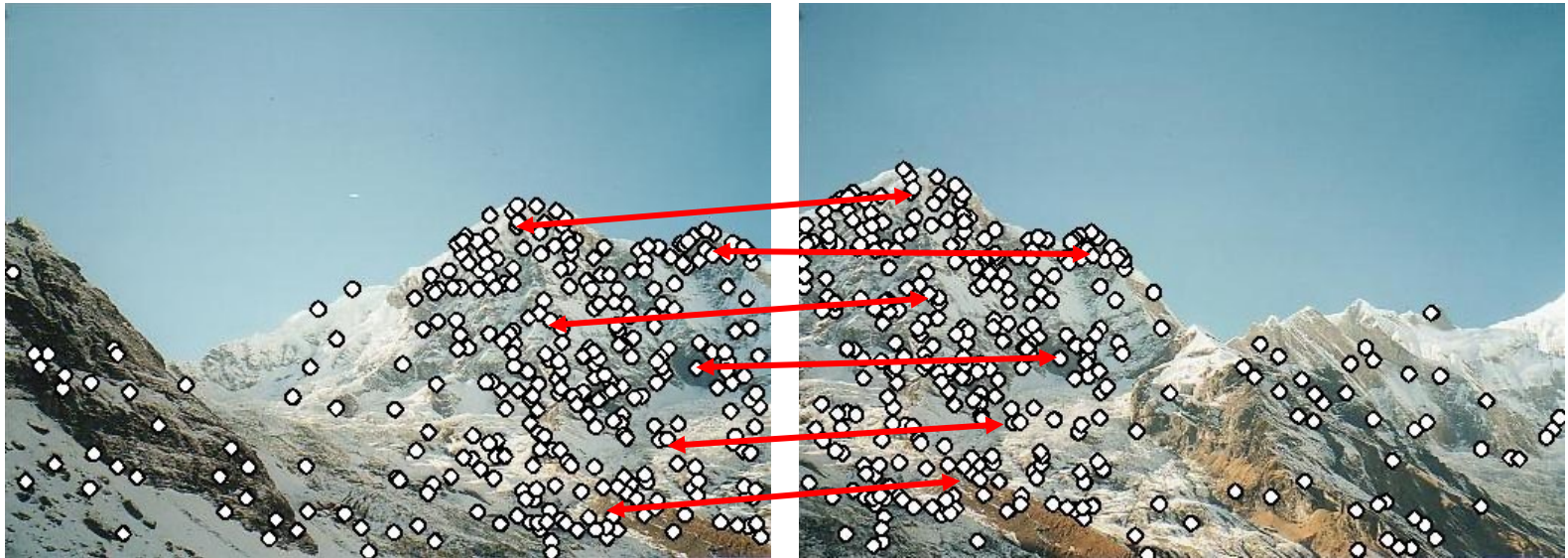
Why extract local features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Why extract local features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract local features

Step 2: match local features

Why extract local features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract local features

Step 2: match local features

Step 3: align images